

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

AI - Based Intrusion Detection System for IoT Networks Using Machine Learning and Deep Learning

Reg. Number	Name	Branch
261100610001	B. Nitin Sri Karthikeya	CSE
261100610017	E. Dhanush Rajkumar	CSE
261100610019	M.B. Usha	CSE

Under the guidance of

Raghudathesh G P
Assistant Professor - Senior Scale,
Manipal School of Information Sciences,
MAHE, MANIPAL

27/10/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

TABLE OF CONTENTS

S.No	Title	Page No
1	**INTRODUCTION**	1
	1.1. Background of the Study	1
	1.2. Problem Statement	3
	1.3. Motivation	4
	1.4. Proposed Solution	5
2	**OBJECTIVES**	7
3	**LITERATURE SURVEY**	8
	3.1. Introduction	8
	3.2. Summary of Existing Research Papers	9
	3.3. Selected Research Paper	11
	3.4. Research Gap	14
4	**SPECIFICATIONS**	16
	4.1. Hardware Specifications	16
	4.2. Software Specifications	16
	4.3. Programming Languages and Libraries	17
	4.4. Dataset Specification	18
	4.5. Performance Evaluation Metrics	18
5	**ARCHITECTURE**	19
	5.1. System Overview	19
	5.2. System Architecture	21
	5.3. System Modules	24
	5.3.1. Dataset Collection	24

	5.3.2. Data Preprocessing	25
	5.3.3. Exploratory Data Analysis (EDA)	26
	5.3.4. Feature Selection using Pearson Correlation Coefficient (PCC)	27
	5.3.5. Feature Selection Using Standard Scaler	29
	5.3.6. Class Balancing using Synthetic Minority Oversampling Technique	30
	5.3.7. Binary Classification Models	33
	5.3.8. Multiclass Classification Models	35
	5.3.9. Performance Evaluation	37
6	**WORK DONE**	39
	6.1. Implementation Workflow	39
	6.2. Dataset Collection	39
	6.3. Dataset Merging	40
	6.4. Dataset Sampling	41
	6.5. Data Cleaning	42
	6.6. Exploratory Data Analysis (EDA)	43
	6.6.1. Dataset Overview	44
	6.6.2. Statistical Summary	44
	6.6.3. Class Distribution Analysis	45
	6.6.4. Correlation Analysis	45
	6.6.5. Observations from EDA	46
	6.7. Binary Classification	47

	6.8. Pearson Correlation Coefficient (PCC) Feature Selection	48
	6.8.1. PCC Implementation	49
	6.8.2. Feature Ranking	49
	6.8.3. Selected Features	50
	6.8.4. Observations	50
	6.9. Binary Class Implementation	50
	6.9.1. Decision Tree	50
	6.9.2. Random Forest	52
	6.9.3. Artificial Neural Network (ANN)	53
	6.9.4. Convolutional Neural Network (CNN)	56
	6.10. Multiclass Classification Implementation	58
	6.10.1. Decision Tree	59
	6.10.2. Random Forest	60
	6.10.3. Artificial Neural Network (ANN)	60
	6.10.4. Convolutional Neural Network (CNN)	61
	6.10.5. XGBoost	61
	6.10.6. Ensemble Learning	62
7	**RESULTS AND DISCUSSION**	63
	7.1. CNN	63
	7.1.1. Architecture	63
	7.1.2. Hyperparameter Configuration	63
	7.1.3. Training Performance	64
	7.1.4. Cross Validation Results	65

	7.1.5. Quantitative Performance	65
	7.1.6. ROC Analysis	66
	7.1.7. Confusion Matrix	67
	7.1.8. Average Metric Comparison	67
	7.1.9. t-SNE	68
	7.1.10. Discussion	68
	7.2. ANN	69
	7.2.1. Architecture	69
	7.2.2. Hyperparameter Configuration	69
	7.2.3. Training Accuracy	69
	7.2.4. Training Loss	70
	7.2.5. Fold Accuracy	70
	7.2.6. Performance Metrics	71
	7.2.7. ROC Curve	71
	7.2.8. Confusion Matrix	72
	7.2.9. Metric Comparison	72
	7.2.10. Discussion	73
	7.3. Random Forest	73
	7.3.1. Hyperparameter	73
	7.3.2. Fold Accuracy	74
	7.3.3. Experiment Summary	74
	7.3.4. Average ROC	75
	7.3.5. Average Confusion Matrix	76

	7.3.6. Metric Comparison	77
	7.3.7. Discussion	77
	7.4. Decision Tree	78
	7.4.1. Hyperparameter Configuration	78
	7.4.2. Cross Validation Results	78
	7.4.3. Performance Metrics	79
	7.4.4. ROC Analysis	79
	7.4.5. Confusion Matrix	80
	7.5. Binary Classification Comparison	81
	7.5.1. Comparative Discussion	81
	7.6. Multiclass Classification	83
	7.6.1. Decision Tree	83
	7.6.1.1. Hyperparameter Configuration	83
	7.6.1.2. Cross Validation Results	83
	7.6.1.3. Performance Metrics	84
	7.6.1.4. Confusion Matrix	84
	7.6.2. Random Forest	85
	7.6.2.1. Hyperparameter Configuration	85
	7.6.2.2. Cross Validation Results	85
	7.6.2.3. Performance Metrics	86
	7.6.2.4. Confusion Matrix	86
	7.6.3. Artificial Neural Network (ANN)	87
	7.6.3.1. Hyperparameter Configuration	87

	7.6.3.2. Training Performance	87
	7.6.3.3. Cross Validation Results	88
	7.6.3.4. Performance Metrics	89
	7.6.3.5. Confusion Matrix	89
	7.6.4. Convolutional Neural Network (CNN)	90
	7.6.4.1. Hyperparameter Configuration	90
	7.6.4.2. Training Performance	90
	7.6.4.3. Cross Validation Results	91
	7.6.4.4. Performance Metrics	92
	7.6.4.5. Confusion Matrix	92
	7.6.5. XGBoost	93
	7.6.5.1. Hyperparameter Configuration	93
	7.6.5.2. Cross Validation Results	93
	7.6.5.3. Performance Metrics	94
	7.6.5.4. Confusion Matrix	94
	7.6.6. Ensemble Learning	95
	7.6.6.1. Hyperparameter Configuration	95
	7.6.6.2. Cross Validation Results	96
	7.6.6.3. Performance Metrics	96
	7.6.6.4. Confusion Matrix	97
	7.7. Multiclass Classification Comparison	97
	7.7.1. Performance Metrics Comparison	97
	7.7.2. Comparative Discussion	98

8	**CONCLUSION**	99
9	**SCOPE FOR FUTURE WORK**	102
10	**REFERENCES**	103

LIST OF FIGURES

S.No	Fig. No	Title	Page No
1	Figure 5.1	Architecture Diagram	21
2	Figure 5.2	Standard Scaler Feature Scaling Process	29
3	Figure 5.3	SMOTE Class Balancing Process	31
4	Figure 5.4	Binary Classification Pipeline	33
5	Figure 5.5	Multiclass Classification Pipeline	35
6	Figure 5.6	Performance Evaluation Framework	37
7	Figure 6.1	Dataset Sampling and Sampled Dataset Generation	42
8	Figure 6.2	Output of Data Cleaning	43
9	Figure 6.3	Dataset Information	44
10	Figure 6.4	Statistical Summary Output	44
11	Figure 6.5	Binary Class Distribution	45
12	Figure 6.6	Correlation Matrix	46
13	Figure 7.1	CNN Architecture	63
14	Figure 7.2	Hyperparameters	63
15	Figure 7.3	Training and Validation Accuracy over Epochs	64
16	Figure 7.4	Training and Validation Loss over Epochs	64
17	Figure 7.5	Accuracy Obtained across 10 Folds	65
18	Figure 7.6	Average ROC Curve across the 10-Fold Curve	66
19	Figure 7.7	Average Confusion Matrix over 10 Validation Folds	67
20	Figure 7.8	Metric Comparison	67

21	Figure 7.9	Two-dimensional t-SNE Visualization of the Binary Intrusion Dataset	68
22	Figure 7.10	ANN Architecture	69
23	Figure 7.11	Hyperparameters	69
24	Figure 7.12	Average Training and Validation Accuracy across 10-Fold Cross Validation	69
25	Figure 7.13	Average Training and Validation Loss over 10-Fold Cross Validation	70
26	Figure 7.14	Fold Accuracy over 10 Folds	70
27	Figure 7.15	Evaluation Metrics	71
28	Figure 7.16	Average ROC Curve	71
29	Figure 7.17	Average Confusion Matrix over 10 Folds	72
30	Figure 7.18	Metric Comparison	72
31	Figure 7.19	Accuracy over 10 Folds	74
32	Figure 7.20	Average ROC Curve of Random Forest	75
33	Figure 7.21	Average Confusion Matrix for Random Forest	76
34	Figure 7.22	Average Metric Comparison	77
35	Figure 7.23	Accuracy over 10 Folds	78
36	Figure 7.24	Performance Metrics of Decision Tree	79
37	Figure 7.25	ROC Curve of Decision Tree	79
38	Figure 7.26	Confusion Matrix of Decision Tree	80
39	Figure 7.27	Cross Validation Accuracy of Decision Tree	83
40	Figure 7.28	Performance Metrics of Decision Tree Multiclass	84
41	Figure 7.29	Confusion Matrix of Decision Tree Multiclass	84

42	Figure 7.30	Cross Validation Accuracy over 10 Folds	85
43	Figure 7.31	Performance Metrics of Random Forest Multiclass Classifier	86
44	Figure 7.32	Confusion Matrix of Random Forest Multiclass Classifier	86
45	Figure 7.33	Average Training and Validation Accuracy in ANN Multiclass	87
46	Figure 7.34	Average Training and Validation Loss in ANN Multiclass	88
47	Figure 7.35	Cross Validation over 10 Folds for ANN Multiclass using SMOTE	88
48	Figure 7.36	Performance Metrics of ANN Multiclass Classifier using SMOTE	89
49	Figure 7.37	Confusion Matrix of ANN Multiclass Classifier using SMOTE	89
50	Figure 7.38	Hyperparameters of CNN Multiclass Classifier using SMOTE	90
51	Figure 7.39	Training and Validation Accuracy over Epochs	90
52	Figure 7.40	Training and Validation Loss over Epochs	91
53	Figure 7.41	Cross Validation of Accuracy over Folds	91
54	Figure 7.42	Evaluation of Metrics for CNN Multiclass using SMOTE	92
55	Figure 7.43	Confusion Matrix of CNN Multiclass Classifier using SMOTE	92
56	Figure 7.44	Cross Validation of Accuracy over Folds	93
57	Figure 7.45	Metrics Evaluation of XGBoost Multiclass Classifier	94

58	Figure 7.46	Confusion Matrix of XGBoost Multiclass Classifier with SMOTE	94
59	Figure 7.47	Mean Accuracy of Models over Folds	96
60	Figure 7.48	Performance Metrics of Models in Ensemble Learning	96
61	Figure 7.49	Confusion Matrix of Weighted Voting	97
62	Figure 7.50	Multiclass Models Comparison	97

LIST OF TABLES

S.No	Table No.	Title	Page No
1	Table 4.1	Hardware Specifications	16
2	Table 4.2	Software Specifications	17
3	Table 4.3	Programming Languages and Libraries	17
4	Table 6.1	Binary Label Classification	48
5	Table 6.2	Binary Class Distribution	48
6	Table 6.3	Libraries Used for Random Forest	51
7	Table 6.4	Libraries Used for ANN	55
8	Table 6.5	Libraries Used for CNN	57
9	Table 7.1	Evaluation Metrics	65
10	Table 7.2	Hyperparameters for Random Forest	73
11	Table 7.3	Evaluation Metrics	74
12	Table 7.4	Hyperparameter Configuration of Decision Tree	78
13	Table 7.5	Binary Classification Model Comparison	81
14	Table 7.6	Decision Tree Hyperparameters for Multiclass	83
15	Table 7.7	Random Forest Hyperparameter Configuration	85
16	Table 7.8	Hyperparameters of ANN Multiclass with SMOTE	87
17	Table 7.9	Hyperparameter Configuration for XGBoost	93
18	Table 7.10	Hyperparameter Configuration for Ensemble Learning	95

ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DL	Deep Learning
EDA	Exploratory Data Analysis
FPR	False Positive Rate
GPU	Graphics Processing Unit
IDS	Intrusion Detection System
IoT	Internet of Things
ML	Machine Learning
PCC	Pearson Correlation Coefficient
RF	Random Forest
ROC	Receiver Operating Characteristic
SSM	State Space Model
TPR	True Positive Rate
UI	User Interface

1. INTRODUCTION

1.1. BACKGROUND OF THE STUDY

Internet of Things (IoT) has turned out to be one of the revolutionary technologies in the contemporary digital age that facilitates communications between physical objects via the Internet. IoT integrates several types of smart devices such as sensors, actuators, wearables, household appliances, industrial machines, and medical equipment that can communicate, process, and exchange data with little human participation. Due to the fast development of wireless communication, cloud computing, edge computing, and artificial intelligence, IoT implementation has been facilitated in many industries. Therefore, IoT has become an important part of a variety of applications like smart homes, smart cities, intelligent transportation, industrial automation, precision agriculture, environment monitoring, and healthcare.

The extensive use of IoT devices has brought about a new means of producing and using information, making processes efficient and cost-effective and leading to intelligent decision-making. While computing systems are composed of computers, IoT systems involve an overwhelmingly large number of communicating devices through various communication channels. These devices produce large amounts of data that can be analyzed for automated and predictive decision-making. However, most IoT devices have been developed with little or no computing power, memory space, and energy. The implication is that implementing security measures may be difficult in such devices, meaning that security is neglected in the process of device development.

Besides, due to the fast expansion of the Internet of Things (IoT), the rate and complexity of cyberattacks have increased significantly. Cybercrimes including DDoS attacks, brute force attacks, reconnaissance attacks, spoofing, botnet attacks, and manipulation of data have become more frequent in the IoT environment. IoT devices, which may have been compromised, could be used to disrupt infrastructures, obtain confidential information, invade user privacy, and carry out attacks on large scale. The Mirai botnet attack is a high-profile example which indicates that how vulnerable IoT devices could be leveraged to compromise millions of internet devices.

In order to protect IoT networks from various emerging threats to their cybersecurity, Intrusion Detection Systems (IDSs) are becoming a vital element of contemporary network security. IDS is a tool that constantly monitors all network traffic and activities in order to detect any malicious activity and intrusions in the network system. Traditional IDS tools basically use the signature-based detection approach, wherein network traffic is checked against a database of attack signatures that have already been identified. These systems prove to be highly effective in detecting known attacks, but tend to fail when it comes to detecting zero-day attacks, evolving malware, and new types of intrusions.

The recent improvements made in the areas of Artificial Intelligence (AI), Machine Learning (ML) and Deep Learning (DL) have greatly improved the functionality of the intrusion detection system. Different from the traditional IDS, AI-enabled IDSs can automatically learn from historical network traffic patterns and detect attacks both known and unknown by improving their accuracy. ML algorithms can successfully classify network traffic based on hand-crafted feature sets, whereas DL algorithms automatically extract the high-level representations of the network traffic patterns from the large dataset. This intelligent approach not only achieves better performance in detecting cybersecurity threats but also reduces the rate of false alarms.

Despite all these developments, creating an intrusion detection system using artificial intelligence in IoT networks is still a difficult problem to solve. Datasets of IoT usually consist of redundant features, irrelevant, and highly correlated features, which add to the complexity of computation and can harm classification accuracy. Moreover, class imbalance, noise in the data, and the high dimensionality of the traffic network are some of the problems faced during the training of the models. Hence, preprocessing and selecting features and evaluating the model becomes extremely important.

In view of these challenges, this project focuses on the development of an AI-based intrusion detection framework for IoT networks by integrating appropriate data preprocessing techniques, feature selection methods, and machine learning and deep learning algorithms. The objective is to enhance intrusion detection performance while reducing computational overhead, thereby contributing to the development of intelligent and reliable security solutions for modern IoT environments.

1.2. PROBLEM STATEMENT

Due to the quick growth in IoT, there has been an increase in the number of devices that interact and operate in different environments, such as smart homes, healthcare systems, industrial automation, smart cities, and intelligent transport systems. Despite the fact that these devices offer better automation and monitoring, they bring about many security issues because of their poor computing capacity, little memory, and light communication protocol. These IoT devices lack good security settings, which makes them vulnerable to attacks from cyber criminals.

The heterogeneity and distribution of the IoT networks make it even harder to implement any security mechanism. The IoT devices exchange private information in numerous communication layers that include perception layer, network layer, and application layer. All these layers are subject to attacks that can be categorized differently depending on the layer. For instance, the perception layer is prone to physical attacks, sensor attack, node capture, and jamming attacks. The network layer becomes a target for Man-in-the-Middle (MITM) attack, spoofing, replay attack, routing attacks, and network reconnaissance attack. In the application layer, the attacker uses software flaws via DDoS attack, malware, unauthorized access, and service disruption.

For overcoming such issues, intrusion detection systems (IDSs) are used extensively for monitoring network traffic and detecting malicious activities. But there are certain limitations associated with traditional methods of intrusion detection systems, namely, signature-based detection system and rule-based detection systems, which are used in dynamic IoT environment. Signature-based IDS uses predefined attack signature for detecting malicious activities. Even though it is highly accurate in detecting attacks that have already been known to us, but it cannot detect new kinds of zero-day attacks because their signature does not exist in the database.

In like manner, rule-based IDS relies on handcrafted rules and expert knowledge in order to recognize malicious behaviors. The process of creating and updating the rule is time-consuming and cumbersome, especially as new attack patterns are constantly emerging. There is need for constant updates in order to maintain good performance of the system, which makes the use of rule-based methods not very scalable in IoT

environments. In addition, both approaches face challenges in dealing with the massive data produced by the IoT networks.

A second problem faced while designing intelligent intrusion detection models is that of high-dimensional data sets, which are filled with redundant and irrelevant features along with high correlations between different features. High-dimensional data sets add to computational overheads, increase the time required for model training, and adversely affect the detection performance because of noise and irrelevant features in the dataset.

Therefore, there is a need for an intelligent, data-driven intrusion detection framework capable of efficiently analyzing IoT network traffic, selecting the most relevant features, and accurately distinguishing between normal and malicious activities. Such a framework should provide high detection performance while minimizing computational overhead, making it suitable for deployment in modern IoT environments.

1.3. MOTIVATION

The emergence of IoT technology at a breakneck pace has changed how different devices interact and communicate with each other through various applications. The smart home, healthcare system, automation industry, agriculture, and transportation all have become dependent on IoT devices to provide automation and real-time monitoring. However, the use of IoT devices has increased the number of attack surfaces and made the network vulnerable to attacks by the cyber-criminals. It has become important to maintain the integrity of these networks.

The conventional methodologies for intrusion detection include signature based and rule based systems. Even though these methods are quite efficient in detecting any attacks that have already been witnessed before, they lack the capability of detecting new attacks or any zero-day attacks since they depend on pre-programmed signatures and detection rules. In addition, the constant changing nature of cyber attacks necessitates regular updates of these detection systems.

The increasing volume of network traffic generated by IoT devices presents another major challenge for intrusion detection systems. Modern IoT datasets contain a large number of features, many of which may be redundant, irrelevant, or highly correlated.

Training machine learning and deep learning models using all available features increases computational complexity, prolongs training time, and may reduce the overall detection performance due to the inclusion of noisy information. Therefore, selecting the most informative features before model training is essential for improving both efficiency and classification accuracy.

The recent developments in the field of Artificial Intelligence (AI) such as Machine Learning (ML) and Deep Learning (DL) technologies have been proven very effective in improving the intrusion detection techniques. As opposed to traditional systems, these smart systems can easily learn complicated features through analysis of the network traffic itself. The machine learning algorithms make use of an effective feature set selection and classification, whereas the deep learning algorithms can easily learn the hierarchy in the network traffic.

Driven by the above challenges and opportunities, this study intends to propose an intelligent intrusion detection approach for IoT network by employing appropriate data preprocessing, feature selection, and AI approaches for classification. In this regard, the CICIoT2023 dataset is used in this study where data preprocessing and feature selection using Pearson Correlation Coefficient (PCC) are done in order to select the most significant features, and RF, ANN, and CNN are evaluated as classifiers for intrusion detection task. Comparison of these models based on performance measures will help to find out the efficient model for intrusion detection.

1.4. PROPOSED SOLUTION

To address the security challenges associated with Internet of Things (IoT) networks, this project implements an Artificial Intelligence (AI)-based intrusion detection system by following the methodology proposed in the selected research study. The implementation focuses on reproducing the data preprocessing, feature selection, model development, and evaluation pipeline presented in the reference work, while analyzing its effectiveness on the selected IoT intrusion dataset.

The proposed methodology begins with the CICIoT2023 dataset, which contains network traffic generated from both normal and malicious IoT activities. Since real-world datasets often contain duplicate records, missing values, infinite values, and redundant information, appropriate preprocessing techniques are applied to improve data quality before model development. The dataset is cleaned and prepared to ensure that it is suitable for training and evaluating machine learning and deep learning models.

Following data preprocessing, Exploratory Data Analysis (EDA) is performed to understand the characteristics of the dataset, identify feature distributions, examine class imbalance, and gain insights into the relationship between different network traffic attributes. This analysis assists in selecting an appropriate preprocessing and feature selection strategy.

To reduce the dimensionality of the dataset and eliminate redundant features, the methodology employs the Pearson Correlation Coefficient (PCC) feature selection technique, as described in the reference paper. PCC measures the correlation between individual features and the target class, enabling the selection of the most relevant attributes for intrusion detection. By retaining only informative features, the computational complexity of the learning models can be reduced while preserving classification performance.

Once the data processing stage of feature selection has been completed, the processed data set is split into training and testing sets. Then, the features selected are used to create and analyze three classification models that include the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN). They are all trained with the processed data set to provide an even ground of comparison.

The overall objective of this implementation is to reproduce the methodology presented in the selected research work and evaluate the performance of traditional machine learning and deep learning techniques for IoT intrusion detection. The outcomes of this study provide insights into the effectiveness of the implemented models and establish a foundation for future enhancements and comparative analyses.

2. OBJECTIVES

- To collect and preprocess an IoT network traffic dataset containing both normal and malicious traffic, including different attack types for binary and multi-class classification.
- To develop and compare Machine Learning and Deep Learning models for accurately detecting malicious IoT network traffic.
- To evaluate the developed intrusion detection models using metrics such as Accuracy, Precision, Recall, F1-Score, ROC-AUC, and False Alarm Rate and develop a visual dashboard to monitor and present their performance.

3. LITERATURE SURVEY

3.1. INTRODUCTION

Fast development of the Internet of Things has resulted in an increasing requirement for effective cybersecurity tools that can ensure the security of the devices connected in the network from possible cyber attacks. With the expansion of IoT networks in different domains of applications, various techniques of intrusion detection have been suggested in order to detect any possible malicious behavior in the network while maintaining the required efficiency and reliability of IoT. Within the last ten years, intrusion detection systems have become more advanced from simple signature-based methods to machine learning-based systems.

A number of researchers have proposed the use of machine learning techniques like Decision Tree, Random Forest, Support Vector Machine, Naive Bayes, and Ensemble Learning in detecting attacks in a network. These approaches have shown encouraging results in recognizing attack signatures and possess better adaptability than the rule-based approaches used traditionally. Nevertheless, most of the classical machine learning approaches depend on hand-crafted feature engineering and their performance may drop while processing high-dimensional data sets which include redundant features.

Given the growing availability of large cybersecurity datasets, deep learning methods have been gaining popularity because of their potential to learn advanced feature representations autonomously from network traffic data. Some of the machine learning algorithms that have been used for the purpose of intrusion detection include ANN, CNN, RNN, and LSTM. These methods have been successful in increasing the detection accuracy for complex attack situations. However, despite these developments, there are issues such as computational complexity, feature redundancy, class imbalance, and scalability that affect intrusion detection system performance.

To address these challenges, recent research has focused on combining efficient data preprocessing techniques with feature selection methods and intelligent classification algorithms. Feature selection techniques such as the Pearson Correlation Coefficient (PCC) have been widely employed to eliminate redundant features, reduce computational complexity, and improve model performance. The research paper

selected for this project follows this approach by integrating PCC-based feature selection with machine learning and deep learning models for IoT intrusion detection. The following sections present a detailed review of the existing literature, highlight recent developments in AI-based intrusion detection systems, and identify the research work that forms the foundation of the present implementation.

3.2. SUMMARY OF EXISTING RESEARCH PAPERS

There has been an improvement in technology of IoT in recent times, resulting in increased research efforts on securing devices that are connected against potential threats on the Internet. As shown by the studies in this paper, the development of the IoT in fields such as smart homes, health care, automation industry, transportation, and smart cities has posed a challenge for achieving a secure communication between resource constrained devices. This is because traditional methods of security cannot guarantee protection for modern day IoT environments owing to the heterogeneity of network traffic. As a result, there has been increased interest in creating IDSs that are intelligent enough to detect any threat with minimal computational overhead.

As far as the literature shows, intrusion detection techniques have come a long way during the last ten years. The first IDS technologies used signature- and rule-based approaches for traffic classification, where attack signatures and rules, which were pre-defined by experts, were used to detect attacks. This approach proved to be quite efficient in case of known attacks; however, it lacked adaptability towards zero-day attacks and new intrusion patterns. Besides, the constant update of signature and rule databases makes these approaches ineffective in ever-changing IoT environments.

The Machine Learning (ML) approach has proved to be one of the most common methods for IoT intrusion detection. There have been multiple studies focused on the evaluation of different supervised machine learning techniques, such as Decision Trees, Random Forest, SVM, Naive Bayes, K-Nearest Neighbors, Logistic Regression, and also ensemble learning approaches. Based on the analyzed literature, Random Forest has been one of the most popular classifiers used in the studies due to its efficiency and reduced risks of overfitting. Moreover, all kinds of ensemble learning approaches have shown better classification results than other approaches, as well as low computational complexity.

Due to increased access to cybersecurity big data, there is a lot of focus on DL approaches owing to their potential in automatic hierarchical representation learning. In the reviewed papers, many different types of ANN models such as CNNs, RNNs, LSTMs, and hybrid deep learning architectures have been used for intrusion detection. Among all these approaches, CNNs have shown better results since they are effective in capturing complex patterns in network traffic without relying much on manual feature extraction. However, one of the limitations of deep learning approaches is that they need bigger datasets, more computation resources, and longer training time as compared to traditional ML models.

Another significant finding from the reviewed literature is the increasing importance of data preprocessing and feature selection. Almost all publicly available intrusion detection datasets include redundant, irrelevant, or highly correlated features which might adversely impact the performance of models and make the computation more expensive. For this reason, many different feature selection algorithms have been investigated, such as filter methods, wrapper methods, embedded methods, and hybrid methods. One of the filter algorithms is the Pearson Correlation Coefficient (PCC), which has attracted much attention because of its low computational cost and effectiveness in selecting highly relevant features while filtering out redundant features.

Benchmark datasets have been pointed out by the literature as a necessary factor to test the intrusion detection systems. These are some of the widely used benchmark datasets – NSL-KDD, UNSW-NB15, CICIDS2017, BoT-IoT, IoTID20, and in recent years CICIoT2023. The datasets have various attack cases as well as traffic features for evaluation of the performance of the research models in various network environments. Evaluations have been carried out through parameters such as Accuracy, Precision, Recall, F1-Score, ROC curve, AUC, and Confusion Matrix.

Even with considerable advances in the state of art as described above, there are many issues that continue to hamper the practical application of AI-driven IDS in IoT environments. Some of the problems that have been identified include issues related to high-dimensional data, class imbalance, noisy network traffic, complexity of algorithms, low interpretability of models, and constant introduction of novel attacks. Increasing number of works deal with creating lightweight and scalable solutions to

intrusion detection problem that provide high precision through efficient preprocessing, feature selection, and machine learning models.

In general, the discussed literature illustrates an evident shift from traditional intrusion detection techniques to intelligent AI-based systems. The overall insights indicate that the combination of successful data preprocessing, efficient feature selection, and powerful machine/deep learning approaches can be considered an effective way towards enhancing the performance of intrusion detection systems within contemporary IoT environments. The outlined findings serve as the base for the methodology used within the current research, as suggested in the chosen PCC-CNN literature review paper.

3.3. SELECTED RESEARCH PAPER

The implementation used in this work is inspired from the research paper titled "Anomaly Based Intrusion Detection System for IoT Application" authored by Mansi Bhavsar, Kaushik Roy, John Kelly, and Odeyomi Olusola. In the research paper, an AI-based intrusion detection system has been suggested, which uses the Pearson Correlation Coefficient (PCC) method to select features along with a CNN for the purpose of anomaly detection in IoT networks. The aim of the research is to increase the performance of intrusion detection by removing redundant features before classification.

The authors recognize that IoT devices continuously generate large volumes of heterogeneous network traffic, making traditional intrusion detection techniques less effective due to increasing computational complexity and evolving attack patterns. They emphasize that datasets used for intrusion detection often contain redundant and highly correlated features, which unnecessarily increase training time and may reduce the effectiveness of learning algorithms. To address this challenge, the proposed framework first performs feature selection using the Pearson Correlation Coefficient (PCC), allowing only the most relevant features to be used for model training.

The methodology that will be adopted for this study involves various stages that are carried out sequentially, starting with data pre-processing. Firstly, the datasets are processed to ensure that they are ready for use in the modeling process. In the next stage, feature selection is performed using PCC. The correlation value of each feature and the target variable is determined. The features having higher correlation values are

selected while those having lower values are disregarded since they do not add much information to the classifier.

Following feature selection, the selected features are provided as input to a Convolutional Neural Network (CNN). Although CNNs are traditionally associated with image processing, the authors demonstrate that one-dimensional convolutional operations can effectively learn local feature relationships present in network traffic data. The CNN automatically extracts hierarchical representations from the selected features, enabling the model to distinguish between normal and malicious network traffic with high accuracy. The learning capability of CNN also reduces the dependence on manually engineered features, making it suitable for complex intrusion detection tasks.

In order to conduct performance analysis of the suggested framework, three datasets that are widely used for intrusions detections were used: NSL-KDD, CICIDS2017, and IoTID20. These datasets together cover different classes of cyber attacks as well as different networks, which allows the methodology to be tested under varied operational conditions. Evaluation of performance was done via calculation of such traditional measures of classification as accuracy, precision, recall, and f1-score.

The experimental results provided by the paper suggest that using the combination of PCC-based feature selection with the CNN model increases intrusion detection accuracy while decreasing the computational complexity of the process. This is achieved because of the reduction of unnecessary data prior to the training process which decreases the amount of calculations needed for learning. Furthermore, the results confirm that CNN can detect complicated patterns in the IoT network traffic effectively.

One of the major strengths of the proposed research is its systematic combination of efficient feature selection and deep learning-based classification. Instead of directly training a deep learning model using all available features, the authors first reduce the dimensionality of the data through PCC-based filtering, thereby improving both computational efficiency and model performance. Additionally, evaluating the framework on three different benchmark datasets enhances the credibility of the reported results by demonstrating consistent performance across multiple network environments.

The study, however, also presents certain limitations. The experimental evaluation is restricted to the NSL-KDD, CICIDS2017, and IoTID20 datasets, which may not fully represent the latest attack patterns observed in modern IoT environments. Recent datasets such as CICIoT2023, which include newer attack scenarios and more diverse IoT traffic, were not considered during evaluation. Furthermore, the research primarily focuses on the CNN-based classification model and does not provide a detailed comparative analysis with multiple machine learning and deep learning classifiers under identical preprocessing conditions. These observations provide opportunities for further evaluation using updated datasets and additional classification models.

The methodology proposed in this research paper serves as the foundation for the implementation carried out in the present project. While preserving the core preprocessing and feature selection strategy proposed by the authors, the current work evaluates the methodology using the CICIoT2023 dataset and investigates the performance of Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models under a common experimental framework. This implementation facilitates a broader comparative analysis while maintaining consistency with the methodology presented in the selected research paper.

The selected research paper was chosen because it presents a structured and well-validated methodology that integrates feature selection with deep learning for IoT intrusion detection. Its evaluation on multiple benchmark datasets, together with its clear preprocessing pipeline, makes it an appropriate foundation for implementation and comparative analysis. The methodology is reproducible and provides a suitable basis for assessing its performance on the CICIoT2023 dataset using additional classification models.

3.4. RESEARCH GAP

Despite the significant progress achieved in Artificial Intelligence (AI)-based Intrusion Detection Systems (IDSs), the literature survey indicates that several challenges continue to exist in securing Internet of Things (IoT) environments. Researchers have successfully demonstrated the effectiveness of machine learning and deep learning techniques in detecting malicious network traffic; however, factors such as rapidly evolving cyberattacks, increasing network complexity, and the growing volume of IoT-generated data continue to influence the performance and adaptability of intrusion detection systems.

A common observation from the reviewed literature is that many intrusion detection models have been evaluated using benchmark datasets such as NSL-KDD, CICIDS2017, and IoTID20. Although these datasets have significantly contributed to the advancement of intrusion detection research, they may not fully represent the characteristics of modern IoT environments or recently emerging attack patterns. As IoT technology continues to evolve, newer datasets containing more diverse network traffic and attack scenarios have become available, creating opportunities to evaluate existing methodologies under updated conditions.

Another important observation is that several studies primarily focus on evaluating a single classification model or comparing only a limited number of machine learning algorithms. While these investigations demonstrate the effectiveness of individual approaches, a broader comparative analysis involving both traditional machine learning and deep learning techniques under a common preprocessing framework can provide a better understanding of their relative strengths and limitations.

The selected research paper proposes an effective framework that combines Pearson Correlation Coefficient (PCC) based feature selection with a Convolutional Neural Network (CNN) for anomaly detection in IoT networks. The reported results demonstrate that reducing redundant features before classification improves computational efficiency while maintaining high detection accuracy. However, the evaluation presented in the paper is limited to the benchmark datasets considered by the authors and primarily focuses on the CNN classifier.

Motivated by these observations, the present project implements the methodology proposed in the selected research paper and evaluates its applicability using the

CICIoT2023 dataset, which contains recent IoT network traffic and attack scenarios. In addition to implementing the CNN model, the project also evaluates the performance of Random Forest (RF) and Artificial Neural Network (ANN) models using the same preprocessing and feature selection pipeline. This enables a consistent comparison of machine learning and deep learning approaches while preserving the core methodology proposed in the reference study.

The objective of the present work is not to propose a new intrusion detection framework but to reproduce, implement, and evaluate the selected methodology using a modern benchmark dataset and multiple classification models. The findings obtained from this implementation are expected to provide insights into the effectiveness, adaptability, and practical applicability of the proposed methodology in contemporary IoT intrusion detection scenarios.

4. SPECIFICATIONS

4.1. HARDWARE SPECIFICATIONS

The implementation of the proposed intrusion detection system was carried out using a high-performance Linux-based GPU server to efficiently process the large-scale CICIoT2023 dataset and train machine learning and deep learning models. The computational resources available on the server significantly reduced preprocessing and model training time compared to conventional desktop environments.

Component	Specification
Processor	Multi-core x86_64 CPU
Operating System	Linux
GPU	2 × NVIDIA A100 PCIe 80 GB
RAM	High-memory server
Storage	SSD Storage
Internet	High-speed network connectivity

Table 4.1. Hardware Specifications

4.2. SOFTWARE SPECIFICATIONS

The project was developed using Python and various open-source libraries for data preprocessing, visualization, feature selection, machine learning, and deep learning. The Linux operating system provided a stable environment for executing the preprocessing pipeline and training the classification models.

Software	Purpose
Ubuntu/Linux	Development Environment
Python 3.x	Programming Language
Jupyter Notebook / VS Code	Development Environment
Google Colab *(Initial Stage)*	Preliminary experimentation
Linux GPU Server	Final implementation
Git	Version Control

Table 4.2. Software Specifications

4.3. PROGRAMMING LANGUAGES AND LIBRARIES

The implementation of the proposed intrusion detection system utilized several Python libraries to perform data preprocessing, feature engineering, model development, visualization, and performance evaluation.

Library	Purpose
Pandas	Data manipulation
NumPy	Numerical computations
Matplotlib	Data visualization
Scikit-learn	ML algorithms and evaluation
TensorFlow / Keras	ANN and CNN implementation
Joblib	Saving trained models

Table 4.3. Programming Languages and Libraries

4.4. DATASET SPECIFICATION

The present study utilizes the CICIoT2023 dataset for implementing and evaluating the proposed intrusion detection framework. The dataset is one of the latest publicly available benchmark datasets designed specifically for Internet of Things (IoT) intrusion detection research. It contains both normal and malicious network traffic generated from realistic IoT environments and includes a wide range of modern cyberattack scenarios.

The dataset is made up of several CSV files with network traffic features that show various communication traits and attack types. The data was preprocessed before developing the models through combining several files, discarding duplicate records, dealing with missing and infinite values, feature selection using the Pearson Correlation Coefficient (PCC), and normalization of the data.

As opposed to earlier benchmark datasets like NSL-KDD, CICIDS2017, and IoTID20, the CICIoT2023 dataset has more current attack patterns and also traffic in large volume and therefore it is ideal for assessing the performance of modern AI-based IDSs.

4.5. PERFORMANCE EVALUATION METRICS

The performance of the implemented classification models is evaluated using standard classification metrics widely adopted in intrusion detection research. These metrics provide a comprehensive assessment of the model's capability to correctly classify normal and malicious network traffic.

The primary evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

These performance measures are calculated after testing each classification model and are used to compare the effectiveness of the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models.

5. ARCHITECTURE

5.1. SYSTEM OVERVIEW

The proposed Intrusion Detection System (IDS) based on Artificial Intelligence (AI) will act as a tool for providing intelligent and efficient means of detection of any malicious activity taking place in the IoT network environment. As a consequence of the proliferation of the use of IoT in different sectors such as health care, smart cities, industrial automation, and intelligent transport systems, the complexity of network traffic has become quite significant. The existing IDS systems that are based on signatures find it difficult to detect new kinds of attacks and need constant updates to remain effective. This study proposes a data-driven IDS system that makes use of machine learning and deep learning methods.

The framework under development has been trained on CICIoT2023, a dataset that has realistic Internet of Things (IoT) network traffic data that includes both normal network communication data and different types of cyberattacks. Given that the dataset has several million records across a number of CSV files, extensive data preparation has been conducted before developing a model on the dataset. The files were first combined into one dataset and cleaned to eliminate duplication, missing data, and inconsistency. After that, exploratory data analysis was conducted to explore the statistics and classes within the dataset.

After performing the above-mentioned preprocessing step, feature selection using Pearson Correlation Coefficient (PCC) method was used for choosing the most valuable network traffic features by eliminating redundant and less important features. In other words, using a reduced number of more correlated features will not only decrease the computational complexity but also improve the learning ability of the classifiers. Furthermore, the chosen features were scaled using the StandardScaler method, making sure that all numerical features had a similar scale and avoided dominating the learning process.

Given that the dataset contained a notable class imbalance between the normal and attack classes, the technique used to handle this imbalance was the Synthetic Minority Oversampling Technique (SMOTE), which helped create new examples from the minority classes. Class imbalance is crucial in intrusion detection since the biased

nature of the training data causes the classification model to favor the majority class but ignore other attack classes that occur infrequently. The three preprocessing techniques created an efficient process that would ensure high-quality data was used by the classification models.

The pre-processed data was then used for building models for both binary and multi-class classifications of intrusions. In the binary classification case, the network traffic was classified into two classes; whether the traffic is normal or malicious in nature. It provided an initial evaluation of the intrusion detection model. The models used for binary classification include the Decision Tree, Random Forest, Artificial Neural Network (ANN) and Convolutional Neural Network (CNN). In the multi-class classification, the malicious traffic was classified based on their classes. Models used for multi-class classification include Decision Tree, Random Forest, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), XGBoost, and Ensemble Learning models.

To ensure a fair and reliable comparison among all classification models, the proposed framework employed Stratified 10-Fold Cross Validation, preserving the original class distribution in every fold throughout the training and testing process. Model performance was evaluated using multiple metrics, including Accuracy, Precision, Recall, F1-Score, Receiver Operating Characteristic - Area Under the Curve (ROC-AUC) for applicable models, and Confusion Matrix analysis. These evaluation measures provide a comprehensive assessment of classification performance and enable an objective comparison of the developed Machine Learning and Deep Learning models.

Overall, the proposed methodology integrates systematic data preprocessing, effective feature engineering, balanced data preparation, and advanced classification techniques into a unified intrusion detection framework. By combining both traditional Machine Learning algorithms and modern Deep Learning architectures under a common evaluation strategy, the proposed system provides an effective solution for detecting and classifying cyberattacks in IoT environments while facilitating a comprehensive comparative analysis of different AI-based intrusion detection approaches.

5.2. SYSTEM ARCHITECTURE

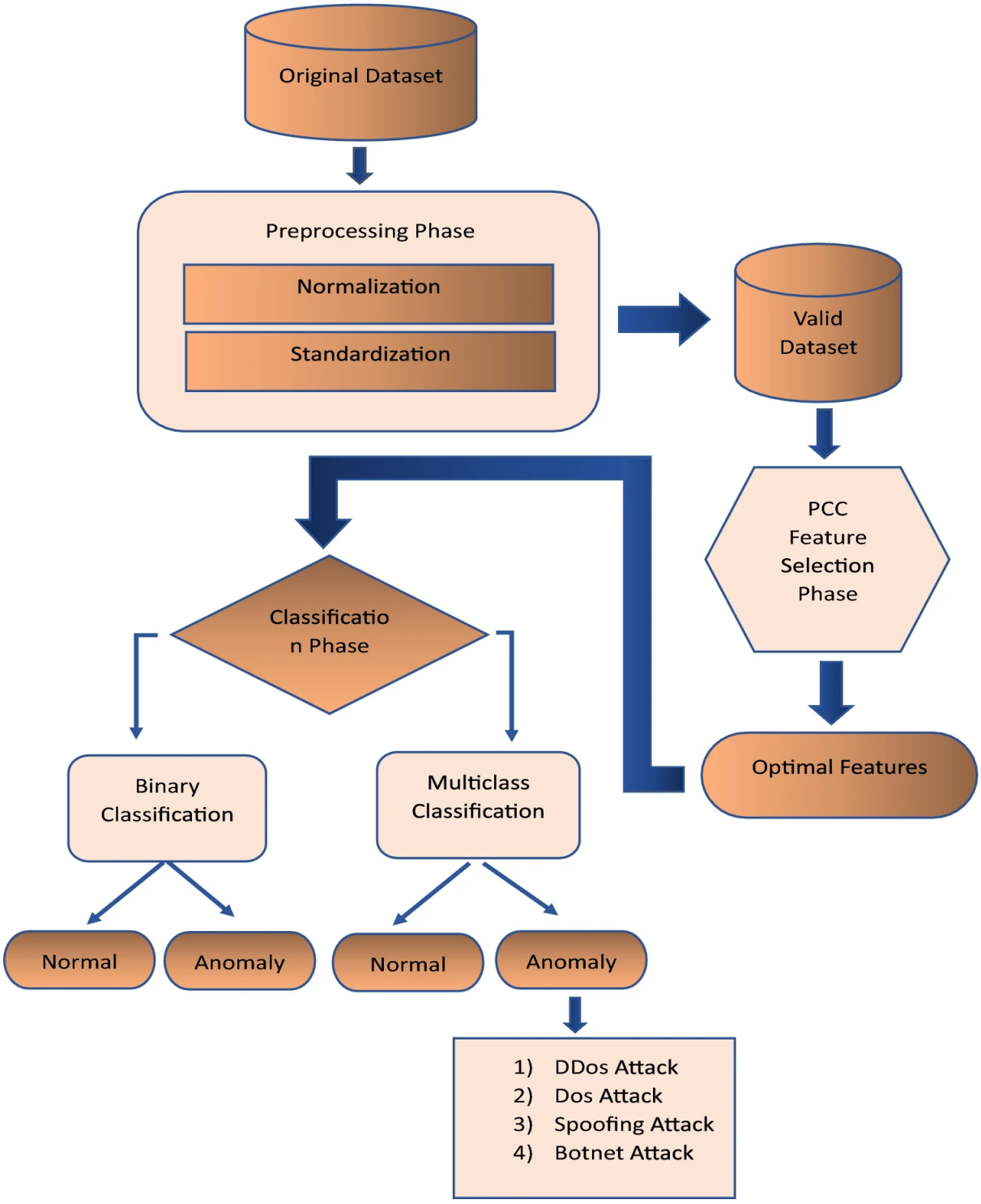


Figure 5.1. Architecture Diagram

The architecture adopted in the present work is based on the framework proposed in the selected research paper, "Anomaly-Based Intrusion Detection System for IoT Application." The framework provides a systematic approach for detecting anomalies in IoT network traffic by integrating data preprocessing, feature selection, deep learning-based classification, and performance evaluation. The overall architecture consists of multiple interconnected stages, where the output of each stage serves as the input for the subsequent stage, thereby ensuring a structured and efficient intrusion detection process.

The implementation begins with the acquisition of network traffic datasets containing both normal and malicious communication records. In the original research paper, the proposed framework was evaluated using the NSL-KDD, CICIDS2017, and IoTID20 benchmark datasets to validate its effectiveness across different intrusion detection scenarios. These datasets contain various categories of cyberattacks together with legitimate network traffic, providing a comprehensive environment for evaluating the proposed methodology.

To improve computational efficiency and eliminate redundant information, Pearson Correlation Coefficient (PCC)-based feature selection is applied to identify the most relevant network traffic attributes. The selected features are subsequently standardized using the StandardScaler technique to ensure that numerical features share a common scale, thereby improving the convergence and learning behaviour of the classification algorithms. Since the dataset contains an imbalanced distribution of attack categories, the Synthetic Minority Oversampling Technique (SMOTE) is employed to generate synthetic samples for minority classes, resulting in a more balanced dataset that reduces classification bias and improves the detection capability of the developed models.

After preprocessing and feature engineering, the architecture divides into two independent classification pipelines. The first pipeline performs binary intrusion detection, where network traffic is classified as either normal or malicious using Decision Tree, Random Forest, Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models. The second pipeline performs multiclass intrusion detection, where malicious traffic is further classified into its corresponding attack categories using Decision Tree, Random Forest, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), XGBoost, and Ensemble learning models.

Maintaining separate classification pipelines enables the proposed framework to evaluate both generalized attack detection and fine-grained attack classification using the same preprocessing strategy.

Predictions from each of the models used in classification are measured through the use of a standardized method of performance evaluation through the use of Stratified 10-Fold Cross Validation method. Accuracy, Precision, Recall, F1-Score, ROC-AUC where applicable, and Confusion Matrix are among the methods used in determining the effectiveness of the model. Finally, a comparative performance analysis is conducted on all the machine learning and deep learning models to determine the most effective model for artificial intelligence-based intrusion detection in IoT environments. In summary, the proposed architecture presents an overall end-to-end approach of efficient data preprocessing, feature engineering, balanced learning, and performance evaluation in an intrusion detection system.

Although the present project follows the overall architecture proposed in the selected research paper, the experimental implementation is performed using the CICIoT2023 dataset. Furthermore, in addition to reproducing the CNN-based methodology, the implementation also evaluates Random Forest (RF) and Artificial Neural Network (ANN) models using the same preprocessing and feature selection pipeline. This enables a comparative analysis of machine learning and deep learning approaches while preserving the fundamental architecture proposed by the original study.

5.3. SYSTEM MODULES

This architecture is divided into several functional modules which are responsible for providing the desired functionality of detecting any potential intrusion. Each module has its own particular role in the whole process, starting from the data collection phase up until the evaluation of the trained classifiers. This design makes the whole process much clearer and guarantees that each step adds to the performance of the intrusion detection system.

5.3.1. Dataset Collection

The first module of the proposed system is responsible for acquiring the network traffic dataset required for intrusion detection. In the original research paper, the methodology was evaluated using the NSL-KDD, CICIDS2017, and IoTID20 benchmark datasets to validate the effectiveness of the proposed framework across different intrusion detection scenarios. These datasets contain both legitimate and malicious network traffic representing multiple categories of cyberattacks.

For the implementation carried out in the present project, the **CICIoT2023** dataset is utilized as the primary source of network traffic. The dataset contains realistic IoT communication generated from multiple smart devices together with a wide range of contemporary cyberattacks. Its large size and diverse attack categories make it suitable for evaluating modern Artificial Intelligence-based intrusion detection systems.

The collected dataset serves as the input for all subsequent stages of the implementation, including preprocessing, feature selection, model training, and performance evaluation.

To ensure consistency throughout the experimentation process, the same merged dataset was used for both Machine Learning and Deep Learning models. This approach eliminated variations arising from different data sources and enabled a fair comparison among Decision Tree, Random Forest, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), XGBoost, and Ensemble learning models. By utilizing a single benchmark dataset throughout the study, the experimental results presented in subsequent chapters accurately reflect the influence of the classification algorithms rather than differences in the underlying data.

5.3.2. Data Preprocessing

Data preprocessing is an essential step in the creation of any ML/DL classifier, since the quality of the data fed into the algorithm has a direct effect on the performance of the final classifier. The raw network traffic dataset may have a number of inconsistencies including but not limited to duplicating entries, infinite values, noisy observations, and missing data. As a result, prior to the beginning of the feature engineering and the model creation process, the unified CICIoT2023 dataset was subjected to the data preprocessing phase.

The first step involved in the preprocessing phase entailed verifying and validating the consolidated dataset that was generated by merging all the datasets from individual CSV files. The dataset was analyzed to detect any duplicates that would create bias in the training of the models. The duplicates were then cleaned to avoid having multiple instances of the same network traffic influencing the learning process. After removing duplicates from the dataset, it was verified for missing or invalid data that might negatively impact the statistical and classification models.

Apart from missing values, the dataset was examined for any presence of infinite numerical values that could emerge as a result of some computational or measurement errors while creating network traffic. These infinite numerical values have a considerable effect on the calculation process and must be appropriately treated to maintain numerical stability. All feature attributes were checked for infinite numerical values. In addition, consistency checks were conducted to make sure that the features, their types, and classes are correct.

Moreover, the preprocessing process also ensured that the data set generated was structured uniformly and could be used for intrusion detection both for binary classification as well as multiclass classification. In the case of binary classification, the labeling process involved the combination of all types of attack categories into one class of malicious traffic and normal traffic into the other class. At the same time, the original attack labels were also kept intact in order to enable multiclass classification of the attacks in order to detect their exact category through intrusion detection.

The output of the preprocessing stage was a clean, consistent, and well-structured dataset that served as the foundation for subsequent feature engineering and model development. The processed dataset was then supplied to the feature selection stage,

where the most informative network traffic attributes were identified using the Pearson Correlation Coefficient (PCC). By ensuring data quality prior to feature engineering, the preprocessing stage contributed significantly to improving the stability, efficiency, and overall performance of the Machine Learning and Deep Learning models developed in this work.

5.3.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted in order to understand all the features of the dataset that was prepared for further feature engineering and model building. EDA helps in spotting any trends within the data, spotting any inconsistencies within the data, understanding how the features behave, and determining the distribution of various types of attacks. In doing so, it is easier to come to the decision about the choice of the features and the data, as well as the model, which needs to be applied.

In the beginning, the cleansed data set was assessed to ensure its overall structure, including the number of observations, features' properties, data types, and class labels. The descriptive statistics like mean, standard deviation, minimum, maximum, and quartiles were analyzed to get information on the numerical properties of each feature. This analysis helped to find out about variability within each feature and to detect those attributes that have very high or non-normal range of values.

The distribution of network traffic classes was subsequently analysed to understand the representation of legitimate traffic and various attack categories within the dataset. Visualizing class frequencies enabled the identification of significant class imbalance, where certain attack classes contained substantially fewer samples than others. Such imbalance is commonly observed in real-world intrusion detection datasets and may result in biased learning if not properly addressed. The findings obtained during this analysis motivated the adoption of an appropriate class balancing technique in the subsequent stages of the proposed methodology.

Further analysis was performed to investigate the relationships among different feature attributes. Correlation analysis assisted in identifying redundant features exhibiting similar behaviour as well as features demonstrating stronger relationships with the target class. Understanding these relationships provided valuable guidance for the feature selection process and contributed to reducing data redundancy while preserving the most informative network traffic characteristics. The insights obtained from

correlation analysis formed the basis for applying the Pearson Correlation Coefficient (PCC)-based feature selection technique described in the following subsection.

In addition to statistical and correlation analysis, graphical visualizations were generated to examine feature distributions and class proportions within the dataset. These visualizations provided an intuitive understanding of the underlying data characteristics and helped validate the effectiveness of the preprocessing pipeline. The information obtained through EDA not only improved the interpretability of the dataset but also supported the selection of suitable preprocessing techniques and classification models for the proposed intrusion detection framework.

In general, EDA provided an in-depth understanding of the dataset that was preprocessed before feature engineering. Through exploring statistical properties, class imbalance, and the correlation between features, EDA made sure that the next steps of feature selection, feature scaling, class balancing, and training the machine learning model were conducted on a dataset that was well-analyzed. This approach played an important role in increasing the reliability of the proposed AI-based intrusion detection system.

5.3.4. Feature Selection using Pearson Correlation Coefficient (PCC)

Feature selection plays a very vital role in the construction of ML and DL models especially where high dimensional datasets having a lot of attributes have been used. While many attributes can give more information, they can also cause redundancy, increased complexity, and even poor performance of the classification algorithms. As such, it becomes necessary to select the most valuable features before training the model for better efficiency and generalization. In the proposed intrusion detection system, the PCC method was used to conduct feature selection with a view to identifying the attributes of network traffic that have the strongest correlation with the target class.

PCC (Pearson Correlation Coefficient) is a statistical tool used to quantify the degree of linear relationship between two variables. In this study, the PCC technique was used to find the correlation between each network traffic attribute and its class label. Attributes that correlated highly with the dependent variable were deemed more useful for detecting intrusions, while those that showed no correlation at all or redundant behavior were not included in any further process. Such an approach helped the

suggested framework include only the important attributes while excluding useless ones.

The application of PCC-based feature selection significantly reduced the dimensionality of the dataset while preserving the information required for effective attack detection. By removing redundant and less informative features, the resulting feature set became more compact and computationally efficient, allowing both Machine Learning and Deep Learning models to learn meaningful patterns from the network traffic data more effectively. Furthermore, reducing feature dimensionality decreases the risk of overfitting and contributes to improved model generalization when evaluated on previously unseen network traffic.

Following the computation of correlation coefficients, the most relevant network traffic attributes were selected to form the final feature set used throughout the experimental phase of this study. These selected features represent important characteristics of network communication, including protocol information, packet statistics, traffic rate, and other behavioural indicators that contribute to distinguishing legitimate traffic from malicious activities. The same feature subset was consistently employed for all binary and multiclass classification models, thereby ensuring a fair and unbiased comparison among the different Machine Learning and Deep Learning algorithms investigated in this work.

The selected feature set was subsequently supplied to the feature scaling stage, where numerical attributes were standardized using the StandardScaler technique before model training. This sequential preprocessing strategy ensured that the input data provided to the classification algorithms was not only relevant but also normalized and computationally suitable for efficient learning. Consequently, the combination of PCC-based feature selection and subsequent preprocessing stages established a robust foundation for the development of the proposed AI-based intrusion detection system.

5.3.5. Feature Selection Using Standard Scaler

Feature scaling is an essential preprocessing technique in Machine Learning (ML) and Deep Learning (DL) applications, particularly when the dataset contains numerical attributes with varying ranges and units of measurement. Since network traffic features are often represented using different numerical scales, attributes with larger values may dominate the learning process, thereby reducing the contribution of features with smaller numerical ranges. To eliminate this imbalance and ensure that all features contribute equally during model training, the proposed intrusion detection framework employs the StandardScaler technique after the feature selection stage.

StandardScaler is an extremely popular approach used to normalize data by subtracting the mean value of each numerical feature from itself and then dividing it by its standard deviation. This way, the normalized feature set will have a mean value of 0 and a standard deviation of 1. Unlike normalization procedures that limit the range within which the data can vary, the StandardScaler approach makes sure that the distribution of the features is kept intact, while all the attributes are on the same scale.

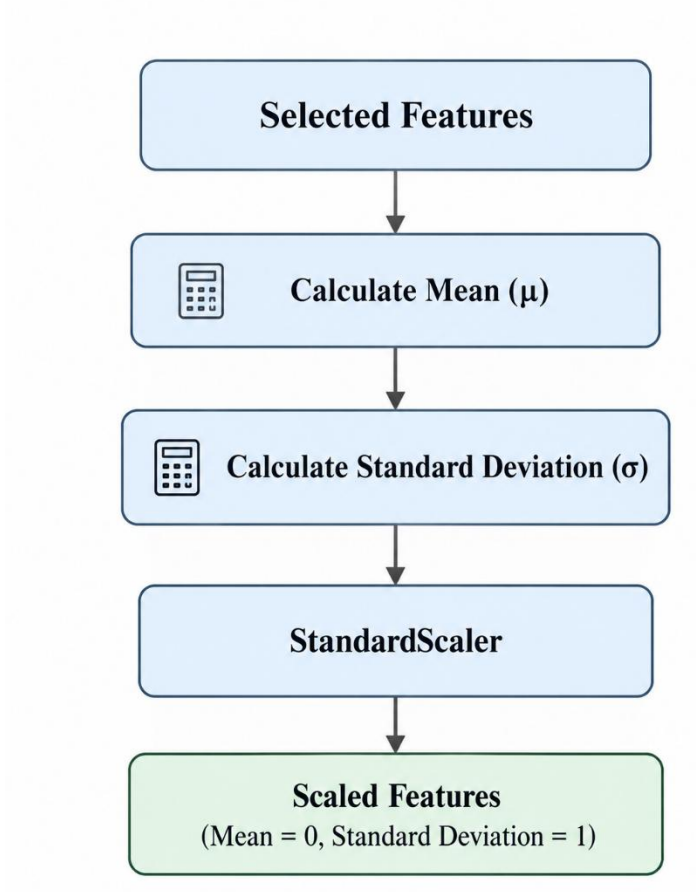


Fig 5.2 Standard Scaler Feature Scaling Process

In the proposed methodology, StandardScaler was applied to the feature subset obtained after Pearson Correlation Coefficient (PCC)-based feature selection. Performing feature scaling after feature selection reduced unnecessary computations by standardizing only the relevant network traffic attributes rather than the complete feature set. This sequential preprocessing strategy improved computational efficiency while ensuring that the selected features were appropriately transformed before model training.

The usage of StandardScaler played an important role in enhancing the learning behavior of the created classification models. Distance-based optimization techniques and gradient-based learning algorithms, in particular Artificial Neural Networks (ANN) and Convolutional Neural Networks (CNN), are very susceptible to the disparity in the magnitude of features. The standardization of the data helped to improve the speed of convergence in the training process, increase the numerical stability, and decrease the probability of biased updating of parameters due to the presence of large features. While tree-based algorithms, such as Decision Tree and Random Forest, are not that sensitive to the scale of features, the same standardized data set was used for all Machine Learning and Deep Learning techniques.

Following feature scaling, the standardized dataset was supplied to the class balancing stage, where the Synthetic Minority Oversampling Technique (SMOTE) was employed to address the imbalance among attack categories. By ensuring that all selected features possessed a common statistical scale before class balancing and model development, StandardScaler played a vital role in enhancing the stability, efficiency, and reliability of the proposed AI-based intrusion detection framework.

5.3.6. Class Balancing using Synthetic Minority Oversampling Technique

Class imbalance is a common challenge in intrusion detection datasets, where certain attack categories are represented by a significantly smaller number of samples than others. Learning algorithms trained on such imbalanced datasets tend to become biased toward majority classes, resulting in poor detection of minority attack categories and an increased number of false negative predictions. Since accurately identifying all types of cyberattacks is a fundamental objective of an Intrusion Detection System (IDS), addressing class imbalance is an essential step in the proposed methodology.

Following feature selection and feature scaling, the processed dataset was analysed to determine the distribution of the target classes. The analysis revealed that several attack categories contained considerably fewer instances compared to the dominant classes, creating an imbalance that could adversely affect the learning capability of both Machine Learning (ML) and Deep Learning (DL) models. To overcome this limitation, the Synthetic Minority Oversampling Technique (SMOTE) was employed to balance the class distribution prior to model training.

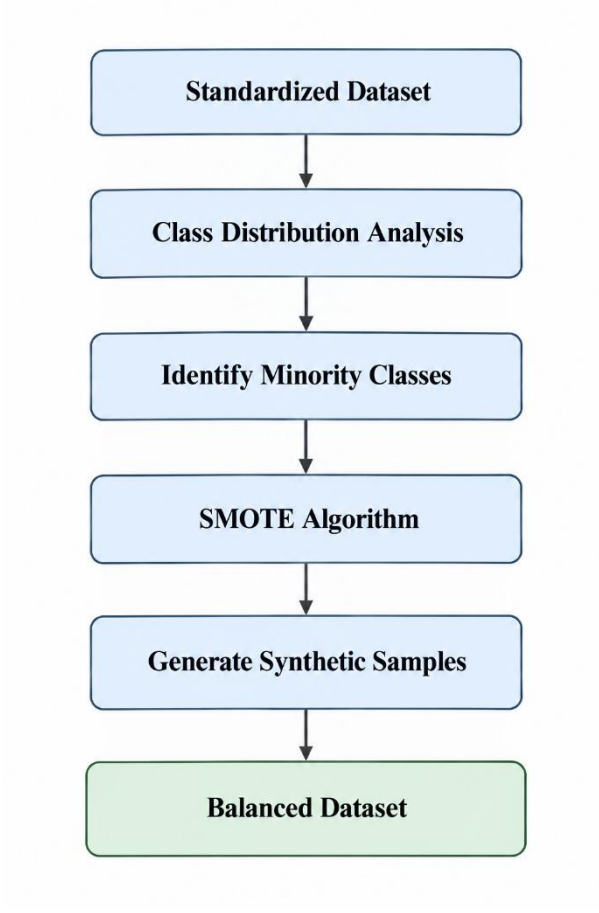


Fig 5.3 SMOTE Class Balancing Process

The SMOTE algorithm works on the principle of oversampling where synthetic instances from minority classes are created rather than using any duplicated instance from the data set. In this way, SMOTE finds the neighboring instances from the same class and then generates new synthetic instances by interpolating between them. This algorithm is superior to random oversampling methods since random oversampling methods tend to make the classifier overfit because of the repetition of observations; however, SMOTE generates diverse and representative synthetic instances that enhance the learning ability of the classifier.

As part of the intrusion detection system, SMOTE was used after feature selection through Pearson Correlation Coefficient (PCC) and feature scaling using StandardScaler. Through this approach, the use of SMOTE after these two preprocessing techniques ensured that the artificial data points were generated from a standardized and reduced feature space. This technique not only enhanced the quality of the generated data points but also ensured minimal computation load.

The balanced dataset obtained through SMOTE was subsequently utilized for training both binary and multiclass classification models. By improving the representation of minority classes, the proposed framework enabled the developed classifiers to learn decision boundaries more effectively and reduced the bias toward majority classes. Consequently, the models achieved improved detection capability for less frequent attack categories while maintaining stable overall classification performance across different intrusion scenarios.

The integration of SMOTE into the preprocessing pipeline significantly enhanced the robustness of the proposed AI-based intrusion detection system. A balanced training dataset contributed to improved Precision, Recall, and F1-Score, particularly for minority attack classes that are typically more challenging to detect. Furthermore, employing the same balanced dataset across all Machine Learning and Deep Learning models ensured a fair experimental comparison and allowed the influence of different classification algorithms to be evaluated under identical training conditions.

5.3.7. Binary Classification Models

Following data preprocessing, feature selection, feature scaling, and class balancing, the prepared dataset was utilized for binary intrusion detection. In the binary classification task, all malicious traffic records were grouped into a single attack class, while legitimate network traffic was retained as the normal class. This simplified classification problem enables the developed models to determine whether a network communication represents normal behaviour or a potential cyberattack. Binary intrusion detection serves as the first level of security analysis and provides a rapid mechanism for identifying suspicious activities before attempting detailed attack categorization.

In order to find out the efficacy of various Artificial Intelligence (AI) algorithms, four classifiers belonging to both conventional Machine Learning (ML) and Deep Learning (DL) paradigms have been deployed. In case of the former, the classifiers are Decision Tree and Random Forest, while in case of the latter, the classifiers are Artificial Neural Networks (ANN) and Convolutional Neural Network (CNN). These classifiers have been chosen because of their diversity in terms of learning approach and give us the opportunity to compare decision rule-based learning, ensemble learning, and other paradigms in one intrusion detection system.

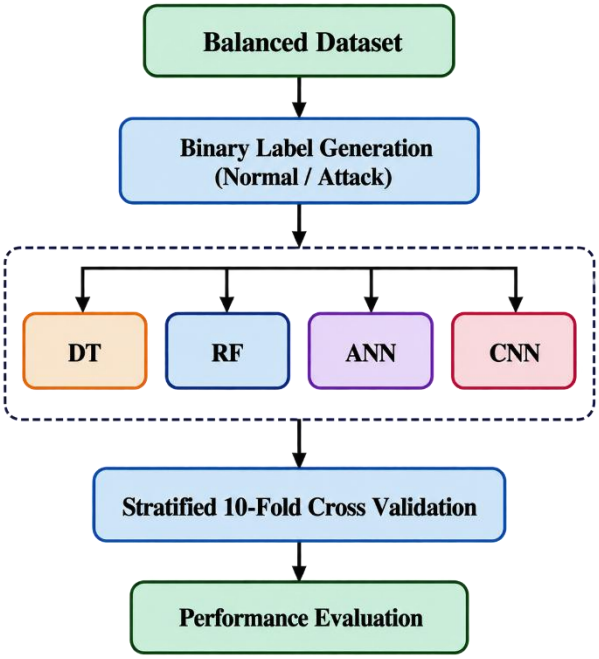


Fig 5.4. Binary Classification Pipeline

In order to find out the efficacy of various Artificial Intelligence (AI) algorithms, four classifiers belonging to both conventional Machine Learning (ML) and Deep Learning (DL) paradigms have been deployed. In case of the former, the classifiers are Decision Tree and Random Forest, while in case of the latter, the classifiers are Artificial Neural Networks (ANN) and Convolutional Neural Network (CNN).

The Decision Tree classifier constructs a hierarchical decision structure by recursively partitioning the feature space based on the most informative attributes. Its simple architecture enables fast training and straightforward interpretation of classification decisions, making it suitable as a baseline Machine Learning model for binary intrusion detection.

Random Forest Classifier is an advancement in Decision Tree Classifier whereby several independently learned decision trees are combined to form an ensemble. The outcome is predicted by using majority voting among these trees to reduce variance and overfitting. It is more efficient computationally compared to Decision Tree Classifier and gives better results.

Artificial Neural Network (ANN) carries out nonlinear classification via several interconnected neurons within its layers. In this regard, ANN learns relationships among the selected features in the network traffic to capture complex patterns and determine whether they represent legitimate traffic or cyber attacks. The capability of ANN to create nonlinear decision boundaries is very suitable for cybersecurity.

The Convolutional Neural Network (CNN) model employs one-dimensional convolutional operations to automatically extract discriminative feature representations from the processed network traffic data. Convolutional layers identify local feature interactions, while subsequent fully connected layers perform the final classification. This architecture enables CNN to learn hierarchical feature representations that improve the detection capability of malicious network traffic.

In order to achieve a fair experimental comparison between the four binary classification models, the exact same preprocessed dataset along with the exact same methodology of evaluation was applied for each algorithm. The Stratified 10 Fold Cross Validation method was used in order to maintain the distribution of classes in every fold, while Accuracy, Precision, Recall, F1 Score, ROC AUC and Confusion Matrix evaluation methods were used as the main performance indicators.

5.3.8. Multiclass Classification Models

While binary intrusion detection is used to classify network traffic as either benign or malicious, it cannot help detect the particular category of attack that causes the intrusion. The ability to detect the particular category of the attack is important in the actual world of cybersecurity to enable the deployment of the right mitigation strategy, conduct forensics investigation, and even understand the behaviour of the attackers. Therefore, apart from the binary classification, our AI-based IDS conducts multiclass classification to classify different attacks in the CICIoT2023 dataset.

In contrast with binary classification, which lumps all the malicious traffic under one single class of attack, the multi-class classification problem retains the original class labels for each attack in the dataset. The result is that the machine learning models created learn to classify the distinct features associated with each attack class while at the same time distinguishing the attacks from the genuine network traffic.

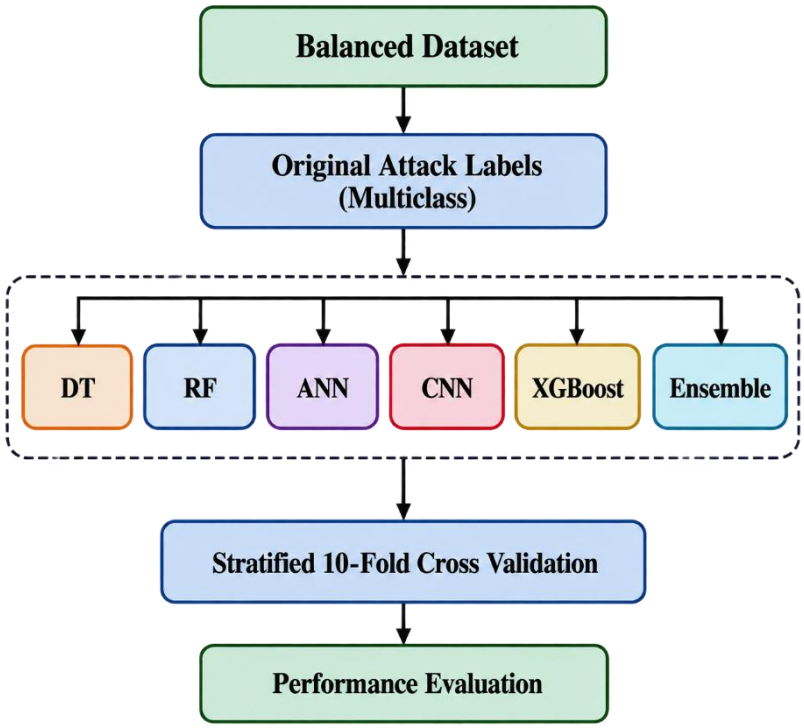


Fig 5.5. Multiclass Classification Pipeline

For analyzing the effectiveness of various learning algorithms, six classification methods have been developed for multiclass intrusion detection. The classification methods used in the study include Decision Tree, Random Forest, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), XGBoost, and Ensemble

Learning method. The choice of these classifiers provides an opportunity to compare their efficiency using Machine Learning, Deep Learning, gradient boosting algorithms, and ensembles of classifiers.

Decision Tree classifier algorithm gives a straightforward method of classification by recursively splitting the feature space according to the most discriminative features. In order to achieve more robustness and prevent over-fitting, Random Forest builds up a collection of Decision Trees through an ensemble technique. Artificial Neural Network (ANN) captures the complex nonlinear interaction between the selected traffic features using a layer of neurons, while Convolutional Neural Network (CNN) uses one dimensional convolution to hierarchically learn features.

Apart from these models, XGBoost was used as another advanced gradient boosting method that can build decision trees in a sequence in order to reduce the errors of classification. The model's ability to enhance the performance of a model by using the method of gradient boosting and regularization is very beneficial in solving complicated problems of multiclass classification. An Ensemble Learning model was built to use the predictions of many classifiers to benefit from the advantages of each classifier, while overcoming its disadvantages at the same time.

In order to make sure that consistency is maintained during the entire process of experiments, all multi-class classifiers were trained on the same pre-processed, standardized and balanced dataset through SMOTE technique. In addition, Stratified 10 Fold Cross Validation was used for testing the performance of each and every classifier in exactly the same experimental conditions, while keeping the proportion of classes intact in each fold. The following parameters were used for performance evaluation:

5.3.9. Performance Evaluation

The last step in the proposed methodological approach is to perform an assessment of the efficiency of the created Machine Learning and Deep Learning algorithms used for the detection and classification of network attacks. The assessment process was done using a systematic approach in order to be sure that all classifiers would be evaluated under the same experimental conditions. Due to the fact that different classifiers were studied for the binary and multiclass intrusion detection problems, a unified evaluation approach was crucial.

For getting reliable performance measures, all classification algorithms were tested based on Stratified 10-Fold Cross Validation. According to this technique of validation, the preprocessed dataset is split into ten parts, but the class ratio within each of them remains similar to the original dataset. Every time one part is taken as a test set, while the rest nine parts constitute a training set. Such validation process repeats for ten times, and during each run, the part serves as the test set. The final performance is calculated as an average value of all performance measures from all validation runs.

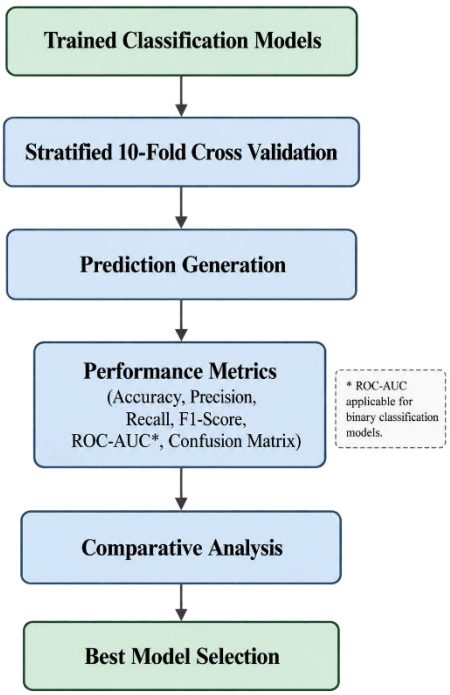


Fig 5.6. Performance Evaluation Framework

The binary classification models, namely Decision Tree, Random Forest, Artificial Neural Network (ANN), and Convolutional Neural Network (CNN), were evaluated using Accuracy, Precision, Recall, F1-Score, Receiver Operating Characteristic–Area Under the Curve (ROC-AUC), and Confusion Matrix analysis. These evaluation measures collectively assess the overall classification performance, the ability to correctly identify malicious traffic, the reduction of false alarms, and the discrimination capability of the developed classifiers.

Similarly, the multiclass classification models, including Decision Tree, Random Forest, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), XGBoost, and Ensemble Learning, were evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix analysis. Since multiclass classification focuses on identifying the correct attack category, these metrics provide a comprehensive assessment of each model's capability to distinguish among multiple intrusion classes while maintaining consistent classification performance across the dataset.

Along with numerical metrics of performance, graphs were used in order to interpret the model behavior in detail. Confusion matrices were drawn for the assessment of the classification performance for separate classes, and ROC curve was investigated for binary classifiers in order to test the discriminatory power of the model at various decision thresholds. Also, accuracy and loss graphs of Deep Learning models were analyzed in order to assess the behavior and overfitting of the learning process.

The performance values obtained from all experimental evaluations were subsequently subjected to a comparative analysis to identify the most effective model for AI-based intrusion detection in IoT networks. Since every model was trained using the same preprocessing pipeline, feature selection strategy, feature scaling technique, class balancing approach, and validation methodology, the comparison reflects the intrinsic performance of the classification algorithms rather than differences in data preparation or evaluation procedures.

6. WORK DONE

6.1. IMPLEMENTATION WORKFLOW

The implementation of the proposed intrusion detection system was carried out by following a structured workflow beginning with the acquisition of the CICIoT2023 dataset and ending with the comparative evaluation of multiple classification models. Each stage of the workflow contributes to improving the quality of the data and enhancing the effectiveness of the intrusion detection system.

Initially, multiple CSV files from the CICIoT2023 dataset were merged to form a unified dataset suitable for analysis. Since the merged dataset contained millions of records, sampling techniques were employed to obtain a representative subset that could be processed efficiently without affecting the distribution of attack classes.

The sampled dataset then underwent data cleaning to remove duplicate records, handle missing values, eliminate infinite values, and improve overall data quality. Exploratory Data Analysis (EDA) was subsequently performed to understand the characteristics of the dataset and identify important statistical properties before model development.

After the pre-processing stage, the Pearson Correlation Coefficient (PCC)-based feature selection method was used to select the most important features for intrusion detection purposes. After that, the selected features were used to build and test the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) classification models. The final step was the comparison of the performance of the three classifiers using standard performance evaluation metrics.

6.2. DATASET COLLECTION

The implementation presented in this project utilizes the CICIoT2023 dataset as the primary source of network traffic for intrusion detection. The dataset was selected because it contains recent IoT communication patterns together with a diverse collection of modern cyberattacks, making it suitable for evaluating Artificial Intelligence-based intrusion detection systems.

The CICIoT2023 dataset is distributed as multiple CSV files, where each file represents network traffic generated under different attack scenarios. Collectively, these files contain millions of network traffic records with numerous numerical features

describing communication characteristics and corresponding class labels representing normal and malicious activities.

Due to the large volume of data, it was not practical to directly train the machine learning and deep learning models using all available files. Therefore, the implementation first focused on organizing and preparing the dataset through merging, sampling, preprocessing, and feature selection before model development.

6.3. DATASET MERGING

The CICIoT2023 dataset is provided in the form of several Comma Separated Value (CSV) files, whereby each CSV file consists of network traffic data for various IoT communication scenarios and types of attacks. Because of the existence of separate files for the CICIoT2023 dataset, it becomes necessary to merge these files first before doing any preprocessing and building the models. It makes working with the dataset easier and more consistent.

All the dataset files were sorted into the Linux server environment at the beginning of the implementation process. The Python code for data processing would then be written to automatically load and check the structure of all the CSV files and append their contents to the same dataframe. Prior to concatenation, all the files were made sure to contain the same features in the same order.

After validating the dataset structure, the CSV files were merged into a single consolidated dataset containing all available network traffic records. The merged dataset preserved the original feature representation while integrating data from multiple attack scenarios into one unified dataset. This consolidated dataset served as the foundation for all subsequent preprocessing, exploratory data analysis, feature selection, and classification stages.

Considering the extremely large size of the merged dataset, directly performing machine learning and deep learning experiments on the complete dataset would require substantial computational resources and processing time. Therefore, after successfully merging all available CSV files, the dataset was prepared for the sampling stage, where a representative subset was generated for efficient experimentation while preserving the original class distribution.

6.4. DATASET SAMPLING

The combined CICIoT2023 dataset comprised about 45 million network traffic entries, making the task of doing data pre-processing, feature selection, and training models computationally costly for such a huge volume of data. The computation of such a big volume of data demands much memory, disk space, and processing time, thus complicating the experiment. As a result, a proper sampling technique was used to form a small sample from the original dataset.

In the present work, a 5% stratified sampling technique was employed to create a manageable dataset for experimentation. Stratified sampling was preferred over random sampling because it preserves the original class distribution of the dataset. Since intrusion detection datasets generally contain multiple attack categories with varying numbers of instances, maintaining the proportion of each class is essential to ensure that the sampled dataset accurately represents the original network traffic.

The sampling was done through Python, whereby each category of attacks was separately sampled depending on the required percentage of sampling. This method made sure that the majority and minority classes were adequately represented in the sample set, thus reducing any sampling bias. Consequently, the obtained dataset became fit for further exploratory data analysis and model development.

Following the sampling process, the generated dataset contained approximately 1.47 million records while retaining the original 40 network traffic features. This reduced dataset significantly lowered computational requirements, enabling efficient preprocessing and model training on the available hardware without compromising the diversity of attack patterns present in the original dataset.

The sampled dataset subsequently served as the input for all remaining stages of the implementation, including data cleaning, exploratory data analysis, Pearson Correlation Coefficient (PCC)-based feature selection, and the training of Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models.

```
✅ Sampling completed successfully!
Sampled dataset saved at:
/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_Sampled_5Percent.csv
```

```
import pandas as pd

sampled = pd.read_csv("/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_Sampled_5.csv")

print (sampled.shape)

sampled['Label'].value_counts()
```

```
(2250933, 40)
```

Figure 6.1. Dataset Sampling and sampled dataset generation

6.5. DATA CLEANING

Cleaning of data is a very vital step in the implementation of machine learning and deep learning algorithms. The quality of data fed to the algorithm determines its performance in terms of accuracy, and therefore it is vital to ensure there are no inconsistencies in the dataset before selecting the features and training the model. Network traffic data in real world contains duplicates, nulls, infinity and wrong values that can influence the learning process.

Once the sampling dataset was created using the CICIoT2023 dataset, the process of data cleaning was done thoroughly. First, the dataset was inspected for any duplicate data entries, which might affect the training process of the model. Duplicate entries were cleaned out to prevent any form of bias in the learning process.

Following duplicate removal, the dataset was analyzed for missing and infinite values. The preprocessing scripts detected a small number of missing values and positive infinite values within a few numerical features. Since these values cannot be directly processed by machine learning and deep learning algorithms, appropriate cleaning operations were performed to replace or remove the affected records. After completing the cleaning process, the dataset was re-evaluated to verify that no missing or infinite values remained.

The cleaning process significantly improved the quality and consistency of the dataset, ensuring that the subsequent stages of exploratory data analysis, feature selection, and model training were performed using reliable and valid network traffic records.

```
Dataset Shape: (2250933, 40)

 NUMBER OF DUPLICATES :  776182

 NUMBER OF DUPLICATES LEFT :  0
```

```
df.to_csv(
        "/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_SAMPLED_5_DUP_REMOVED.csv",
          index = False
          )
```

```
df = pd.read_csv("/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_SAMPLED_5_DUP_REMOVED.csv")

print("Dataset Shape:", df.shape)
print("DUPLICATES :",df.duplicated().sum())
```

```
Dataset Shape: (1474751, 40)
DUPLICATES : 0
```

Figure 6.2. Output of Data Cleaning

6.6. EXPLORATORY DATA ANALYSIS (EDA)

After completing the data cleaning process, Exploratory Data Analysis (EDA) was performed to gain a comprehensive understanding of the characteristics of the cleaned dataset before proceeding with feature selection and model development. EDA plays an important role in identifying the statistical properties of the dataset, understanding the distribution of network traffic records, detecting class imbalance, examining feature relationships, and validating the effectiveness of the preprocessing stage.

The cleaned dataset consisted of 1,474,710 network traffic records with 40 network traffic features, representing both normal and malicious network activities. Various statistical analyses and visualization techniques were employed to examine the quality of the dataset and verify that it was suitable for machine learning and deep learning applications.

The primary objectives of the exploratory data analysis were:

- To understand the structure of the dataset.
- To examine the distribution of attack and normal traffic.
- To identify possible class imbalance.
- To analyse the statistical behaviour of numerical features.
- To understand feature relationships before feature selection.

6.6.1. Dataset Overview

The first stage of EDA involved examining the overall structure of the cleaned dataset. The dimensions of the dataset, feature names, data types, and summary statistics were analysed to verify data consistency after preprocessing.

The analysis confirmed that the cleaned dataset contained 1,474,710 records and 40 numerical features, with no remaining missing or infinite values. The dataset was therefore considered suitable for further statistical analysis and model development.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1474751 entries, 0 to 1474750
Data columns (total 40 columns):
```

Figure 6.3. Dataset Information

6.6.2. Statistical Summary

To understand the behaviour of the numerical features, descriptive statistical measures such as mean, standard deviation, minimum value, maximum value, and quartiles were calculated.

These statistics provided an overview of the range and variability of different network traffic features. Since intrusion detection datasets contain features measured on different scales, the statistical summary also helped determine the need for feature scaling during the preprocessing stage.

	count	mean	std	min	25%	50%	75%	max
Header_Length	1474710.0	14.053424	8.799794	0.000000e+00	8.000000	19.880000	20.000000	6.000000e+01
Protocol Type	1474710.0	10.218475	10.371693	0.000000e+00	6.000000	6.000000	17.000000	4.700000e+01
Time_To_Live	1474710.0	67.850776	17.607711	0.000000e+00	64.000000	64.000000	64.440000	2.550000e+02
Rate	1474710.0	22886.907150	32402.897037	3.973936e-04	6942.947476	16578.276680	30779.364497	7.340032e+06
fin_flag_number	1474710.0	0.075578	0.260457	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
syn_flag_number	1474710.0	0.208103	0.397873	0.000000e+00	0.000000	0.000000	0.010000	1.000000e+00
rst_flag_number	1474710.0	0.084742	0.267301	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
psh_flag_number	1474710.0	0.095662	0.271660	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
ack_flag_number	1474710.0	0.149674	0.328427	0.000000e+00	0.000000	0.000000	0.010000	1.000000e+00
ece_flag_number	1474710.0	0.000042	0.002673	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
cwr_flag_number	1474710.0	0.000022	0.001954	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
ack_count	1474710.0	10.271491	27.653751	0.000000e+00	0.000000	0.000000	1.000000	1.000000e+02

Figure 6.4. Statistical Summary Ouput

6.6.3. Class Distribution Analysis

Understanding the distribution of attack and normal traffic is essential before training classification models. Therefore, the class labels were analysed to determine the proportion of malicious and legitimate network traffic present in the dataset.

The analysis revealed that the dataset is highly imbalanced, with malicious traffic representing the majority of the records while normal traffic accounts for only a small proportion. Such imbalance is commonly observed in intrusion detection datasets and must be considered while interpreting classification results.

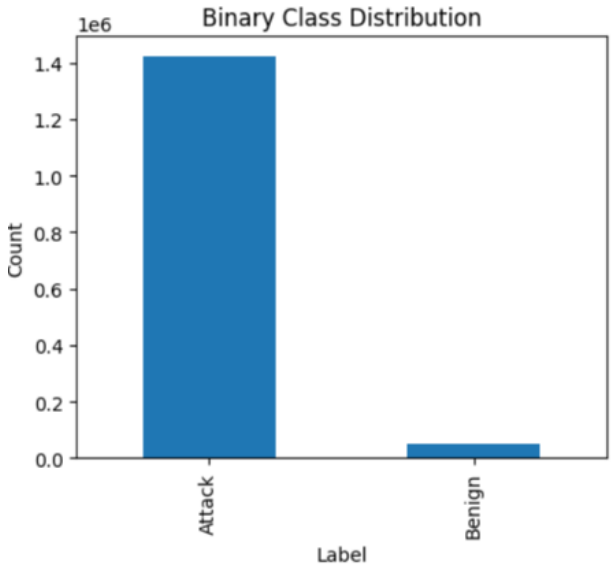


Figure 6.5 Binary Class Distribution

6.6.4. Correlation Analysis

Correlation analysis was performed to understand the relationships among different network traffic features before feature selection. Highly correlated features often contain redundant information and may unnecessarily increase the computational complexity of classification models.

A correlation matrix was generated to visualize the relationships between numerical features. The observations obtained from this analysis supported the subsequent application of the Pearson Correlation Coefficient (PCC) for selecting the most informative features.

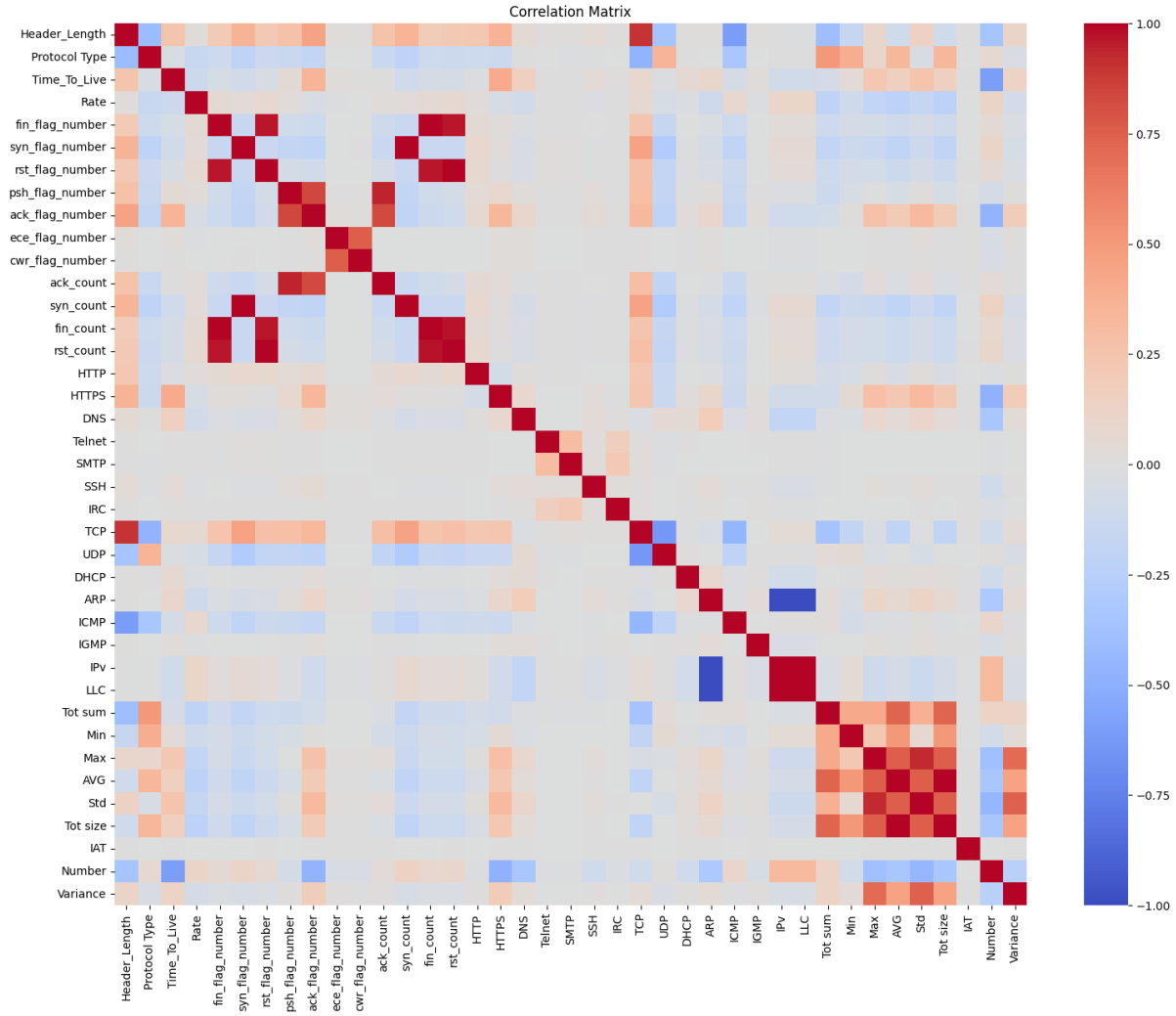


Figure 6.6 Correlation Matrix

6.6.5. Observations from EDA

The exploratory data analysis provided several important observations that influenced the subsequent stages of implementation:

- The cleaned dataset contained no duplicate, missing, or infinite values.
- The dataset consisted of approximately 1.47 million records with 40 network traffic features.
- A significant class imbalance existed between attack and normal traffic.
- Feature values exhibited varying statistical ranges, indicating the importance of appropriate preprocessing.

6.7. BINARY CLASSIFICATION

The original CICIoT2023 dataset contains multiple categories of network traffic representing different cyberattacks together with normal communication records. While multi-class classification enables the identification of specific attack types, the selected research paper focuses on binary intrusion detection, where the primary objective is to determine whether a network traffic instance is normal or malicious.

To maintain consistency with the methodology presented in the selected research paper, the class labels in the CICIoT2023 dataset were converted into a binary representation. During this process, all malicious attack categories were grouped into a single Attack class, while legitimate network traffic was retained as the Normal class. This transformation simplified the classification problem and enabled a direct comparison of the implemented models under a binary intrusion detection framework.

The binary conversion process was implemented using Python by mapping the original labels into two output classes. The Normal class was assigned the label 0, whereas every attack category was assigned the label 1. This approach ensured that the classification models learned to distinguish legitimate network activity from malicious behaviour without considering the individual attack categories.

After completing the binary label conversion, the dataset was re-examined to verify the distribution of both classes. The analysis confirmed that the dataset remained highly imbalanced, with malicious traffic representing the majority of the records. Although class imbalance is a common characteristic of intrusion detection datasets, preserving the original distribution allows the implemented models to be evaluated under realistic network conditions.

The binary dataset generated during this stage served as the final input for the Pearson Correlation Coefficient (PCC) feature selection process and the subsequent training of the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models.

Original Label	Converted Label
Normal	0
All Attack Categories	1

Table 6.1. Binary Label Classification

Class	Label	Number of Records
Normal	0	52,519
Attack	1	1,422,191
Total		**1,474,710**

Table 6.2. Binary Class Distribution

6.8. Pearson Correlation Coefficient (PCC) Feature Selection

Feature selection is an important process in building machine learning or deep learning models especially when dealing with large data sets of intrusion detection with many network traffic features. Redundant features or those that are irrelevant or loosely correlated will only lead to high computational complexity, long training time, and reduced classification accuracy. It is therefore important to choose the most relevant features prior to building the model.

To address this challenge, the present work adopts the Pearson Correlation Coefficient (PCC) feature selection technique proposed in the selected research paper. PCC is a statistical measure that quantifies the strength and direction of the linear relationship between two variables. In this implementation, the correlation between each network traffic feature and the target class was calculated to determine the importance of individual features for intrusion detection.

Correlation coefficient lies in the range of -1 to +1. When the value is close to +1, it shows that there is a high positive correlation, while if the value is close to -1, it depicts high negative correlation. If the value is close to zero, it means there is not much of linear correlation. Characteristics which have higher correlations with the class of interest are important for intrusion detection.

6.8.1. PCC Implementation

Once data pre-processing and binarization of labels had been done, the Pearson Correlation Coefficient was determined for each numerical attribute that appeared in the processed dataset. Each attribute was examined separately in order to find out its importance in differentiating normal and malicious traffic.

Based on the computed correlation values, a predefined threshold was applied to identify the most relevant features. Features with correlation values satisfying the threshold criterion were retained, while weakly correlated attributes were removed from the dataset. This reduced the dimensionality of the dataset and eliminated redundant information before training the classification models.

The implementation followed the same feature selection methodology presented in the selected research paper, thereby ensuring consistency between the proposed framework and the present work.

6.8.2. Feature Ranking

The correlation analysis produced a ranked list of network traffic features according to their relationship with the target class. Features with higher correlation coefficients were considered more significant for intrusion detection. This ranking helped identify the attributes that contributed most to distinguishing between normal and malicious network activities.

The correlation coefficient indicates the strength and direction of the relationship between an individual feature and the target class. A positive correlation suggests that an increase in the feature value is generally associated with the presence of a particular attack class, while a negative correlation indicates an inverse relationship. Features with correlation values close to zero were considered to have a weaker linear relationship with the target variable.

Feature ranking also helps reduce the dimensionality of the dataset by identifying less informative attributes. Removing irrelevant or redundant features can reduce computational complexity, shorten training time, and potentially improve model performance. However, correlation alone does not necessarily indicate that a feature is independently important, since two highly correlated features may contain similar information.

6.8.3. Selected Features

Based on the selected threshold, the final feature subset consisted of the most informative attributes required for intrusion detection. These selected features were used as the input for all classification models implemented in this project.

During the feature selection process, attributes exhibiting very low correlation with the target class were removed. Eliminating these redundant features reduced computational complexity while preserving the information necessary for accurate intrusion detection.

Following PCC feature selection, the dataset dimensionality was reduced from 40 original features to 15 highly relevant features, which were subsequently used for model development.

6.8.4. Observations

The implementation of Pearson Correlation Coefficient feature selection successfully reduced the dimensionality of the dataset while retaining the most informative network traffic features. The reduction in feature space decreased computational requirements during model training and simplified the learning process for both machine learning and deep learning models.

The selected feature subset served as the common input for the Random Forest, Artificial Neural Network, and Convolutional Neural Network classifiers. Consequently, all three models were trained and evaluated using an identical feature set, ensuring a fair and consistent comparison of their intrusion detection performance.

6.9. BINARY CLASS IMPLEMENTATION

6.9.1. Decision Tree

In the implementation of the AI-based Intrusion Detection System (IDS), the Decision Tree classifier was chosen as the baseline Machine Learning algorithm for the binary classification of intrusions. One of the most common algorithms for classification tasks is the Decision Tree algorithm because of its simplicity, quick execution time, and the generation of easily understood decision rules. The Decision Tree algorithm was used in the proposed IDS for classification of network traffic samples into two classes – normal and attack depending on the selected network traffic features.

The Decision Tree classifier was implemented through the use of the binary dataset that had been derived from the feature selection via Pearson Correlation Coefficient (PCC), feature scaling via StandardScaler, and class balancing via Synthetic Minority Oversampling Technique (SMOTE). The preprocessed feature set acted as the input of the classifier, while the binary labels of the classes acted as the outputs. As the algorithm iteratively determined the best splitting criterion, it would build a tree structure that would be able to differentiate between normal and malicious network data. The same preprocessing framework was kept intact in the process for consistency with other classification techniques.

The Decision Tree approach served as an efficient computational starting point for intrusion detection using two classes and created a point of reference for the evaluation of Machine Learning and Deep Learning techniques. Due to the comprehensible nature of its algorithm and ease of implementation, the classifier helped understand the impact of specific traffic features on the process of intrusion detection. Despite the fact that some more complex algorithms were considered during the research, the Decision Tree still served as an important point of comparison.

After training, the model was evaluated on the testing dataset using standard classification metrics, including Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix. These metrics provide a comprehensive assessment of the classifier's ability to correctly identify malicious network traffic while minimizing false classifications.

Library	**Purpose**
Scikit-learn	Random Forest implementation
Pandas	Dataset handling
NumPy	Numerical operations
Matplotlib	Result visualization
Joblib	Model serialization

Table 6.3. Libraries used for Random Forest

The implementation generated the following outputs:

- Trained Random Forest model.
- Predicted class labels.
- Confusion Matrix.
- Classification Report.
- Accuracy, Precision, Recall, and F1-Score.
- Model execution statistics.

These outputs were stored for comparative analysis with the ANN and CNN models.

6.9.2. Random Forest

The Random Forest classifier algorithm is one of the Machine Learning models used for binary intrusion detection. Contrary to the Decision Tree model, the Random Forest algorithm creates several decision trees during the training procedure and generates the output based on the results from all decision trees through majority voting. The combination of numerous decision trees enhances the reliability of classification due to decreased variance and overfitting risk; thus, the Random Forest algorithm is highly effective for working with large-scale traffic datasets. In the context of the suggested AI-based IDS, the Random Forest algorithm was designed to classify the traffic as Normal or Attack based on the features extracted during the pre-processing stage.

The deployment of the Random Forest classifier was done using the binary dataset that was developed through feature selection using the Pearson Correlation Coefficient (PCC), feature scaling using the StandardScaler method, and SMOTE class balancing. The network traffic features that were chosen acted as the inputs to the classifier while the binary labels were used for the target outputs. In the process of training, the Random Forest algorithm created several bootstrap samples of the training dataset, thereby ensuring that each decision tree had access to different parts of the training dataset.

Every decision tree in the Random Forest separately analyzed the selected features of the network traffic and made its own prediction of the class of the traffic data. The prediction of the final class was made through a majority voting scheme, where the class that received the most votes among all decision trees was chosen as the final

prediction. Such a voting scheme minimized the effect of errors of separate trees and allowed for the classifier to make a more reliable prediction than using only one Decision Tree.

The Random Forest model was implemented using the Scikit-learn machine learning library within the Python programming environment. Similar to the Decision Tree classifier, the model was trained using the standardized and balanced binary dataset and evaluated using Stratified 10-Fold Cross Validation. Employing the same validation strategy for all classification models ensured consistency throughout the implementation process and provided a fair basis for subsequent experimental comparison.

The Random Forest implementation formed an important component of the proposed intrusion detection framework by combining the interpretability of decision tree learning with the improved robustness offered by ensemble techniques. Its ability to analyse complex feature interactions while reducing overfitting makes it well suited for cybersecurity applications involving large-scale IoT network traffic.

6.9.3. Artificial Neural Network (ANN)

Artificial Neural Network (ANN) was among the Deep Learning techniques used for binary intrusion detection in the designed Artificial Intelligence (AI) based Intrusion Detection System. In contrast to the conventional Machine Learning approaches which use clearly defined decision boundaries, ANN models learn complicated nonlinear features from the data directly without the need of any decision boundaries due to the interconnected layers of artificial neurons. Due to this capability of learning complex features, ANN is an appropriate model for detecting complicated patterns from the network traffic data which makes it easy to differentiate legitimate traffic from the intrusion attempts.

The implementation of the ANN model utilized the binary dataset obtained after Pearson Correlation Coefficient (PCC)-based feature selection, StandardScaler-based feature scaling, and Synthetic Minority Oversampling Technique (SMOTE) class balancing. The standardized feature vectors were supplied to the input layer of the neural network, where they were processed through multiple fully connected hidden layers. During training, the network progressively learned representative feature patterns by adjusting the weights of the interconnected neurons through iterative

optimization, enabling accurate classification of network traffic into Normal and Attack categories.

The implemented network architecture consisted of an input layer, multiple fully connected hidden layers, and an output layer designed for binary classification. The hidden layers employed nonlinear activation functions to capture complex relationships among the selected network traffic features, while dropout layers were incorporated between successive hidden layers to reduce overfitting and improve the model's generalization capability. The output layer utilized the Sigmoid activation function to produce a probability score representing the likelihood of a network traffic instance belonging to the attack class.

The artificial neural network was modeled using the TensorFlow and Keras deep learning libraries with the use of the Python programming language. This process used iterative optimization through forward and backpropagation, enabling the network to keep updating its parameters with the help of prediction errors. The loss function Binary Cross Entropy was applied in the case of binary classification, whereas the optimizer used was Adam optimizer to ensure quick and effective convergence. The model was also evaluated using Stratified 10-Fold Cross Validation, which ensured that each validation fold retained the same proportion of classes as before.

The implementation of the Artificial Neural Network demonstrated the applicability of Deep Learning techniques for IoT intrusion detection by enabling automatic extraction of complex feature relationships from standardized network traffic data. The incorporation of multiple hidden layers and nonlinear activation functions allowed the network to learn sophisticated decision boundaries beyond those achievable using conventional Machine Learning algorithms.

The implementation of the ANN model provides a deep learning baseline for comparison with the Random Forest classifier and the Convolutional Neural Network implemented in this project.

Library	Purpose
TensorFlow	Deep learning framework
Keras	ANN model development
NumPy	Numerical computation
Pandas	Dataset manipulation
Scikit-learn	Data splitting and evaluation
Matplotlib	Training visualization

Table 6.4. Libraries used for ANN

The ANN implementation generated the following outputs:

- Trained ANN model.
- Predicted binary class labels.
- Training and validation loss.
- Training and validation accuracy.
- Confusion Matrix.
- Classification Report.

These outputs were saved for comparison with the Random Forest and Convolutional Neural Network models.

The Artificial Neural Network successfully learned the underlying patterns present in the selected network traffic features and produced binary predictions for normal and malicious network activities. The model demonstrated the capability of deep learning techniques to capture complex relationships within IoT network traffic. The final performance of the ANN model was evaluated using standard classification metrics and later compared with the Random Forest and Convolutional Neural Network models to determine the most effective approach for IoT intrusion detection.

6.9.4. Convolutional Neural Network (CNN)

The Convolutional Neural Network (CNN) was implemented as the second Deep Learning model for binary intrusion detection within the proposed Artificial Intelligence (AI)-based Intrusion Detection System. Although CNNs are traditionally associated with image processing applications, one-dimensional convolutional neural networks have demonstrated significant effectiveness in analysing sequential and structured data. In this work, a 1D Convolutional Neural Network (Conv1D) architecture was employed to automatically learn discriminative feature representations from the selected network traffic attributes. The ability of convolutional layers to capture local feature dependencies enables the model to identify complex intrusion patterns that may not be easily detected by conventional Machine Learning algorithms.

The adoption of the CNN model was done based on the binary dataset resulting from feature selection using the Pearson Correlation Coefficient (PCC), feature scaling using the StandardScaler method, and class balancing using Synthetic Minority Oversampling Technique (SMOTE). The normalized feature vectors acted as the input to the neural network enabling the convolution layers to analyze the relations among neighboring features. The automatic feature extraction ability of the CNN model helped in reducing the dependence on the manually engineered representation of features.

The CNN architecture that was adopted comprised of an input layer followed by convolutional layers whose role was to extract feature patterns from the input. Nonlinear activation functions were applied following each step of the convolution operation to enable the network to learn complex interactions between the network traffic features. The max pooling operation was applied in order to decrease the dimensionality of the feature map and retain the important information. Dropout layers were applied in order to avoid overfitting.

After the completion of the feature extraction phase, the extracted features were converted into a one-dimensional vector via a flattening procedure prior to being inputted to fully connected dense layers that would be responsible for the subsequent classification task. The output layer made use of the Sigmoid activation function in order to compute the probability of each network traffic sample as belonging to either the Normal or Attack category.

CNN model was designed using deep learning tools namely TensorFlow and Keras in Python language environment. The training process of the neural network was carried out through forward propagation of producing predictions and backpropagation for updating the parameters of the network based on minimizing the Binary Cross Entropy Loss Function. For effective training of the model, the optimization method called Adam was used. In the same manner as other binary classifiers models, Stratified 10-Fold Cross Validation was utilized for training consistency.

The implementation of the Convolutional Neural Network demonstrated the applicability of deep convolutional learning techniques for intrusion detection in IoT environments. By automatically extracting hierarchical feature representations from standardized network traffic data, the CNN model provided a robust learning framework capable of identifying complex intrusion patterns without requiring extensive manual feature engineering.

Library	**Purpose**
TensorFlow	Deep learning framework
Keras	CNN model implementation
NumPy	Numerical computations
Pandas	Dataset handling
Scikit-learn	Data preprocessing and evaluation
Matplotlib	Visualization of results

Table 6.5. Libraries Used for CNN

The CNN implementation generated the following outputs:

- Trained CNN model.
- Predicted binary class labels.
- Training and validation accuracy.
- Training and validation loss.
- Confusion Matrix.

- Classification Report.
- Accuracy, Precision, Recall, and F1-Score.

These outputs were recorded for comparison with the Random Forest and Artificial Neural Network models in the Results chapter.

The Convolutional Neural Network successfully learned the complex relationships present within the selected network traffic features and produced binary predictions for normal and malicious network traffic. By automatically extracting discriminative feature representations during training, the CNN provided a robust deep learning approach for intrusion detection. The final performance of the CNN model was evaluated using standard classification metrics and compared with the Random Forest and Artificial Neural Network models to determine the most effective classifier for the proposed intrusion detection framework.

6.10. MULTICLASS CLASSIFICATION IMPLEMENTATION

Following the successful implementation of the binary intrusion detection models, the proposed Artificial Intelligence (AI)-based Intrusion Detection System was extended to perform multiclass intrusion detection. Unlike binary classification, where all malicious network traffic is grouped into a single attack class, multiclass classification preserves the original attack labels present in the CICIoT2023 dataset. This enables the developed models to identify the specific type of intrusion responsible for malicious network activity, thereby providing more detailed information that can support attack analysis, incident response, and network security management within Internet of Things (IoT) environments.

The development of the multiclass classification pipeline involved using the same preprocessed data set that was obtained by applying the following preprocessing steps: PCC-based feature selection, StandardScaler-based feature scaling, and SMOTE-based class balancing. Using the same preprocessing pipeline for all experiments provided consistency and prevented variations caused by using different data preprocessing procedures. The processed feature matrix was used as the input for the models, while the attack labels were used as the target for the output.

To investigate the effectiveness of different Artificial Intelligence techniques, six classification models were implemented for multiclass intrusion detection, namely Decision Tree, Random Forest, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), XGBoost, and Ensemble Learning. These models represent a combination of traditional Machine Learning, Deep Learning, gradient boosting, and ensemble-based learning approaches. Implementing multiple classifiers enabled a comprehensive evaluation of different learning paradigms for recognizing diverse attack categories within IoT network traffic.

All multiclass classification models were implemented using the Python programming language together with widely adopted Machine Learning and Deep Learning libraries. Similar to the binary classification implementation, Stratified 10-Fold Cross Validation was employed to ensure that the distribution of attack categories remained consistent across all validation folds. Using the same validation strategy for every classifier provided a uniform implementation framework and established a fair basis for the comparative performance analysis presented in the subsequent chapter.

6.10.1. Decision Tree

The Decision Tree was designed for multiclass intrusion detection in order to classify the particular type of attacks present in each network traffic case. The multiclass version of Decision Tree is used because multiclass detection is required. In binary classifiers, all the attacks are classified under one group, but in multiclass versions, the attack types are maintained as they were in the CICIoT2023 dataset.

The implementation utilized the preprocessed dataset obtained after Pearson Correlation Coefficient (PCC)-based feature selection, StandardScaler-based feature scaling, and Synthetic Minority Oversampling Technique (SMOTE) class balancing. The selected network traffic features were provided as input to the Decision Tree, while the original attack labels served as the target output.

The Decision Tree model was implemented using the Scikit-learn library within the Python programming environment. Stratified 10-Fold Cross Validation was employed during training to preserve the class distribution across all validation folds and ensure a consistent evaluation framework.

6.10.2. Random Forest

The Random Forest algorithm has been employed in multiclass intrusion detection where the network traffic is classified based on the respective attack types found in the CICIoT2023 data set. As an ensemble learning algorithm, Random Forest makes use of several decision trees to make the final prediction. This allows the model to create complex boundaries while still maintaining stability for different attacks.

The implementation utilized the preprocessed multiclass dataset obtained after PCC-based feature selection, StandardScaler-based feature scaling, and SMOTE class balancing. During training, multiple decision trees were constructed using bootstrap samples of the dataset, and each tree independently learned the relationships between the selected network traffic features and the attack categories. The final prediction was obtained through a majority voting mechanism among all decision trees, resulting in a more reliable multiclass classification model.

The Random Forest model was implemented using the Scikit-learn library in Python and trained using Stratified 10-Fold Cross Validation to maintain consistent class distribution across all folds.

6.10.3. Artificial Neural Network (ANN)

The Artificial Neural Network (ANN) was implemented to perform multiclass intrusion detection by learning complex nonlinear relationships among the selected network traffic features. Unlike the binary implementation, the multiclass ANN was designed to classify each network traffic instance into its corresponding attack category, enabling more detailed intrusion analysis within IoT environments.

The network was trained using the standardized and SMOTE-balanced multiclass dataset produced after the preprocessing and feature engineering stages. Multiple fully connected hidden layers were employed to extract meaningful feature representations, while the output layer generated probability scores for each attack class using an appropriate multiclass activation function. The learning process was carried out through iterative forward propagation and backpropagation, allowing the network to optimize its internal parameters during training. The ANN model was implemented using the TensorFlow and Keras frameworks within Python and trained using Stratified 10-Fold Cross Validation.

6.10.4. Convolutional Neural Network (CNN)

The Convolutional Neural Network (CNN) was implemented to perform multiclass intrusion detection by automatically extracting hierarchical feature representations from the selected network traffic attributes. A one-dimensional convolutional architecture was employed to learn local feature relationships and distinguish among multiple attack categories without relying solely on manually engineered feature interactions.

The architecture was based on the standardized multiclass dataset derived from the feature selection using PCC and class balancing using SMOTE. The convolutional layer helped to extract discriminative feature patterns, whereas pooling and fully connected layers transformed them into multiclass classifications. Such an architecture allowed the neural network to learn complicated behavioral patterns for various intrusion classes.

The CNN model was developed using the TensorFlow and Keras frameworks and trained using Stratified 10-Fold Cross Validation to ensure consistency throughout the implementation.

6.10.5. XGBoost

The XGBoost algorithm has been employed as an efficient Machine Learning technique to build a multiclass intrusion detection model in the proposed AI-Based Intrusion Detection System. The XGBoost algorithm is an optimized version of a gradient boosting algorithm, which generates decision trees one after another, with each new tree concentrating on rectifying the mistakes made by previous trees. In contrast to traditional tree models, XGBoost not only employs gradient boosting but also incorporates techniques of regularization, which allow it to obtain better predictive performance while avoiding overfitting issues.

This implementation used the multiclass data generated by performing feature selection using PCC, feature scaling using StandardScaler, and SMOTE for class balancing. The selected network traffic features were used as the input for the XGBoost classifier, and the original attack categories were used as the output labels. The iterative process involved in building the classifier entailed building a series of decision trees where each

next tree learns from the errors of the previous model. Such a sequential process helped the classifier learn progressively.

A significant advantage of XGBoost is its ability to incorporate regularization during the learning process, which improves model generalization and reduces the likelihood of overfitting. In addition, the algorithm supports efficient handling of large datasets through parallel tree construction and optimized memory utilization, making it particularly suitable for the large-scale CICIoT2023 dataset used in this work. These characteristics allowed the model to effectively learn complex relationships among the selected network traffic features while maintaining computational efficiency throughout the training process.

The XGBoost classifier was implemented using the XGBoost library within the Python programming environment and trained using Stratified 10-Fold Cross Validation to ensure consistent class distribution across all validation folds. The implementation followed the same preprocessing and validation framework adopted for the remaining classification models, thereby enabling a fair comparison under identical experimental conditions.

6.10.6. Ensemble Learning

The Ensemble Learning model represents the final stage of the proposed multiclass intrusion detection framework. Rather than relying on the prediction of a single classifier, the ensemble approach combines the outputs of multiple Machine Learning and Deep Learning models to produce a final prediction. This strategy leverages the strengths of individual classifiers while reducing their respective limitations, resulting in a more robust and reliable intrusion detection system.

The ensemble model integrated the predictions generated by the individual multiclass classifiers implemented in this study. By aggregating these predictions through an ensemble decision mechanism, the framework was able to improve classification stability and enhance the overall capability of identifying diverse attack categories. The ensemble implementation maintained the same preprocessing pipeline and validation strategy adopted throughout the proposed framework, ensuring consistency across all classification models. The Ensemble Learning model was implemented within the Python environment and evaluated using Stratified 10-Fold Cross Validation.

7. RESULTS AND DISCUSSION

7.1. CNN

7.1.1. Architecture

Stage	Module	Output Shape	Kernel	Stride	Parameters	% Total	MACs
0	Conv1D	(None, 15, 96)	4	1	480	0.14	-
1	Conv1D	(None, 15, 64)	3	1	18496	5.45	-
2	Conv1D	(None, 15, 32)	2	1	4128	1.22	-
3	Flatten	(None, 480)	-	-	0	0.0	-
4	Dense	(None, 512)	-	-	246272	72.6	-
5	Dense	(None, 128)	-	-	65664	19.36	-
6	Dense	(None, 32)	-	-	4128	1.22	-
7	Dense	(None, 1)	-	-	33	0.01	-

Fig 7.1. CNN Architecture

7.1.2. Hyperparameter Configuration

Hyperparameter	Value
Input Features	15
Optimizer	Adam
Learning Rate	0.001
Epochs	20
Batch Size	64
Loss Function	Binary Crossentropy
Conv1 Filters	96
Conv1 Kernel	4
Conv2 Filters	64
Conv2 Kernel	3
Conv3 Filters	32
Conv3 Kernel	2
Dense Layer 1	512
Dense Layer 2	128
Dense Layer 3	32
Output Activation	Sigmoid

Fig 7.2. Hyperparameters

7.1.3. Training Performance

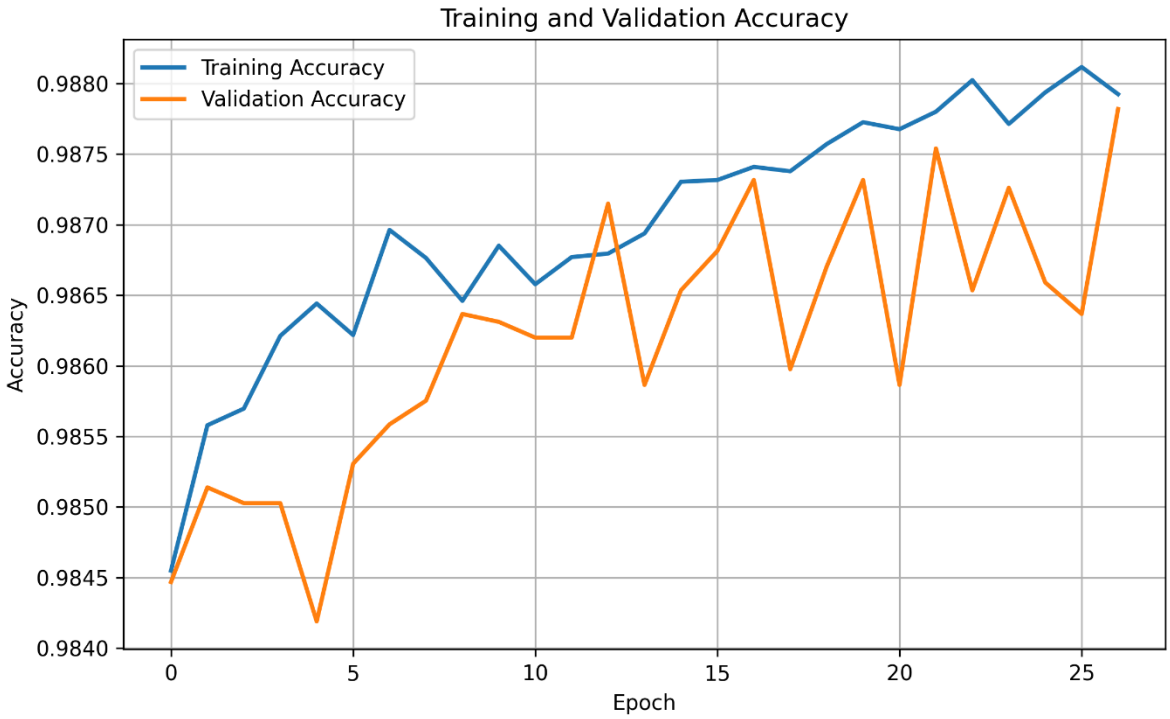


Figure 7.3. Training and Validation Accuracy over epochs

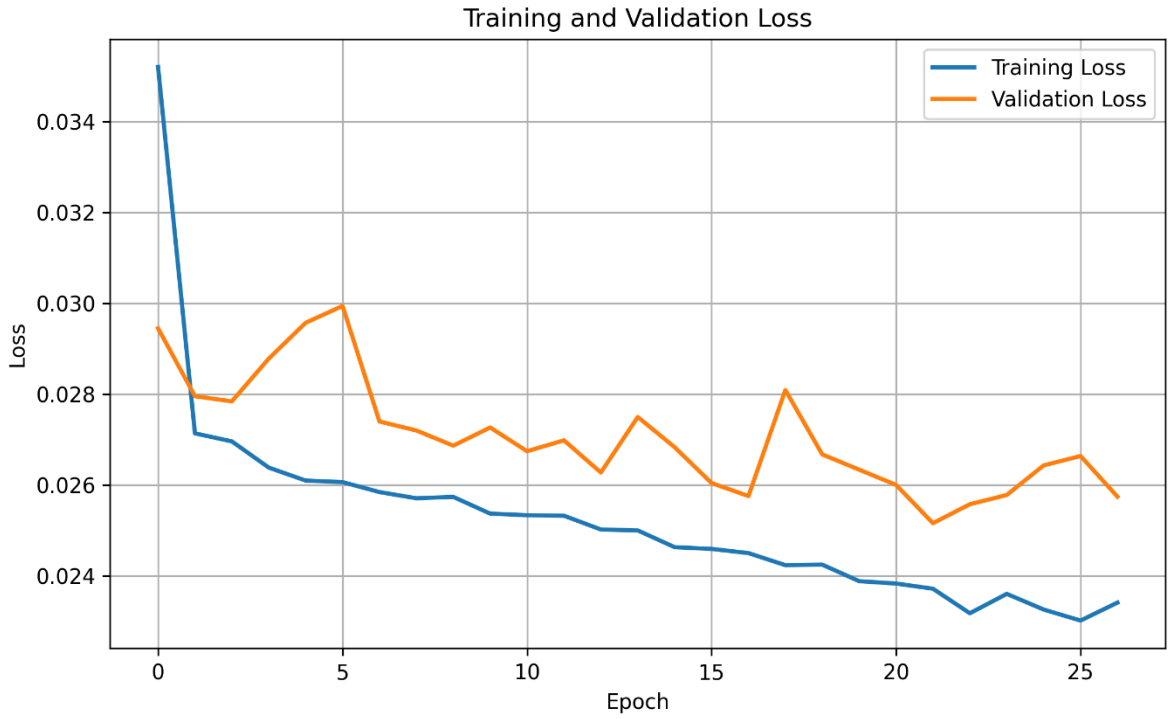


Figure 7.4. Training and Validation Loss over epochs

7.1.4. Cross Validation Results

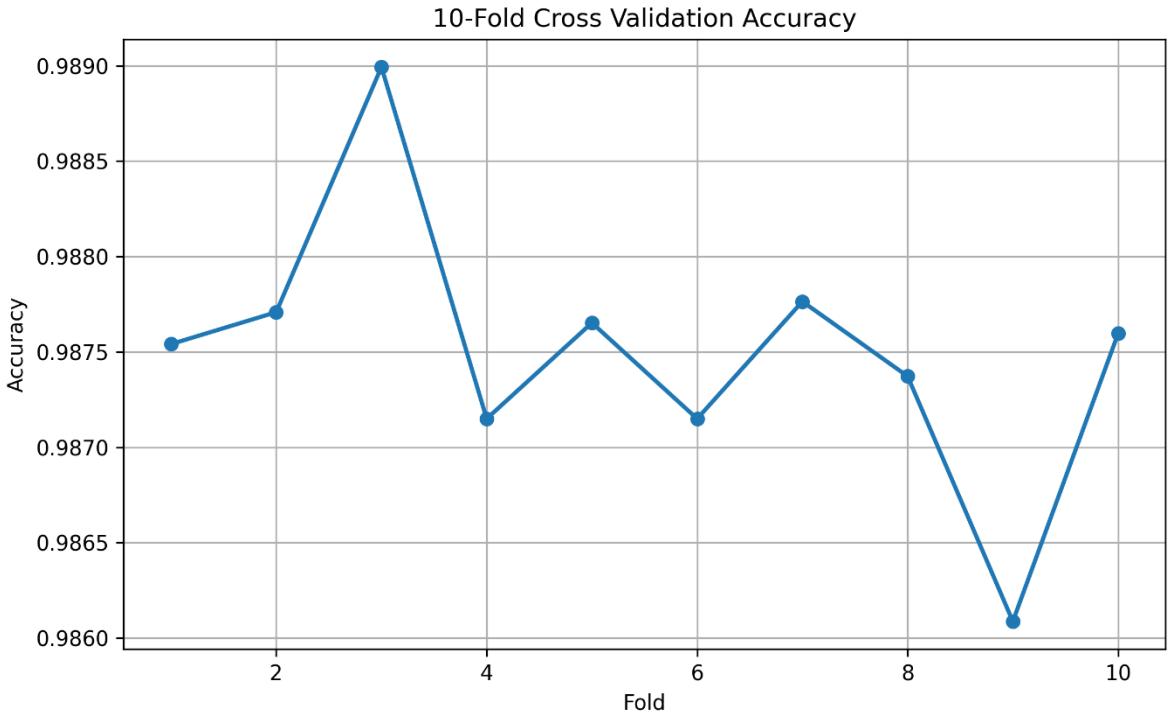


Figure 7.5. Accuracy obtained across 10 Folds

7.1.5. Quantitative Performance

Metric	Mean	Std
Accuracy	0.987501179	0.000717648
Balanced Accuracy	0.876193227	0.012983422
Precision	0.994214858	0.000632967
Sensitivity	0.992979702	0.001015329
F1	0.993596372	0.00037025
Macro F1	0.86693722	0.006469721
AUC	0.995647856	0.000363316
Specificity	0.759406751	0.026656187

Table 7.1. Evaluation Metrics

7.1.6. ROC Analysis

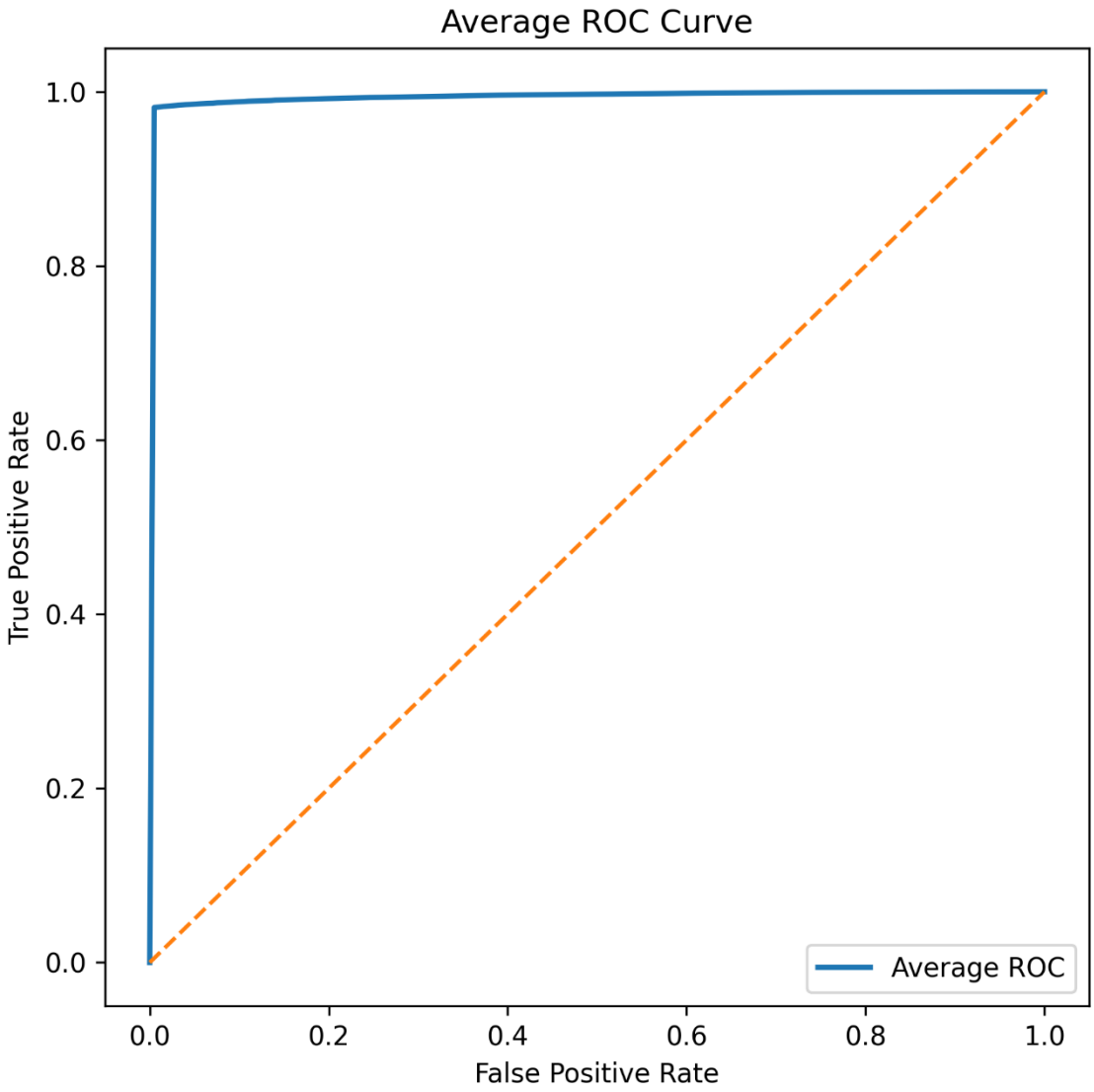


Figure 7.6. Average ROC curve across the 10 – fold curve.

7.1.7. Confusion Matrix

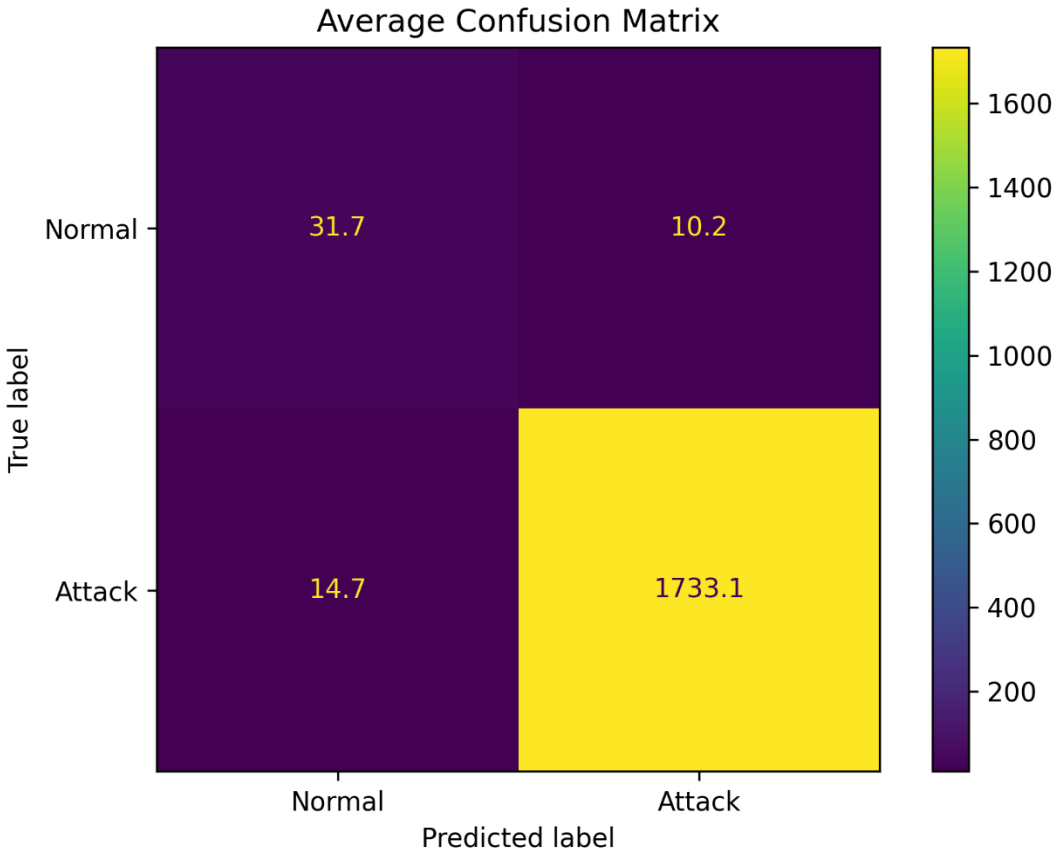


Figure 7.7. Average Confusion Matrix over 10 validation folds.

7.1.8. Average Metric Comparison

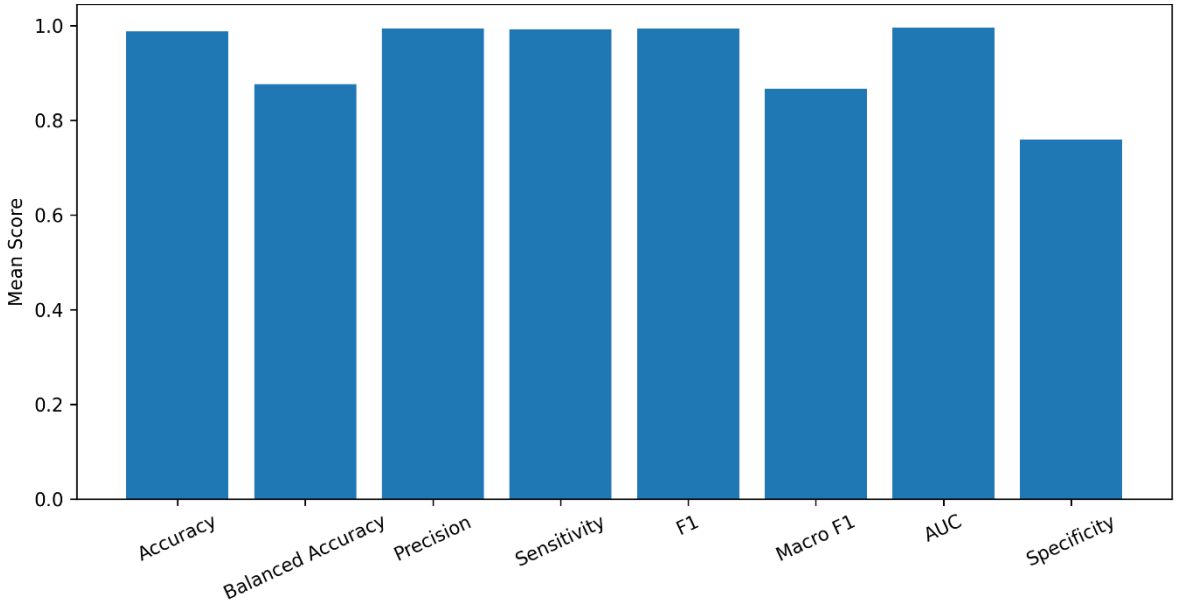


Figure 7.8. Metric Comparison

7.1.9. t - SNE

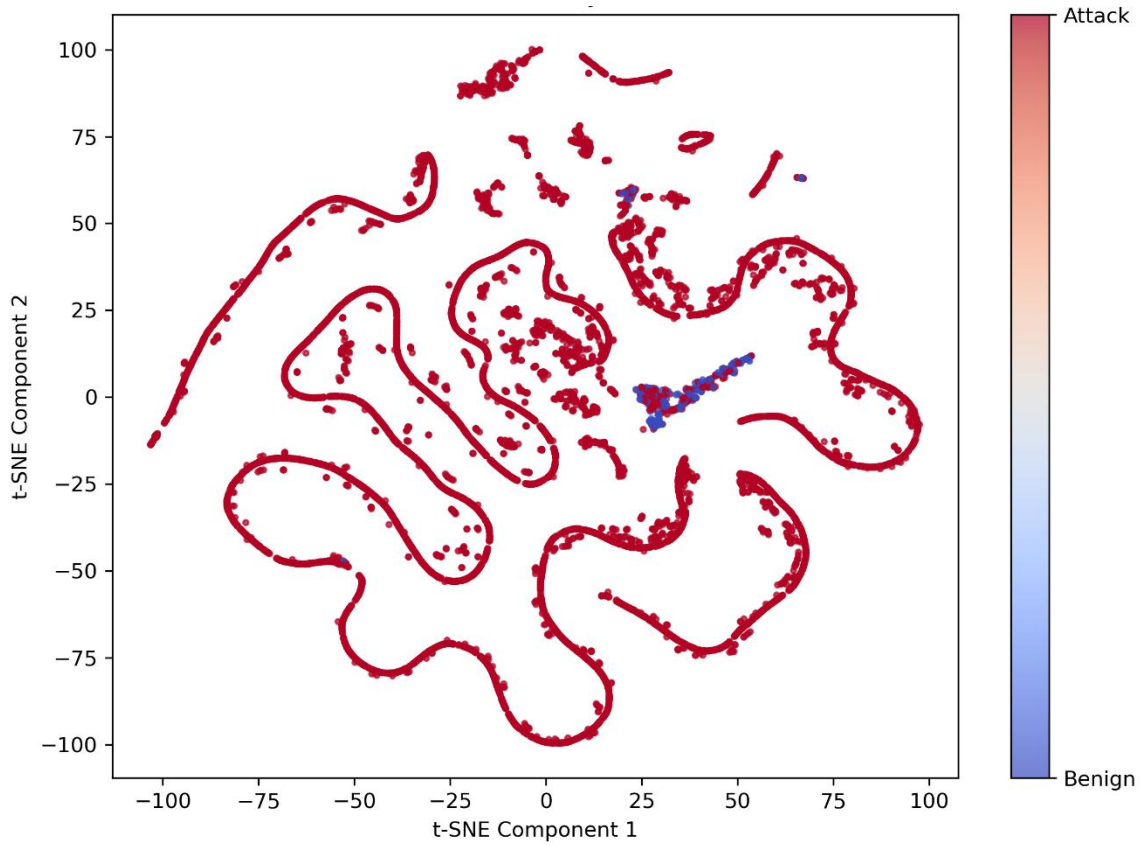


Figure 7.9 Two-dimensional t-SNE visualization of the binary intrusion dataset

7.1.10. Discussion

In terms of binary intrusion detection, the Convolutional Neural Network (CNN) gave an outstanding performance of 98.75% for average accuracy, 99.42% for precision, 99.30% for sensitivity (or recall), 99.36% for F1-score, and 99.56% for the area under curve (AUC) in 10-fold cross-validation. The low standard deviation for all metrics means that the model generated stable and reliable predictions in all folds. The ROC curve stays close to the upper left corner, which proves good discrimination power between benign and malicious traffic. The average confusion matrix showed that the CNN correctly classified most attack examples with minimal false positives and false negatives. The lower specificity (75.94%) as compared to sensitivity shows that the classification of benign traffic was more difficult because of the class imbalance issue in the data, when attacks were much more than normal. The convolutional layers successfully generated local feature patterns of the selected network features, while the dense layers made high-level classification.

7.2. ANN

7.2.1. Architecture

Stage	Module	Output Shape	Activation	Parameters
0	Input	(15)	-	0
1	Dense	(128)	ReLU	2048
2	Dense	(64)	ReLU	8256
3	Dense	(32)	ReLU	2080
4	Dense	(1)	Sigmoid	33

Figure 7.10 ANN Architecture

7.2.2. Hyperparameter Configuration

Hyperparameter	Value
Input Features	15
Optimizer	Adam
Learning Rate	0.001
Epochs	20
Batch Size	64
Loss Function	Binary Crossentropy
Dense Layer 1	128 ReLU
Dense Layer 2	64 ReLU
Dense Layer 3	32 ReLU
Output Layer	Sigmoid

Figure 7.11. Hyperparameters

7.2.3. Training Accuracy

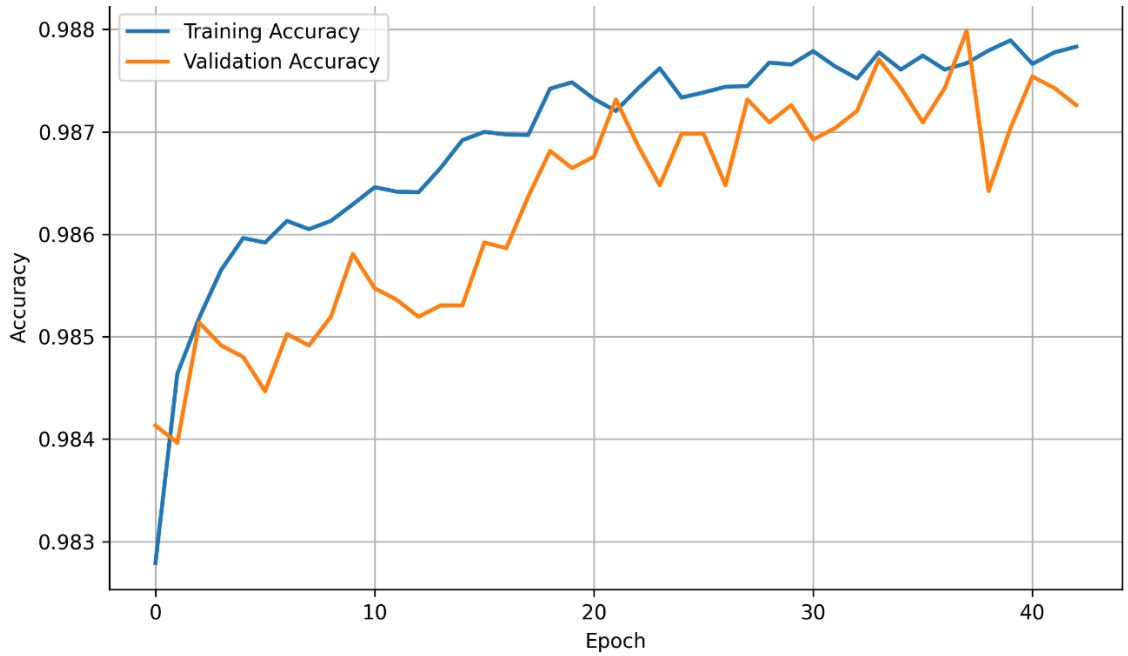


Figure 7.12 Average Training and Validation across 10-fold cross validation

7.2.4. Training Loss

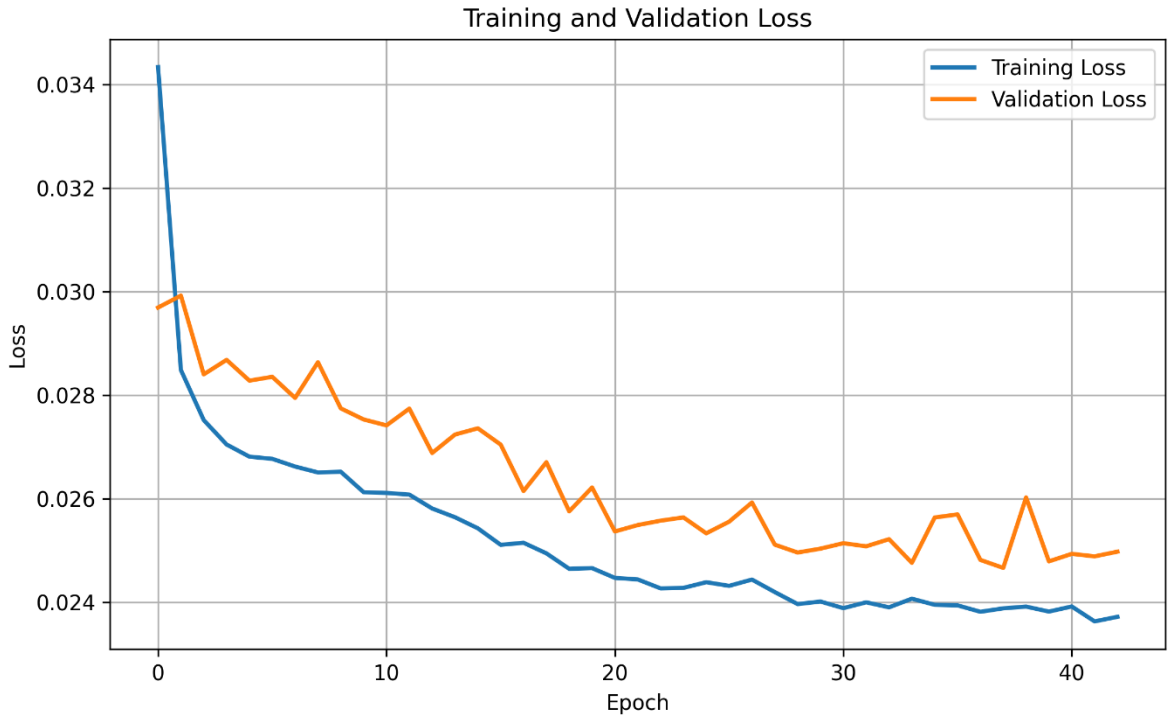


Figure 7.13 Average Training and Validation Loss over 10-fold cross validation

7.2.5. Fold Accuracy

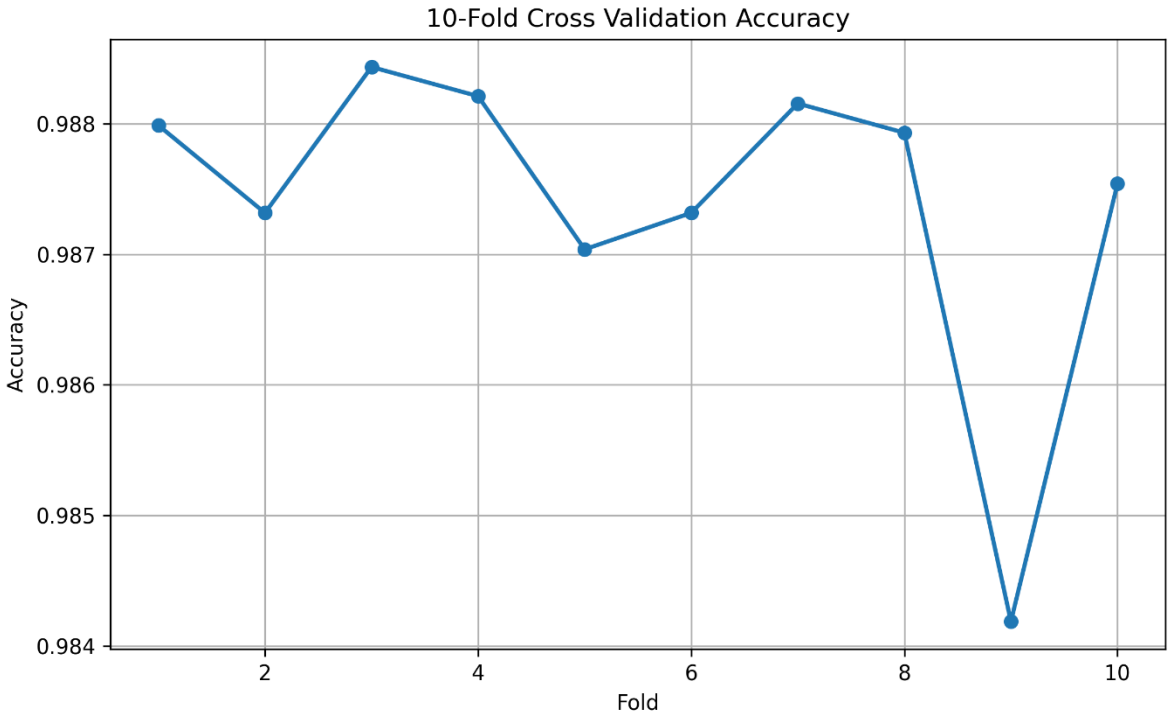


Figure 7.14. Fold Accuracy over 10 Folds

7.2.6. Performance Metrics

Experiment Summary

Metric	Mean	Std
Accuracy	0.987489995723221	0.0009859214401362
Balanced Accuracy	0.889203273944948	0.0198802791359088
Precision	0.994850366280784	0.0009605572347773
Recall	0.99232744799218	0.0010957775755908
F1	0.9935865752207136	0.0005058954377463
Macro F1	0.8700348719566142	0.010898940709801
AUC	0.995517999572184	0.0005852016192289
Specificity	0.7860790998977156	0.0403228720402634

Figure 7.15. Evaluation Metrics

7.2.7. ROC Curve

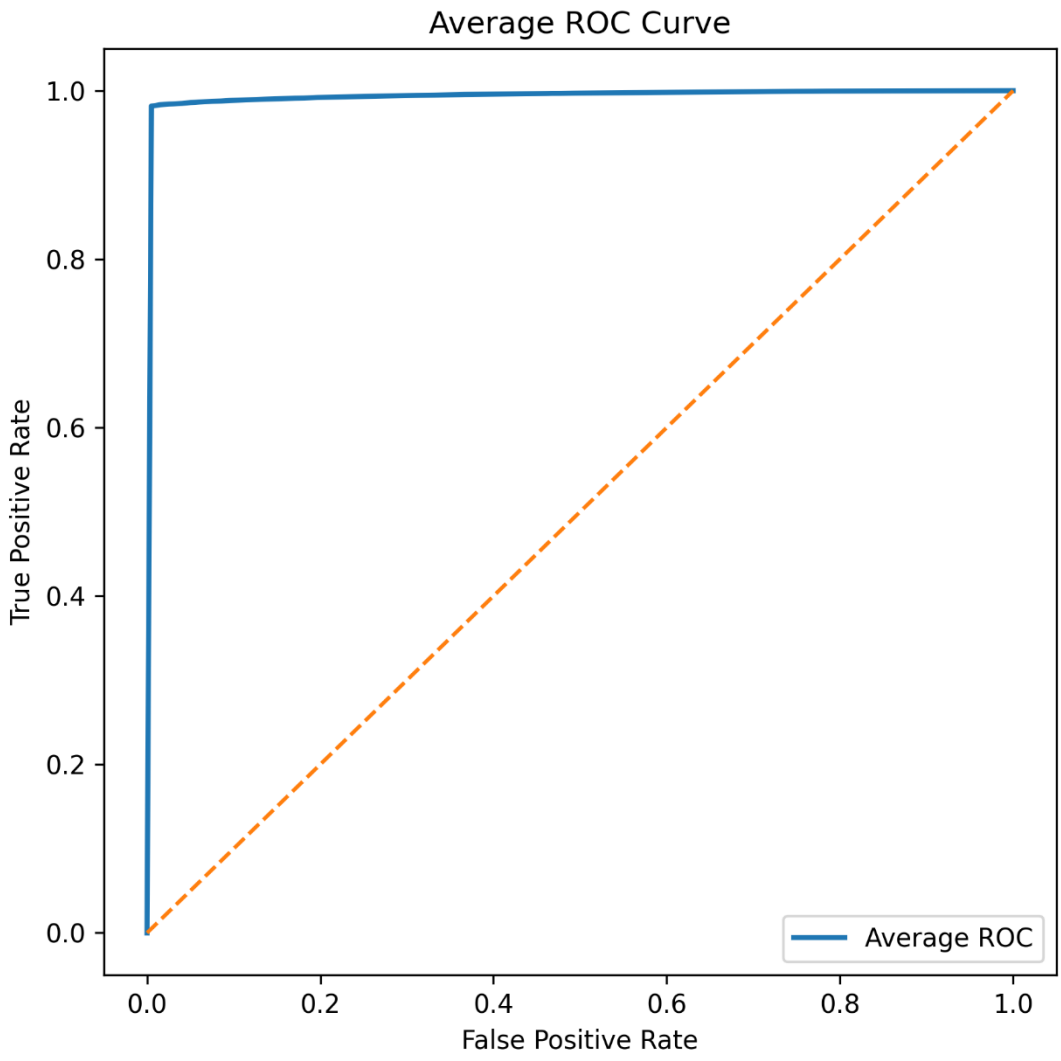


Figure 7.16 Average ROC curve

7.2.8. Confusion Matrix

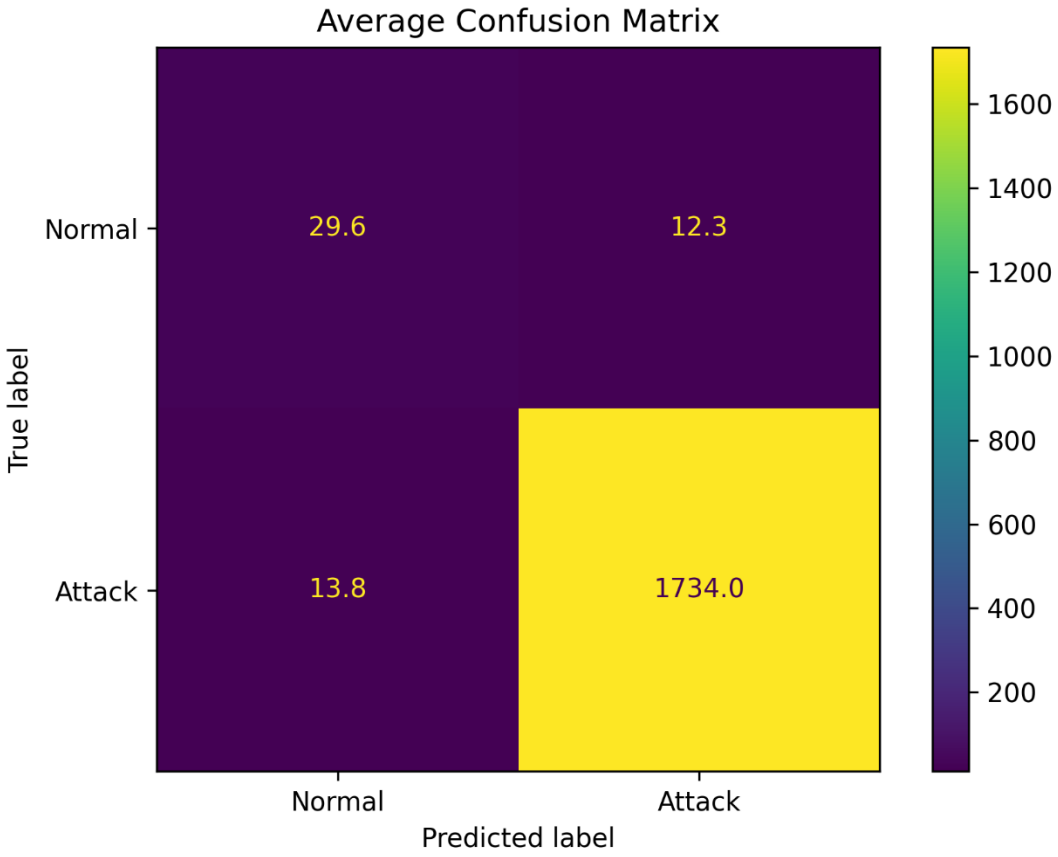


Figure 7.17 Average Confusion Matrix over 10 Folds

7.2.9. Metric Comparison

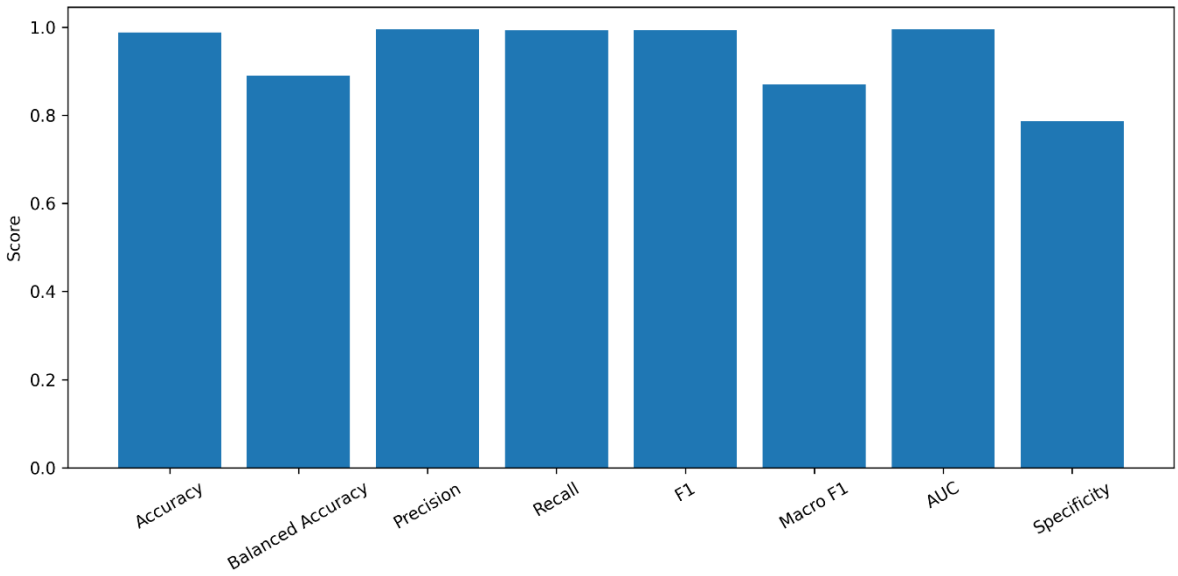


Figure 7.18 Metric Comparison

7.2.10. Discussion

The Artificial Neural Network (ANN) produced an average performance of 98.75% accuracy, 99.49% precision, 99.23% sensitivity, 99.36% F1-Score, and 99.55% AUC, thus exhibiting high-performance characteristics in binary classification. The low standard deviation seen in the performance measures shows that the learning is consistent during cross validation. The ROC plot nearly touches the perfect classifier, showing that the network has effectively classified attack traffic from benign traffic at different levels of threshold. The confusion matrix shows that most attack instances have been identified, and only a few benign instances have been misclassified. Although the ANN has achieved higher specificity (78.61%) compared to the CNN, the accuracy and F1-Score of the two algorithms is similar. The fully connected network was capable of learning nonlinear interactions between the selected PCC features.

7.3. Random Forest

7.3.1. Hyperparameter

Hyperparameter	Value
Algorithm	Random Forest Classifier
Number of Trees	100
Random State	42
CPU Cores	-1 (All Available Cores)
Classification	Binary
Decision Threshold	0.1
Cross Validation	Stratified 10-Fold
Number of Folds	10
Feature Selection	PCC (15 Features)
Feature Scaling	StandardScaler

Table 7.2. Hyperparameters for Random Forest

7.3.2. Fold Accuracy

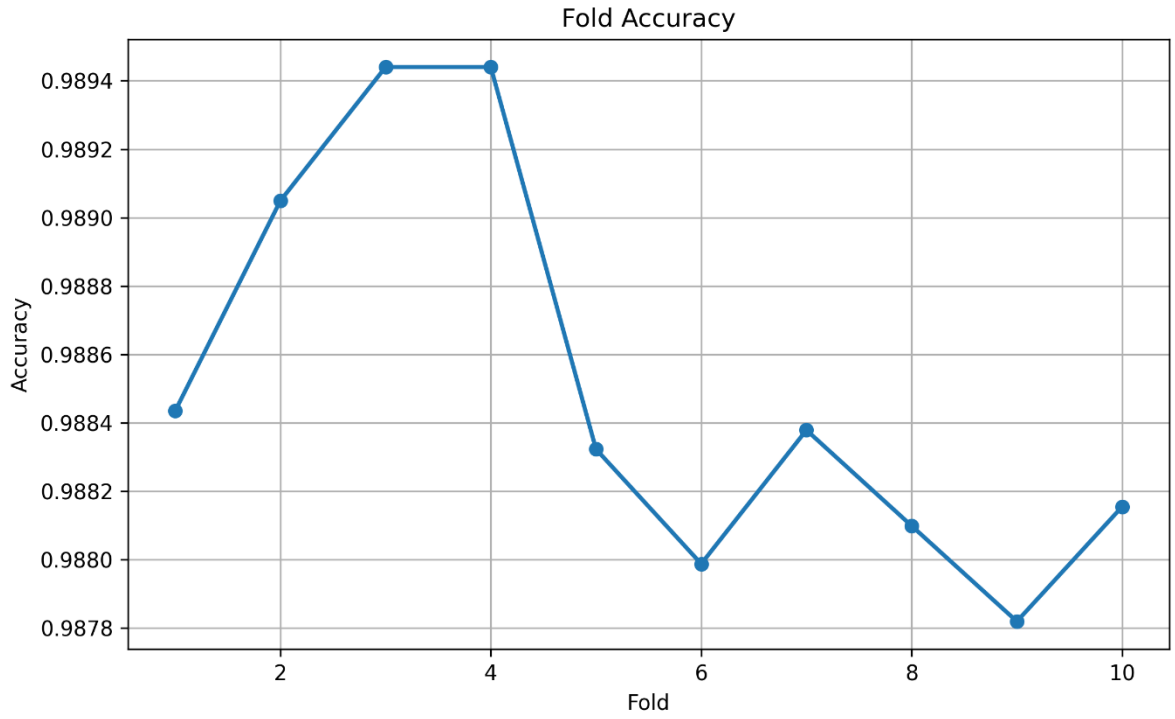


Figure 7.19 Accuracy over 10 Folds

7.3.3. Experiment Summary

Metric	Mean	Std
Accuracy	0.988512482	0.000589248
Balanced Accuracy	0.879030782	0.012734442
Precision	0.994333787	0.000612124
Recall	0.993900865	0.000666847
F1	0.99411699	0.00030196
Macro F1	0.875666644	0.006801352
AUC	0.995457652	0.001926717
Specificity	0.7641607	0.025835273

Table 7.3. Evaluation Metrics

7.3.4. Average ROC

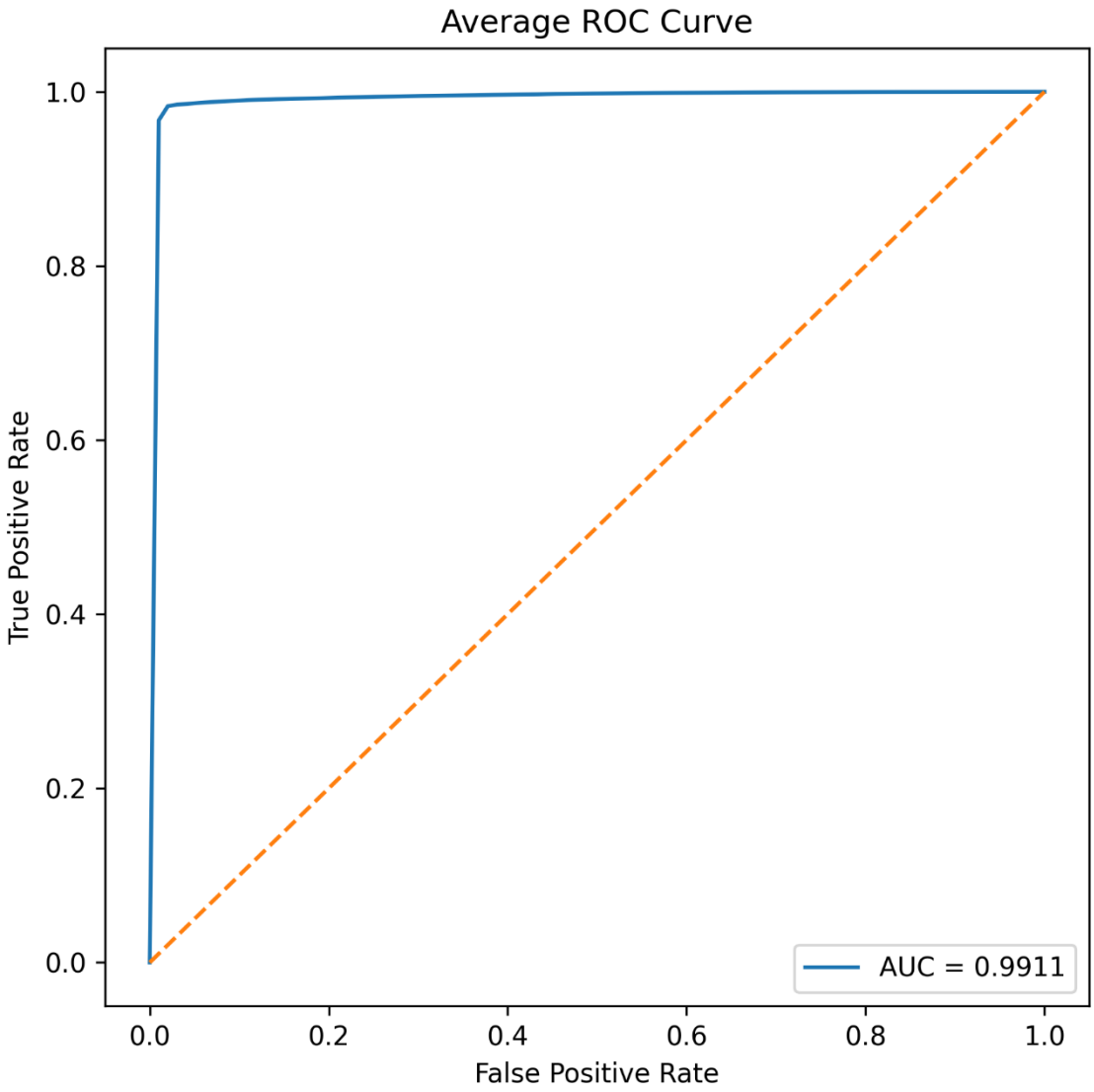


Figure 7.20 Average ROC curve of Random Forest

7.3.5. Average Confusion Matrix

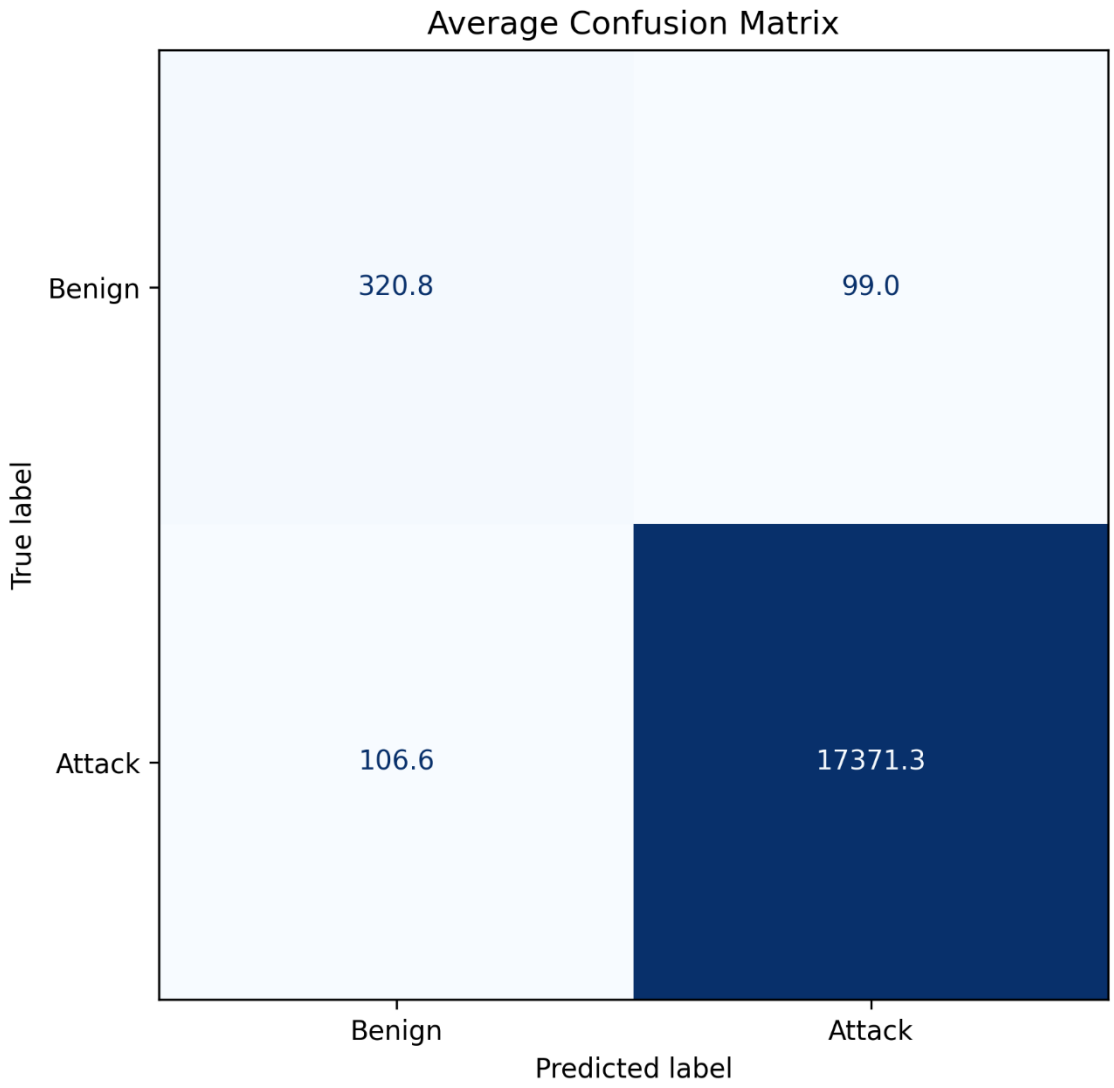


Figure 7.21 Average Confusion Matrix for Random Forest

7.3.6. Metric Comparison

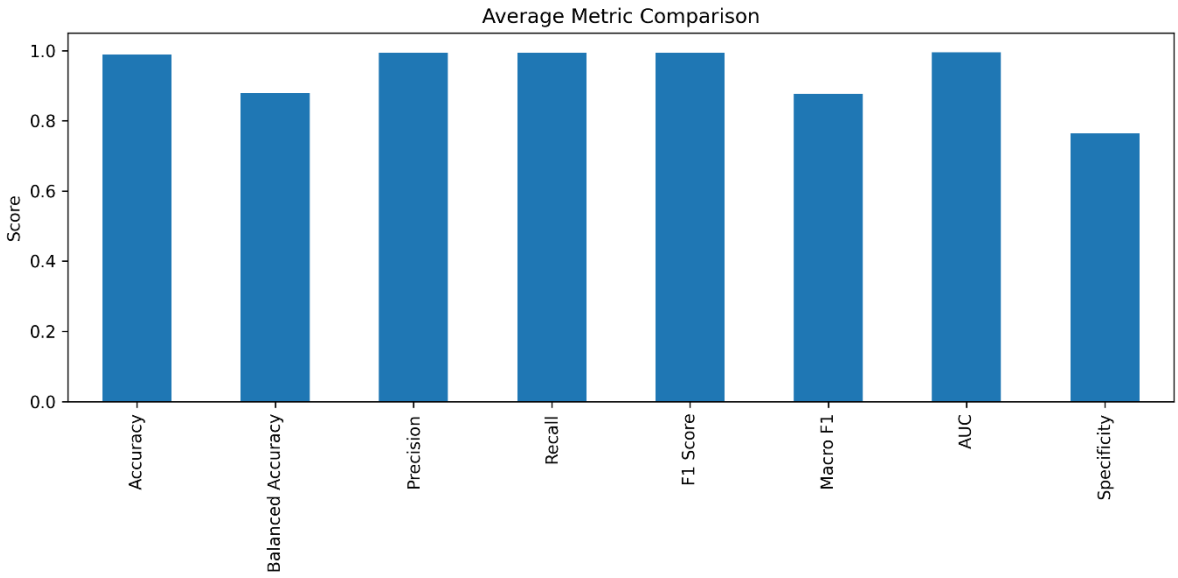


Figure 7.22. Average Metric Comparison

7.3.7. Discussion

Random Forest Classifier showed the best overall results among all the tested models, with an average accuracy of 98.85%, precision of 99.43%, recall of 99.39%, F1-score of 99.41% and an AUC of 99.55%. Stable values of the standard deviation for all metrics confirm the effectiveness and reliability of the proposed ensemble learning method. The ROC curve still keeps its distance from the ideal classifier, showing good separability of benign and malicious traffic. From the confusion matrix, it follows that most of attacks were recognized properly, with almost no false negatives and false positives. In comparison with the deep learning models, specificity (76.42%) is still not that high due to an imbalance of data set; nevertheless, the voting technique used in ensembles reduced classification mistakes.

7.4. DECISION TREE

7.4.1. Hyperparameter Configuration

Hyperparameter	Value
Algorithm	Decision Tree Classifier
Random State	42
Classification	Binary
Decision Threshold	0.1
Cross Validation	Stratified 10-Fold
Number of Folds	10
Feature Selection	PCC (Reduced Dataset)
Feature Scaling	StandardScaler

Table 7.4. Hyperparameter Configuration of Decision Tree

7.4.2. Cross Validation Results

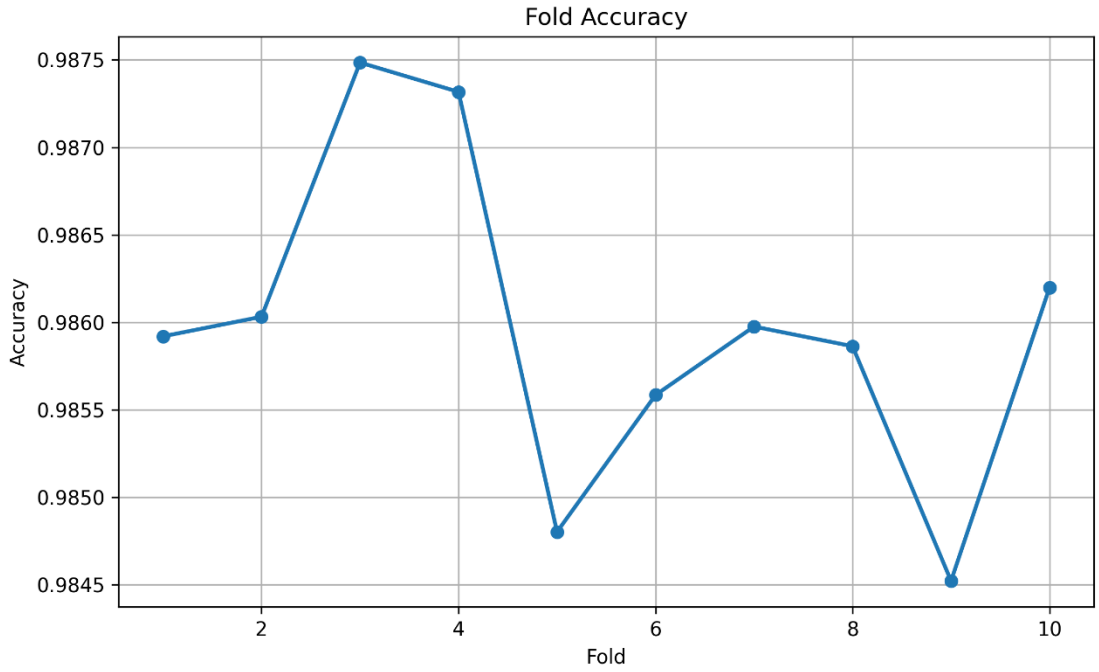


Fig 7.23 Accuracy over 10 Folds

7.4.3. Performance Metrics

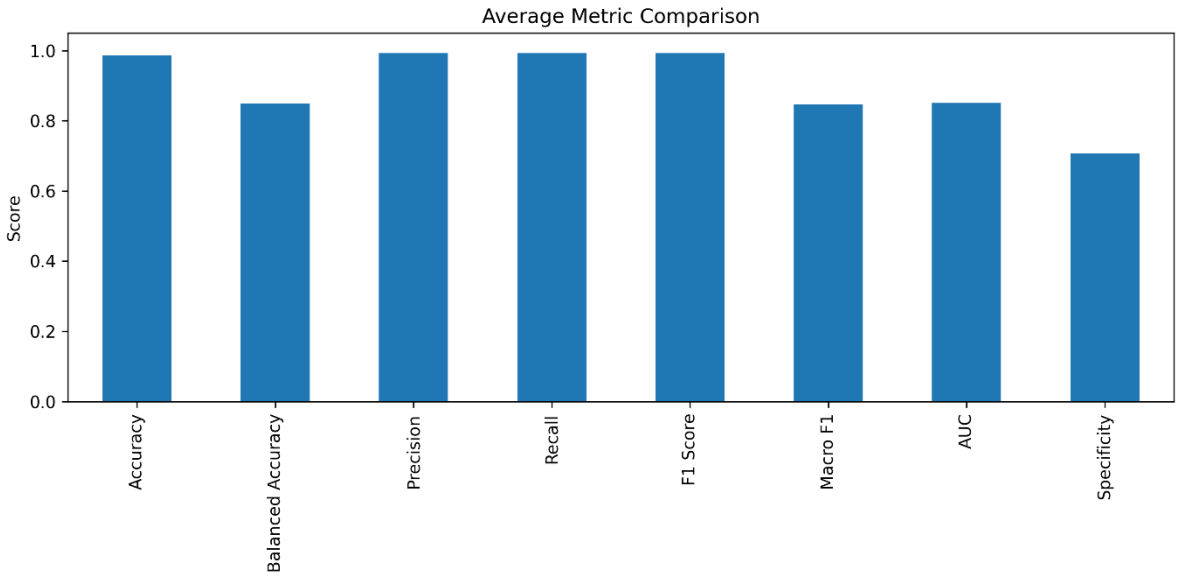


Fig 7.24 Performance Metrics of Decision Tree

7.4.4. ROC Analysis

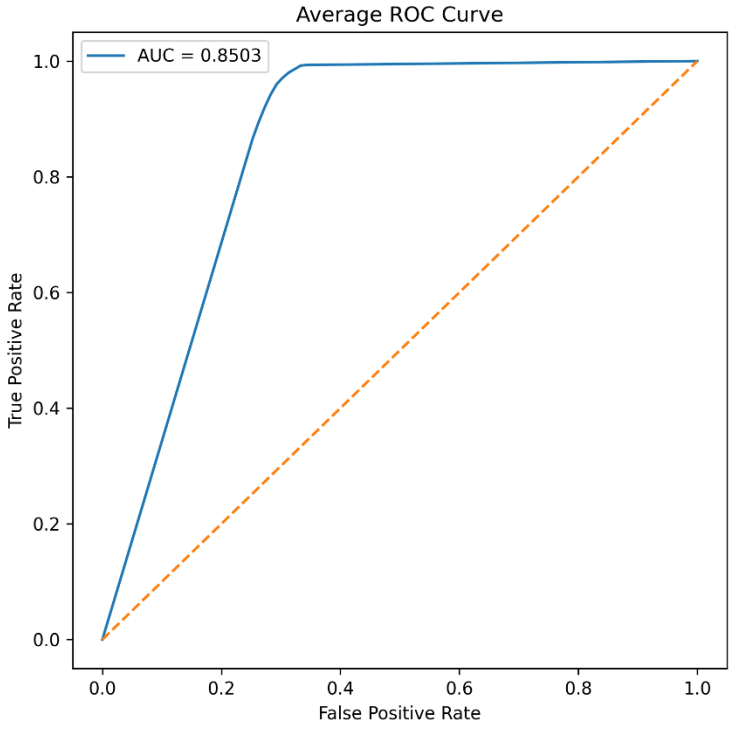


Fig 7.25 ROC Curve of Decision Tree

7.4.5. Confusion Matrix

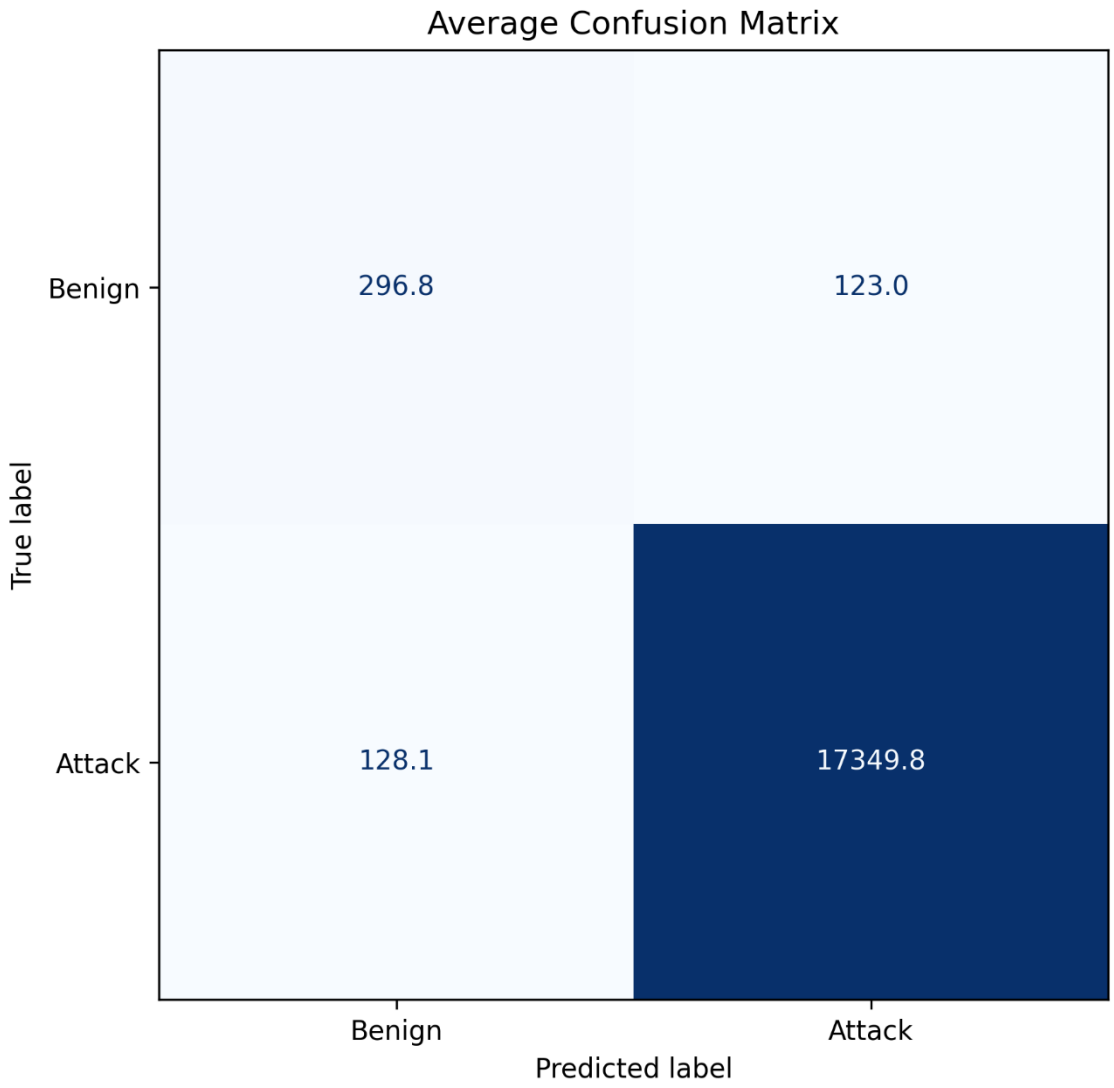


Fig 7.26 Confusion Matrix of Decision Tree

7.5. BINARY CLASSIFICATION COMPARISION

Metric	CNN	ANN	RF	DT
Accuracy	98.75	98.75	98.85	98.59
Balanced Accuracy	87.62	88.92	87.90	84.93
Precision	99.42	99.49	99.43	99.29
Recall/Sensitivity	99.30	99.23	99.39	99.26
F1	99.36	99.36	99.41	99.28
Macro F1	86.69	87.00	87.57	84.76
AUC	99.56	99.55	99.55	85.02
Specificity	75.94	78.61	76.42	70.69

Table 7.4. Binary Classification Model Comparison

7.5.1. Comparative Discussion

The comparison demonstrates that all four models achieved excellent classification performance, with an overall accuracy exceeding 98.5%, indicating the effectiveness of the proposed preprocessing pipeline comprising PCC-based feature selection, StandardScaler-based feature scaling, and SMOTE class balancing.

Among the evaluated models, the Random Forest classifier achieved the highest overall performance with an Accuracy of 98.85% and an F1-Score of 99.41%, indicating its superior capability in correctly distinguishing between normal and malicious network traffic. The ensemble nature of Random Forest enables multiple decision trees to collaboratively reduce variance and improve classification stability, resulting in better generalization compared with the single Decision Tree model. Furthermore, the model achieved a Macro F1-Score of 87.57%, demonstrating improved performance across both classes and confirming its effectiveness in handling the inherent class imbalance present in intrusion detection datasets.

The Artificial Neural Network (ANN) and Convolutional Neural Network (CNN) also demonstrated outstanding classification performance, each achieving an overall Accuracy of 98.75%. The ANN produced the highest Balanced Accuracy (88.92%) and Specificity (78.61%), indicating a better capability to correctly identify normal network traffic while maintaining a high attack detection rate. On the other hand, the CNN achieved the highest AUC of 99.56%, reflecting its excellent discriminative ability across different classification thresholds. These results demonstrate that the deep learning models successfully learned complex feature representations from the selected network traffic attributes, enabling highly accurate binary intrusion detection.

Although the Decision Tree classifier achieved a competitive Accuracy of 98.59%, its performance was comparatively lower than the remaining models across most evaluation metrics. In particular, the model recorded the lowest Balanced Accuracy (84.93%), Macro F1-Score (84.76%), AUC (85.02%), and Specificity (70.69%), indicating a reduced capability to generalize compared with ensemble and deep learning approaches. These observations are expected because a single decision tree is more susceptible to overfitting and may not effectively capture the complex decision boundaries associated with diverse IoT network traffic patterns.

Overall, the comparative analysis indicates that all four binary classification models achieved high intrusion detection performance, validating the effectiveness of the proposed preprocessing and feature engineering framework. While the Decision Tree served as a reliable baseline model, the Random Forest consistently provided the best overall performance among the Machine Learning approaches. Similarly, the ANN and CNN demonstrated comparable predictive capability, with the ANN offering improved balanced classification performance and the CNN exhibiting superior discriminative ability through its AUC score. These findings establish a strong foundation for the subsequent multiclass classification experiments, where the models are further evaluated for their ability to identify specific attack categories rather than simply distinguishing between normal and malicious traffic.

7.6. MULTICLASS CLASSIFICATION

7.6.1. DECISION TREE

7.6.1.1. Hyperparameter Configuration

Hyperparameter	Value
Algorithm	Decision Tree Classifier
Random State	42
Classification	Multiclass
Decision Threshold	0.90
Cross Validation	Stratified K-Fold
Number of Folds	10
Feature Selection	PCC-Based Feature Selection
Feature Scaling	StandardScaler
Splitting Criterion	Gini

Table 7.5. Decision Tree Hyperparameters for Multiclass

7.6.1.2. Cross Validation Results

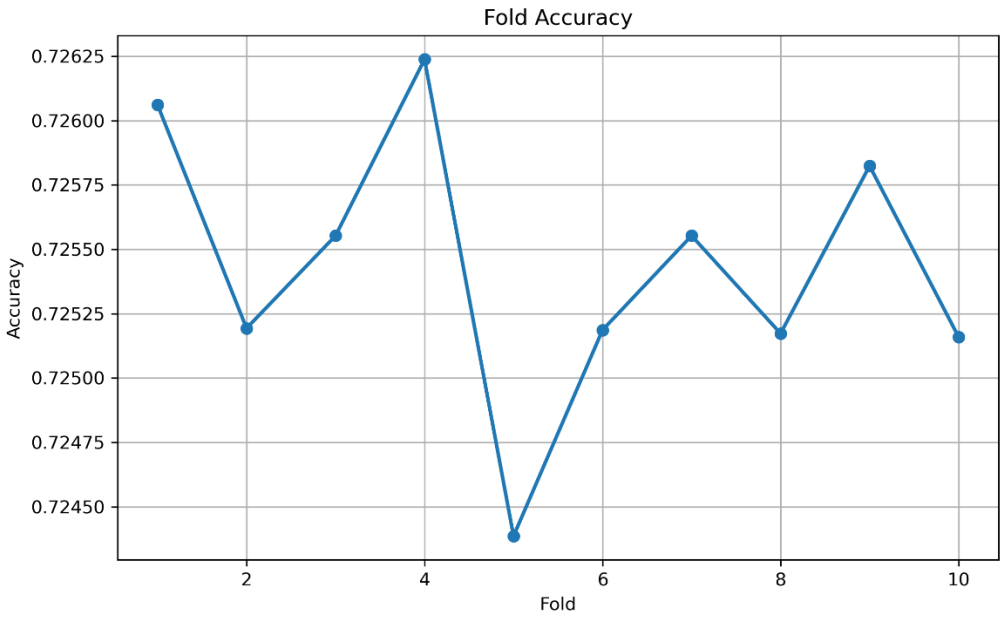


Fig 7.27 Cross Validation Accuracy of Decision Tree

7.6.1.3. Performance Metrics

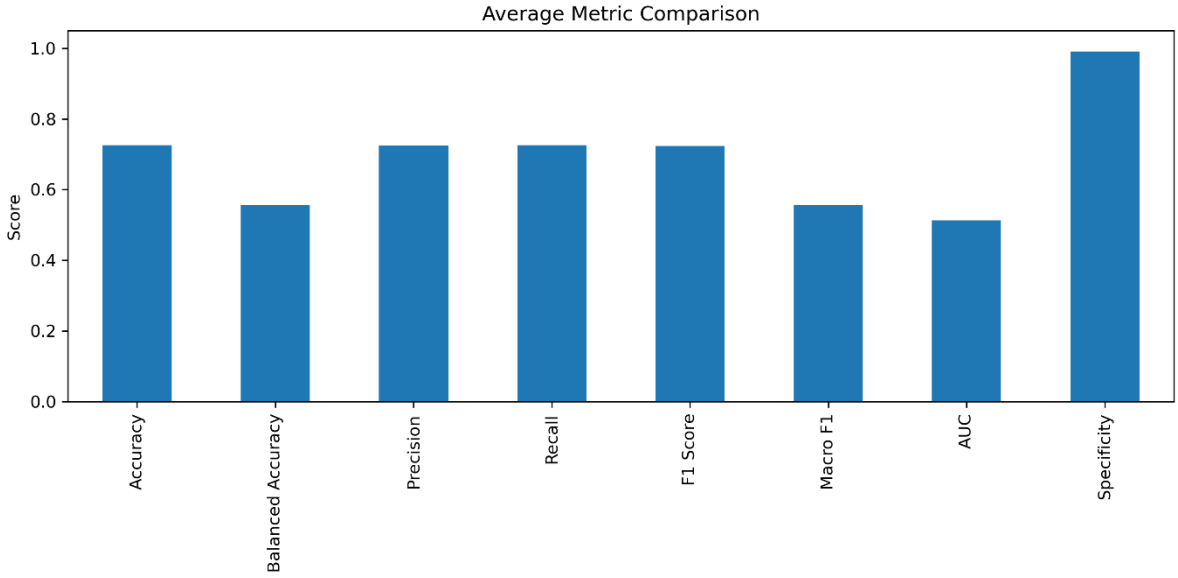


Fig 7.28 Performance Metrics of Decision Tree Multiclass

7.6.1.4. Confusion Matrix

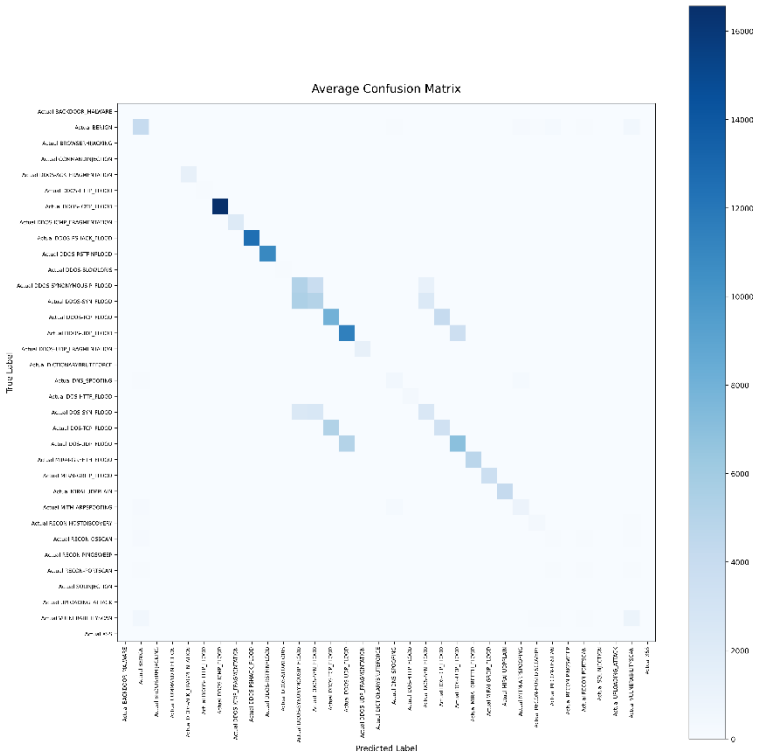


Fig 7.29 Confusion Matrix of Decision Tree Multiclass

7.6.2. Random Forest

7.6.2.1. Hyperparameter Configuration

Parameter	Value
Algorithm	Random Forest Classifier
Random State	42
Classification	Multiclass
Decision Threshold	0.90
Cross Validation	Stratified K-Fold
Number of Folds	2
Feature Selection	PCC-Based Feature Selection
Feature Scaling	StandardScaler
Splitting Criterion	Gini

Table 7.6 Random Forest Hyperparameter Configuration

7.6.2.2. Cross Validation Results

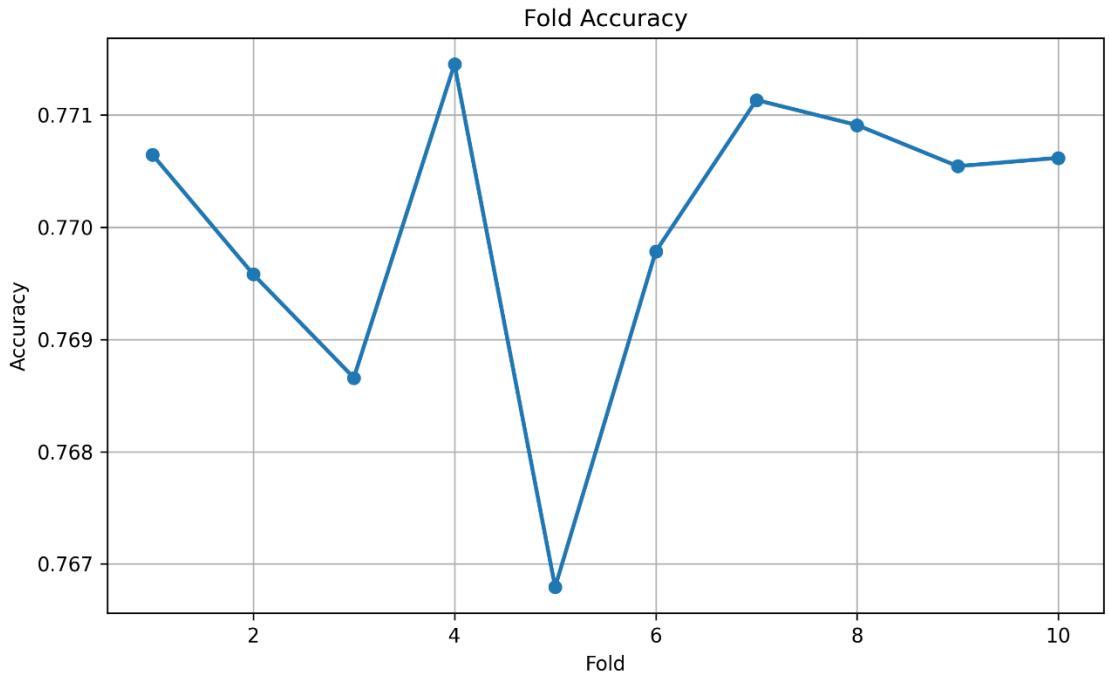


Fig 7.30 Cross Validation Accuracy over 10 Folds

7.6.2.3. Performance Metrics

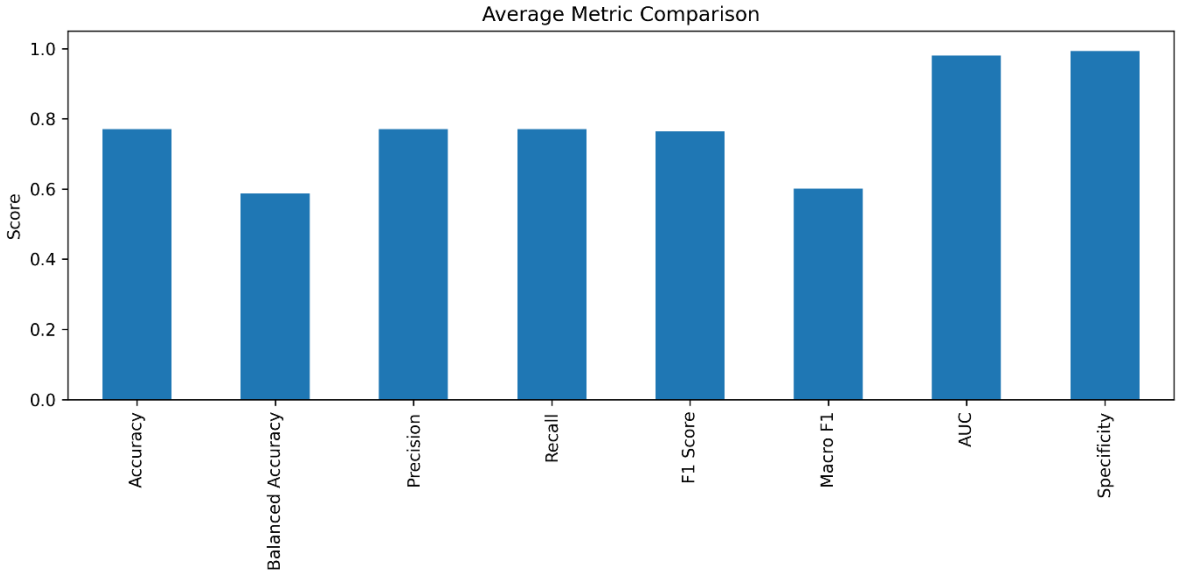


Fig 7.31 Performance Metrics of Random Forest Multiclass Classifier

7.6.2.4. Confusion Matrix

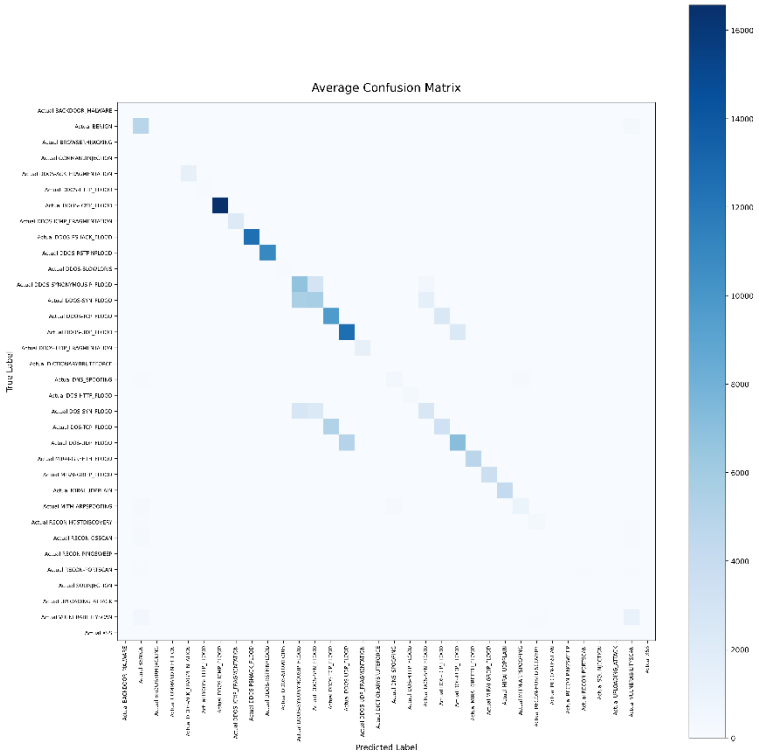


Fig 7.32 Confusion Matrix of Random Forest Multiclass Classifier

7.6.3. Artificial Neural Network (ANN)

7.6.3.1. Hyperparameter Configuration

Hyperparameters

Parameter	Value
Epochs	50
Batch Size	512
Learning Rate	0.001
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Hidden Layers	512 -> 128 -> 64
Activation	ReLU + Softmax
Dropout	nan
Cross Validation	10-Fold Stratified
Early Stopping	10
Class Weights	Balanced

Table 7.7 Hyperparameters of ANN Multiclass with SMOTE

7.6.3.2. Training Performance

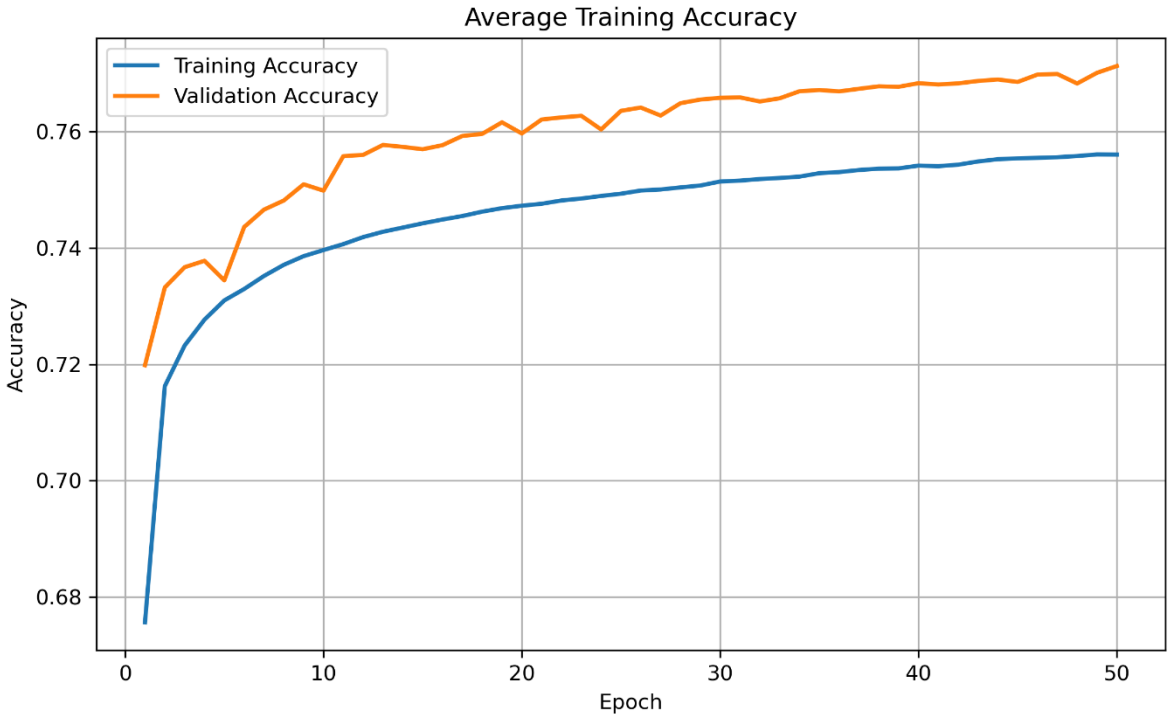


Fig 7.33 Average Training and Validation Accuracy in ANN Multiclass

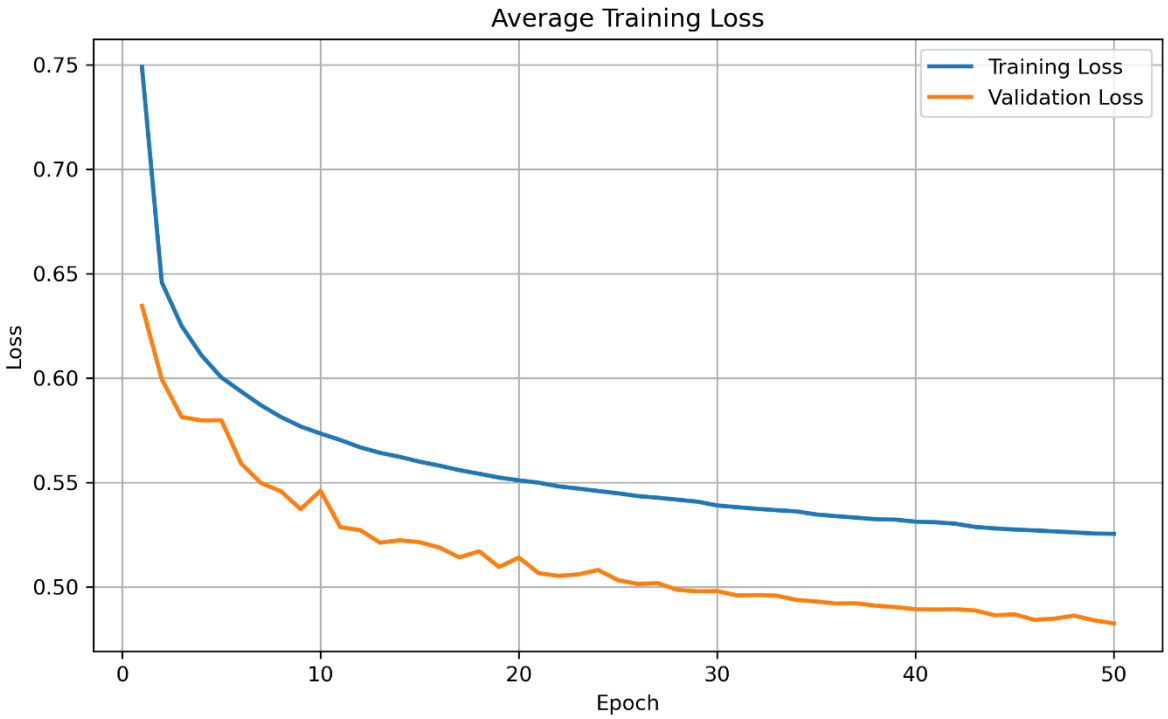


Fig 7.34 Average Training and Validation loss in ANN Multiclass

7.6.3.3. Cross Validation Results

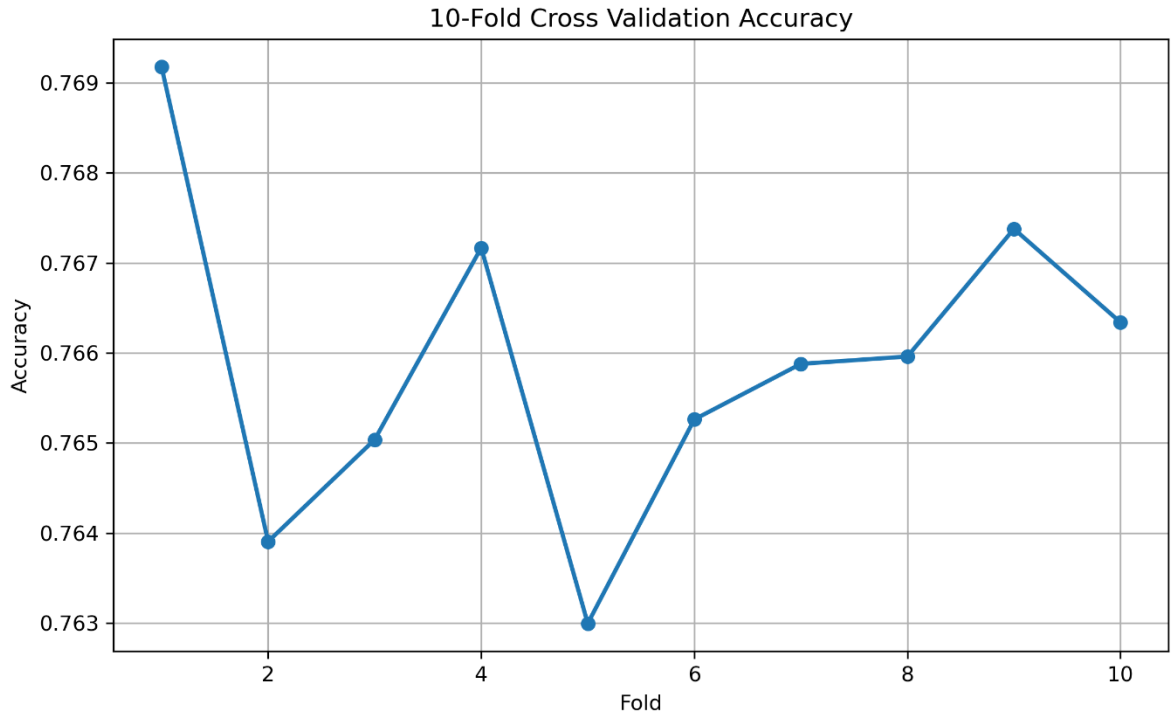


Fig 7.35 Cross Validation over 10 Folds for ANN Multiclass using SMOTE

7.6.3.4. Performance Metrics

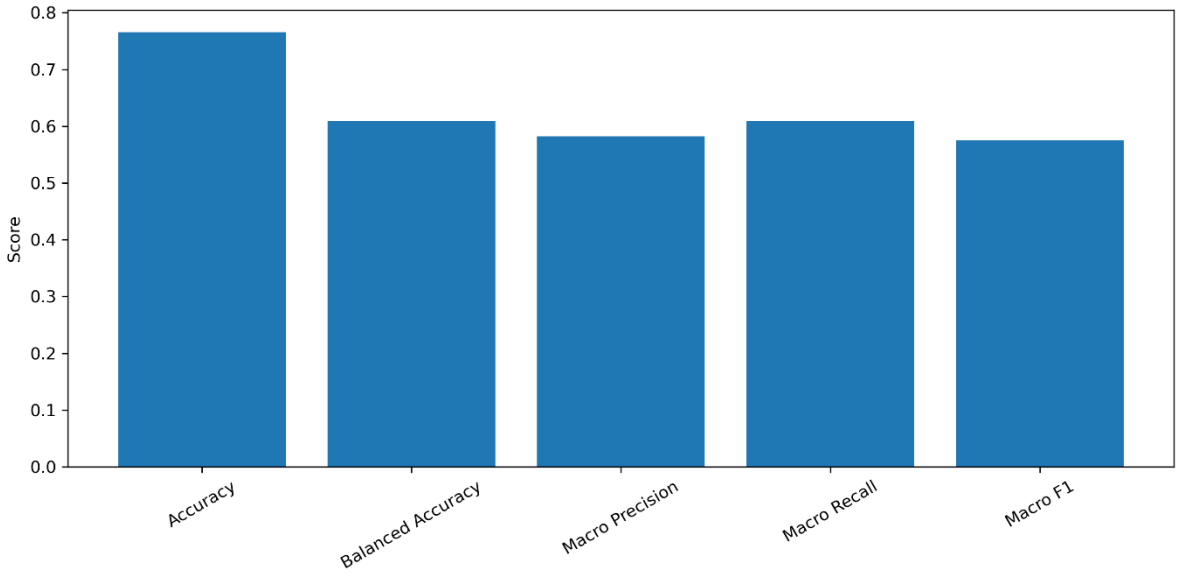


Fig 7.36 Performance Metrics of ANN Multiclass Classifier using SMOTE

7.6.3.5. Confusion Matrix

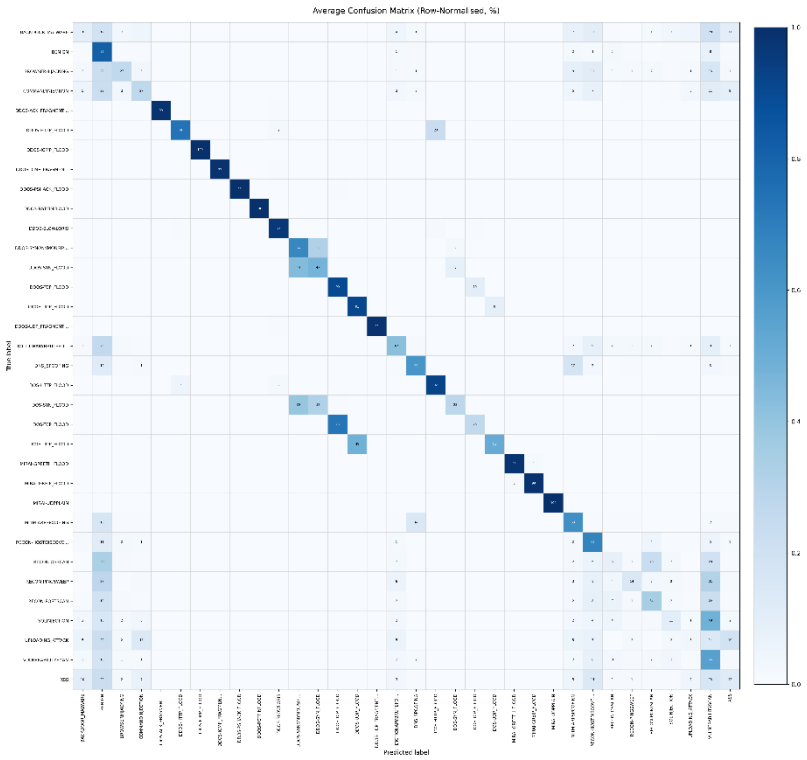


Fig 7.37 Confusion Matrix of ANN Multiclass Classifier using SMOTE

7.6.4. Convolution Neural Network (CNN)

7.6.4.1. Hyperparameter Configuration

Hyperparameters

Parameter	Value
Epochs	50
Batch Size	512
Learning Rate	0.001
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Conv1D Filters	96 -> 64 -> 32
Kernel Size	3
Activation	ReLU + Softmax
Cross Validation	10-Fold Stratified
Early Stopping	10
Class Weights	Balanced

Fig 7.38 Hyperparameters of CNN Multiclass Classifier using SMOTE

7.6.4.2. Training Performance

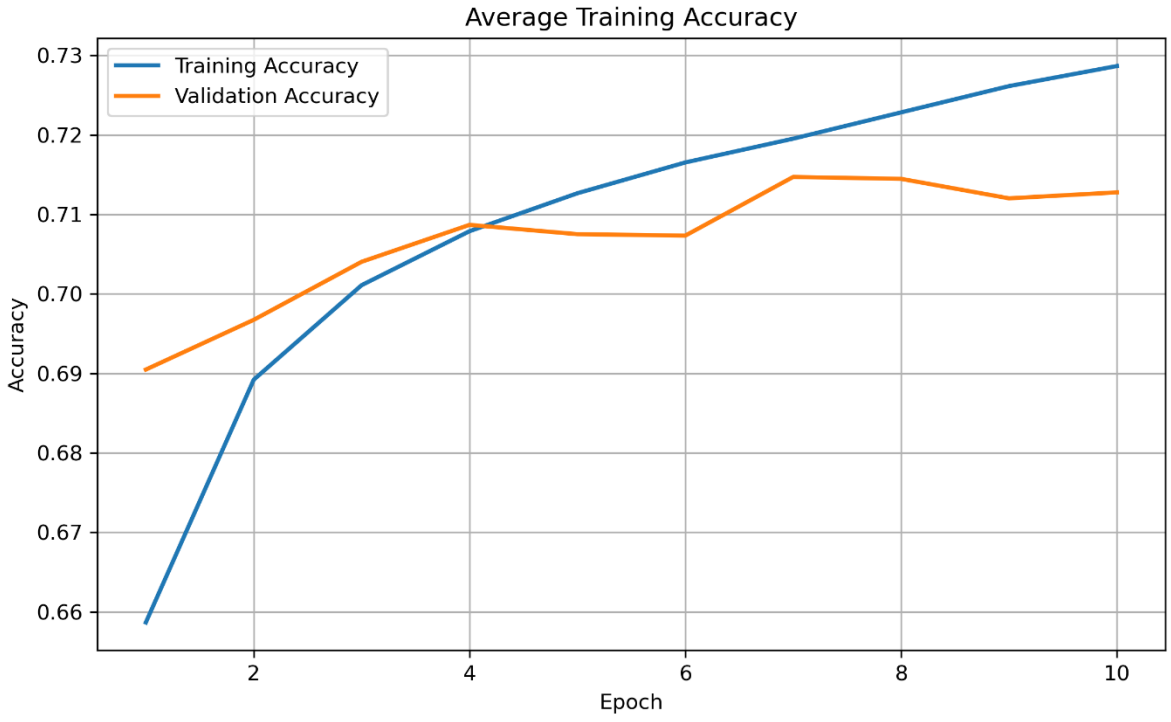


Fig 7.39 Training and Validation Accuracy over epochs

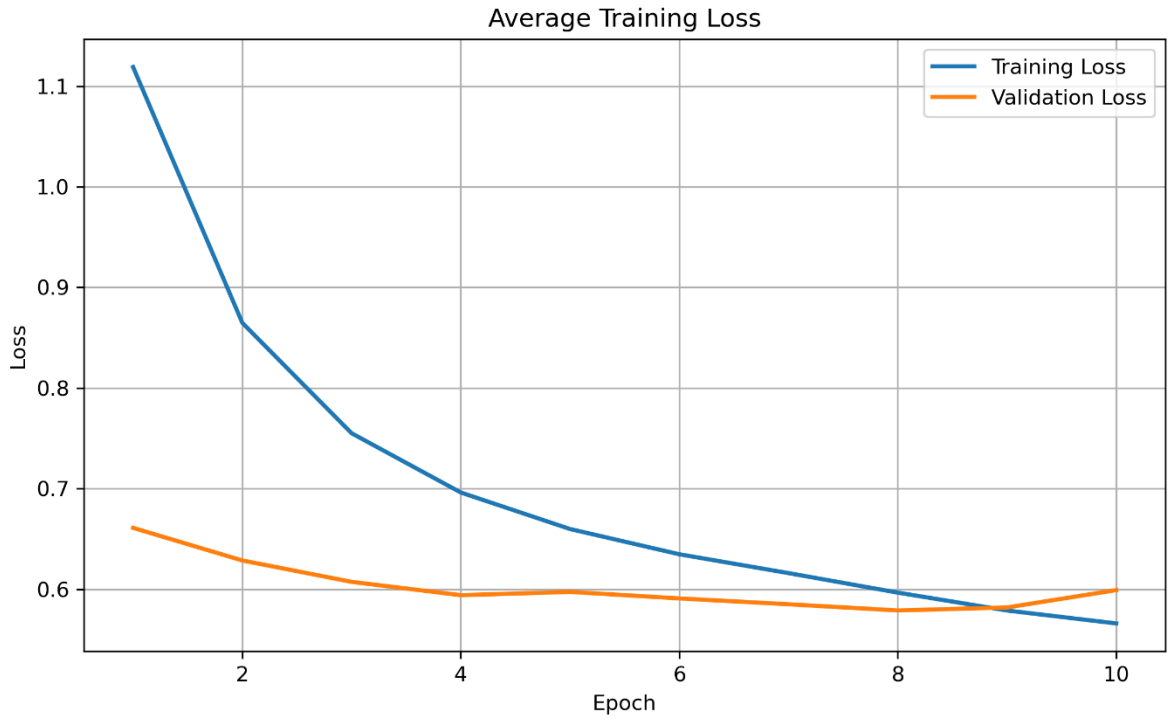


Fig 7.40 Training and Validation Loss over epochs

7.6.4.3. Cross Validation Results

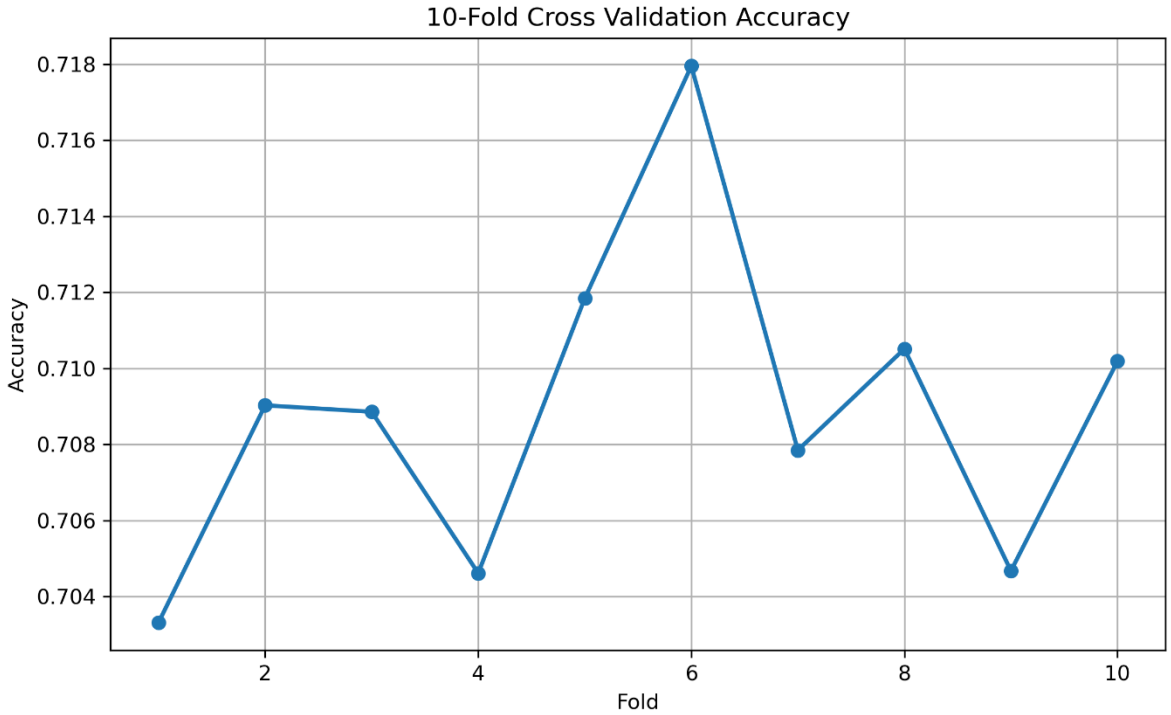


Fig 7.41 Cross Validation of Accuracy over Folds

7.6.4.4. Performance Metrics

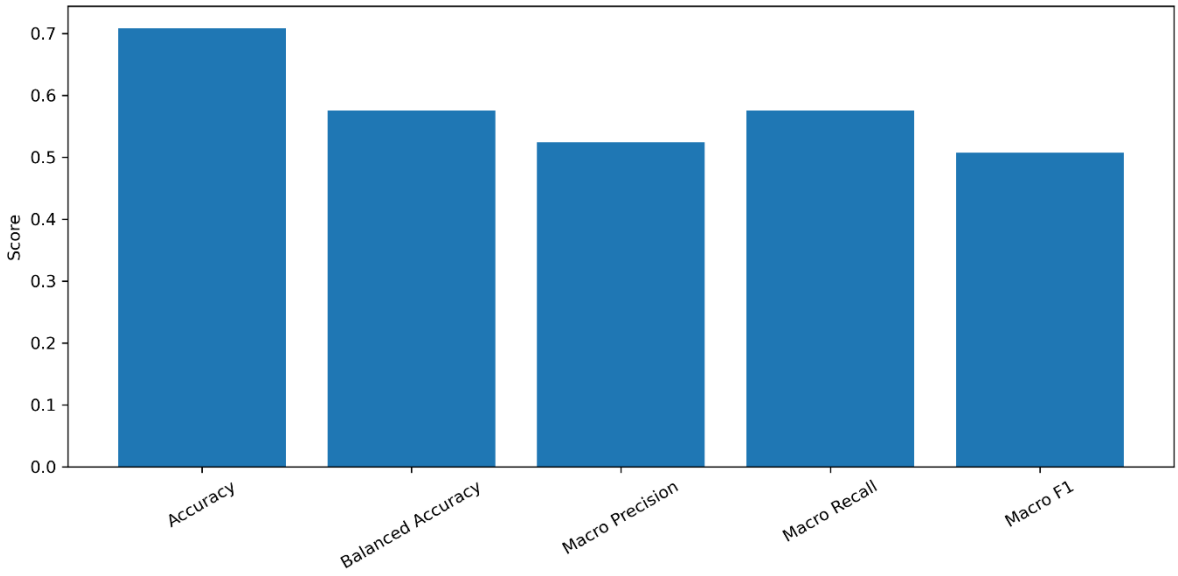


Fig 7.42 Evaluation of Metrics for CNN Multiclass using SMOTE

7.6.4.5. Confusion Matrix

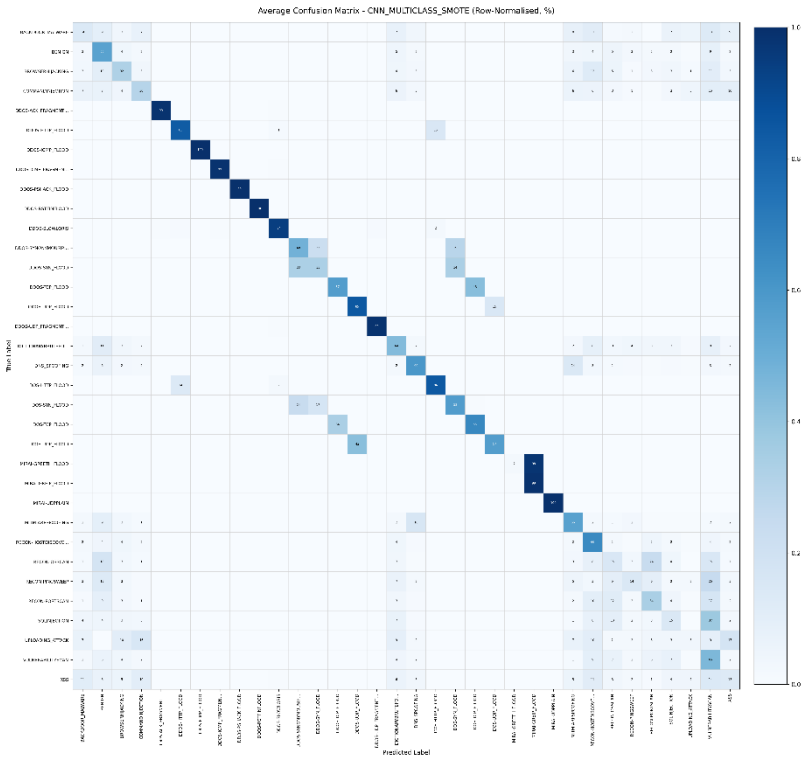


Fig 7.43 Confusion Matrix of CNN Multiclass Classifier using SMOTE

7.6.5. XGBoost

7.6.5.1. Hyperparameter Configuration

Parameter	Value
Algorithm	XGBoost Classifier
Random State	42
Classification	Multiclass
Decision Threshold	0.9
Cross Validation	Stratified K-Fold
Number of Folds	10
Feature Selection	PCC-Based Feature Selection
Feature Scaling	StandardScaler
Splitting Criterion	Gamma and Tree-Based Split Optimization (XGBoost)

Table 7.7 Hyperparameter Configuration for XGBoost

7.6.5.2. Cross Validation Results

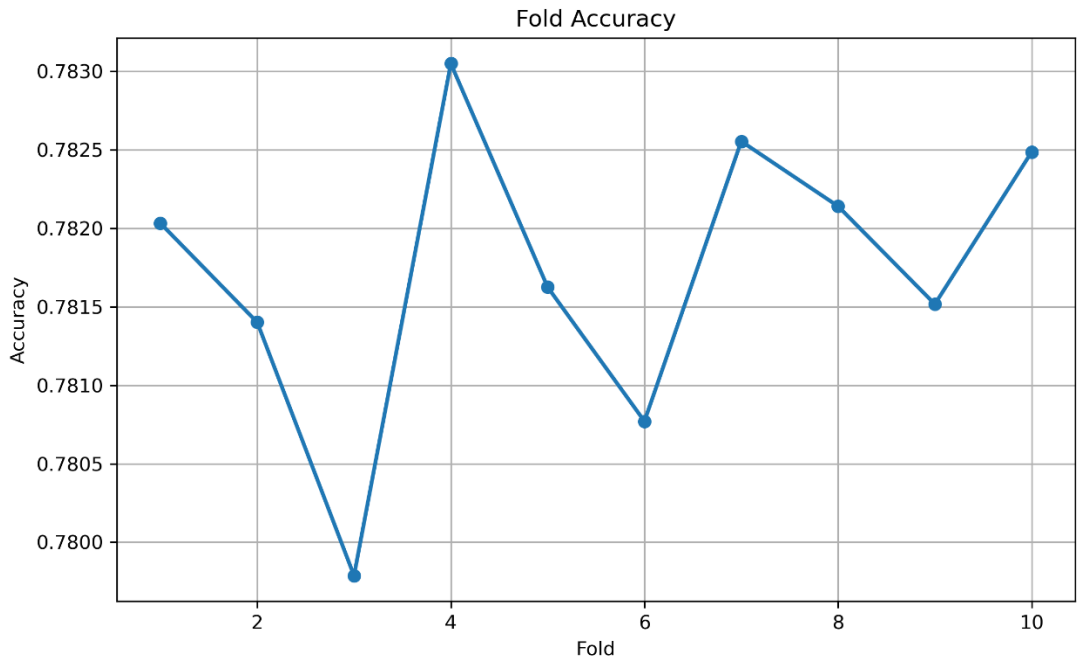


Fig 7.44 Cross Validation of Accuracy over folds

7.6.5.3. Performance Metrics

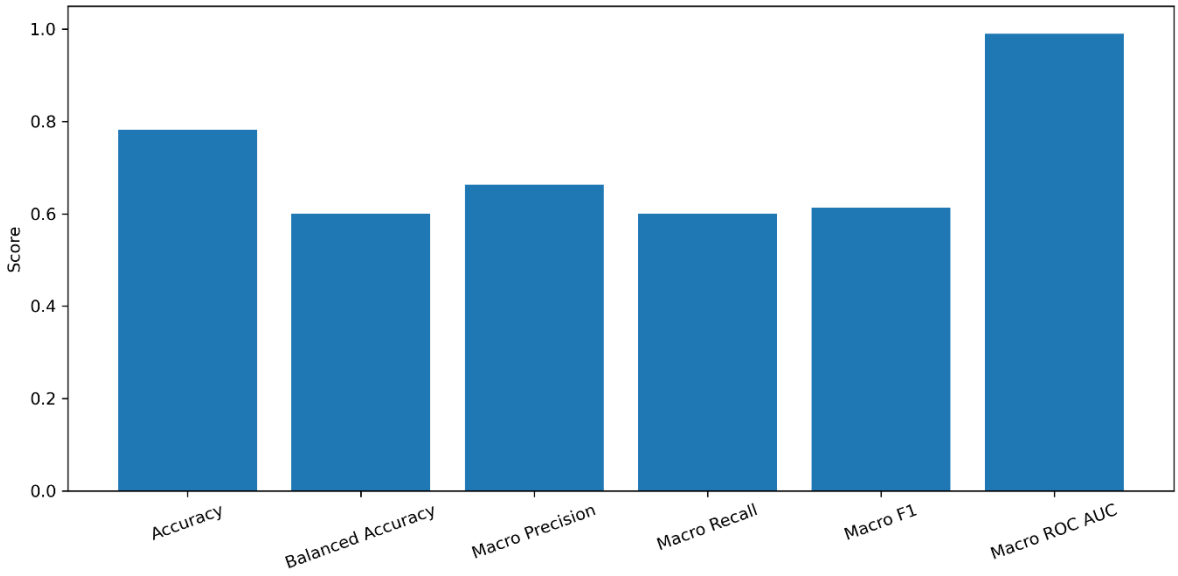


Fig 7.45 Metrics Evaluation of XGBoost Multiclass Classifier

7.6.5.4. Confusion Matrix

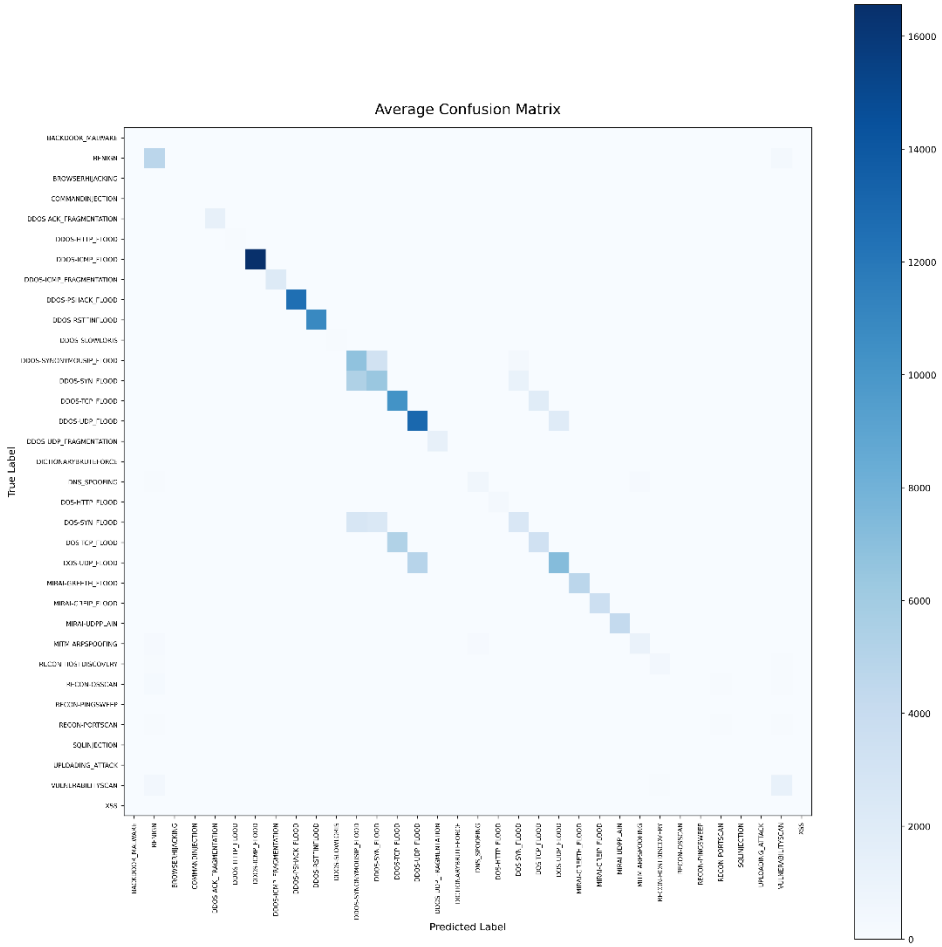


Fig 7.46 Confusion Matrix of XGBoost Multiclass Classifier with SMOTE

7.6.6. Ensemble Learning

7.6.6.1. Hyperparameter Configuration

Parameter	Value
Decision Tree Criterion	gini
RF Trees	150
RF Max Samples	0.5
RF Max Features	sqrt
XGB Max Trees	400
XGB Early Stopping Rounds	20
XGB Learning Rate	0.05
XGB Max Depth	12
XGB Subsample	0.8
XGB Colsample	0.8
XGB Min Child Weight	3
XGB Gamma	0
XGB Regularization Alpha	0
XGB Regularization Lambda	1
SMOTE (XGBoost only)	TRUE
Stacking	TRUE
Cross Validation	5-Fold Stratified

Table 7.8 Hyperparameter Configuration for Ensemble Learning

7.6.6.2. Cross Validation Results

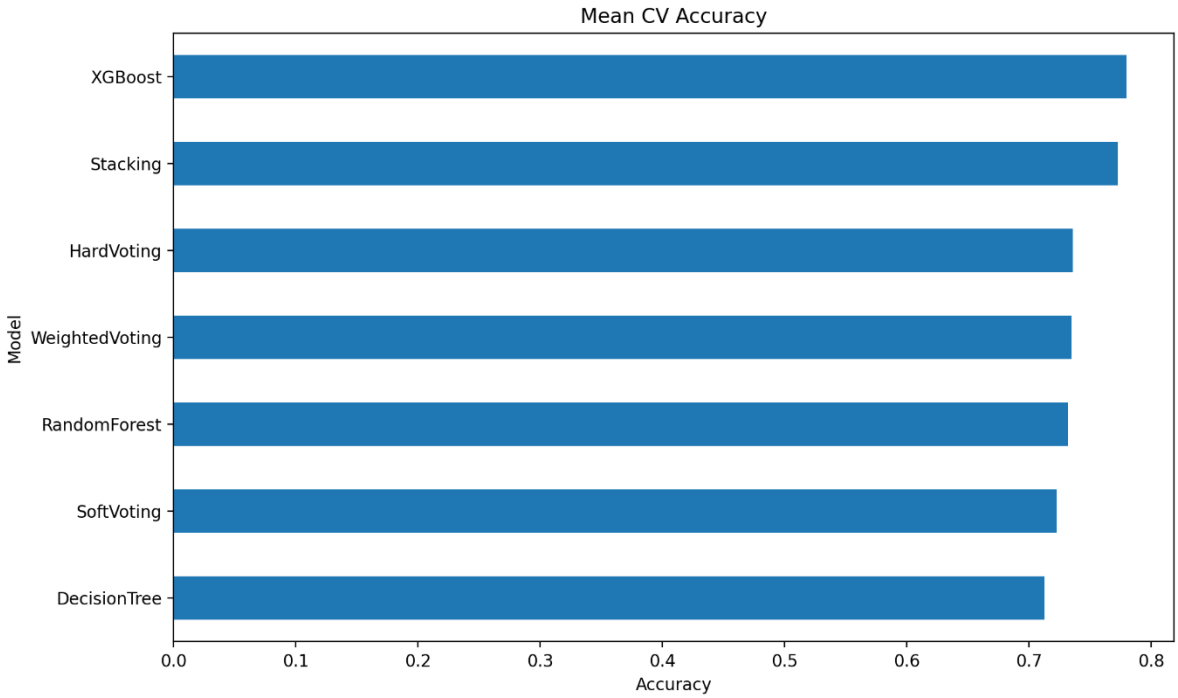


Fig 7.47 Mean Accuracy of Models over folds

7.6.6.3. Performance Metrics

Model	Accuracy (%)	Balanced Accuracy (%)	Macro Precision (%)	Macro Recall (%)	Macro F1 (%)	Macro ROC AUC (%)
XGBoost	**77.87**	**60.25**	**63.58**	**60.25**	**60.61**	**98.96**
WeightedVoting	74.25	58.50	65.67	58.50	59.76	98.83
HardVoting	74.34	58.14	62.32	58.14	59.15	78.66
RandomForest	74.10	57.20	68.74	57.20	58.73	93.80
SoftVoting	72.26	57.00	58.67	57.00	57.59	98.79
Stacking	76.22	61.75	56.88	61.75	56.29	98.84
DecisionTree	71.27	55.17	55.24	55.17	55.18	77.27

Fig 7.48 Performance Metrics of models in Ensemble Learning

7.6.6.4. Confusion Matrix

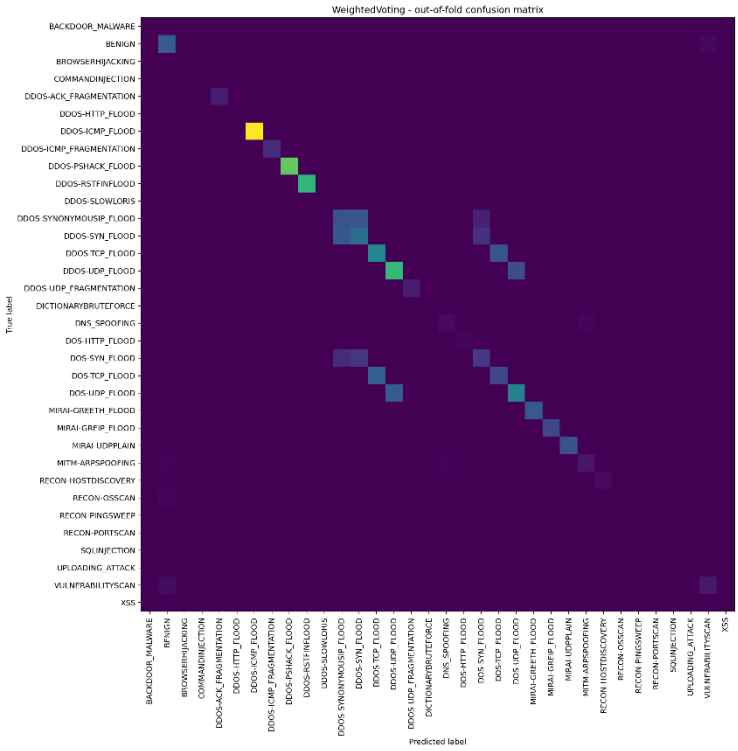


Fig 7.49 Confusion Matrix of Weighted Voting

7.7. MULTICLASS CLASSIFICATION COMPARISION

7.7.1. Performance Metrics Comparison

Model	Accuracy (%)	Balanced Accuracy (%)	Macro Precision (%)	Macro Recall (%)	Macro F1 (%)	Macro ROC AUC (%)
XGBoost	**77.87**	60.25	63.58	60.25	**60.61**	98.96
CNN	76.63	**60.97**	58.29	**60.97**	57.42	98.89
Hard Voting (Ensemble)	74.34	58.14	62.32	58.14	59.15	78.66
Random Forest	74.10	57.20	**68.74**	57.20	58.73	93.80
ANN	70.87	57.66	52.37	57.66	50.75	98.90
Decision Tree	71.27	55.17	55.24	55.17	55.18	77.27

Fig 7.50 Multiclass Models Comparison

7.7.2. Comparative Discussion

The comparative analysis of the suggested machine learning, deep learning, and ensemble classifiers shows that each classifier possesses its own advantages in terms of various performance measures. Among all the tested models, XGBoost showed the best overall performance in classification and achieved the highest accuracy of 77.87% and Macro F1-score of 60.61%. It means that the suggested model is the one with the best combination of precision and recall while having high overall predictive abilities in multiclass IoT intrusion detection. The ROC-AUC score of 98.96% obtained by the model proves its high ability to differentiate between different attacks.

The CNN model emerged as the strongest deep learning approach. Although its overall accuracy (76.63%) was slightly lower than XGBoost, it achieved the highest Balanced Accuracy (60.97%) and Macro Recall (60.97%) among all models. These results indicate that CNN was more effective in correctly identifying samples from minority attack classes, making it a robust choice for handling class imbalance in multiclass intrusion detection.

The Hard Voting Ensemble demonstrated improved stability over individual traditional machine learning models. By combining the predictions of Decision Tree, Random Forest, and XGBoost, the ensemble achieved an Accuracy of 74.34%, Balanced Accuracy of 58.14%, and a Macro F1-score of 59.15%. Although its overall accuracy was lower than XGBoost and CNN, the ensemble produced consistent performance across multiple evaluation metrics, illustrating the advantage of combining diverse classifiers to improve prediction reliability.

Among the conventional machine learning models, Random Forest produced competitive results with an Accuracy of 74.10% and achieved the highest Macro Precision (68.74%). This indicates that Random Forest generated the fewest false positive classifications among all evaluated models. However, its Balanced Accuracy and Macro Recall remained lower than those achieved by CNN and XGBoost, suggesting comparatively weaker performance on minority classes.

The ANN model achieved an Accuracy of 70.87% with a Balanced Accuracy of 57.66%. While its overall accuracy was acceptable, its Macro F1-score of 50.75% indicates comparatively weaker performance in maintaining balanced classification across all attack categories. This suggests that the shallow neural network architecture

was less effective than the convolutional architecture for extracting discriminative intrusion patterns.

The Decision Tree served as the baseline machine learning classifier and recorded an Accuracy of 71.27% with a Macro F1-score of 55.18%. Although it provided fast and interpretable predictions, its comparatively lower Balanced Accuracy and ROC-AUC indicate limited generalization capability when handling complex multiclass intrusion patterns.

In general, it should be noted that the results of experiments showed that XGBoost outperforms other standalone classifiers on the basis of its high efficiency in terms of both accuracy and class-wise effectiveness. As for deep learning models, it can be argued that CNN turned out to be the most powerful, as it demonstrated the highest values of both Balanced Accuracy and Recall and, therefore, was capable of working effectively with minority attack classes. The results of experiments with the Hard Voting Ensemble demonstrated the possibility of increasing the consistency of predictions through their combination, but, at the same time, this approach did not surpass XGBoost.

8. CONCLUSION

In this project, the implementation of an anomaly-based Intrusion Detection System (IDS) for IoT networks through machine learning and deep learning algorithms is carried out. The increased use of IoT technology poses many challenges to cybersecurity because of the high number of traffic in a heterogeneous environment and the increased complexity of cyberattacks. In order to counter this challenge, the design of the IDS through the methodology in the chosen research paper is done.

The implementation began with the collection and preprocessing of the CICIoT2023 dataset. Since the original dataset consisted of a large number of network traffic records distributed across multiple files, the data was merged, sampled, cleaned, and transformed into a binary classification dataset. Exploratory Data Analysis (EDA) was then performed to understand the characteristics of the dataset and validate its suitability for machine learning and deep learning applications.

To reduce data dimensionality and eliminate redundant attributes, Pearson Correlation Coefficient (PCC)-based feature selection was applied. The selected feature subset was subsequently used to train three classification models, namely Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN). Implementing multiple classifiers enables a comparative evaluation of traditional machine learning and deep learning approaches for IoT intrusion detection.

The implementation presented in this work establishes a complete intrusion detection framework, beginning with raw network traffic data and progressing through preprocessing, feature selection, model development, and evaluation. The experimental analysis and comparative performance of the implemented models are presented in the Results chapter, providing a detailed assessment of their effectiveness in detecting malicious network traffic.

Following the completion of the initial project review, the experimental study was further extended by implementing additional machine learning and ensemble learning techniques for multiclass intrusion detection. A Decision Tree classifier was first developed to establish a baseline for comparison among traditional machine learning approaches. Subsequently, XGBoost was implemented and extensively evaluated using the same preprocessing pipeline and feature set. The experimental results demonstrated that XGBoost consistently achieved the highest overall classification performance, recording an accuracy of 77.87% and the highest Macro F1-score of 60.61%, thereby outperforming all other standalone machine learning and deep learning models. These findings highlight the effectiveness of gradient boosting in capturing complex relationships within IoT network traffic while maintaining strong generalization capability.

In addition to standalone classifiers, an ensemble learning framework was developed by combining the predictions of multiple machine learning models using a Hard Voting strategy. The ensemble model provided more stable and reliable predictions than individual conventional classifiers, achieving an overall accuracy of 74.34% with improved class-wise performance. A comprehensive comparison of all evaluated models, including CNN, ANN, Decision Tree, Random Forest, XGBoost, and the proposed Hard Voting Ensemble, confirmed that XGBoost emerged as the best-performing standalone classifier, while the ensemble approach enhanced prediction

robustness by leveraging the complementary strengths of multiple models. These additional experiments provide a more comprehensive evaluation of machine learning and deep learning techniques for IoT intrusion detection and further validate the effectiveness of the proposed framework.

Overall, the work demonstrates a structured implementation of an anomaly-based intrusion detection framework for IoT environments and provides a foundation for further improvements in intelligent network security systems.

9. SCOPE FOR FUTURE WORK

Despite the fact that the developed intrusion detection system offers a methodological way of detecting malicious network traffic in the Internet of Things setting, there are still multiple possibilities for its future development.

Currently, the classification in the system is based on the two-class model that includes classifying the network traffic as being normal or malicious. Future developments of the system could lead to multi-class classification and the possibility of identifying particular classes of attacks including DDoS, spoofing, reconnaissance, brute force attacks, among others.

The current study utilizes Pearson Correlation Coefficient (PCC) for feature selection. Future research may investigate alternative feature selection techniques such as Mutual Information, Recursive Feature Elimination (RFE), Principal Component Analysis (PCA), or hybrid feature selection methods to evaluate their impact on detection performance and computational efficiency.

Additional machine learning and deep learning algorithms, including Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), Transformer-based architectures, and hybrid ensemble models, may also be explored for improving intrusion detection accuracy and robustness.

The framework can further be evaluated using additional benchmark intrusion detection datasets, such as BOT-IoT, UNSW-NB15, TON-IoT, CICIDS2017, NSL-KDD, and IoTID20, to assess its generalization capability across diverse IoT environments.

Finally, the proposed intrusion detection framework can be integrated into real-time IoT systems and edge computing platforms, enabling continuous monitoring of network traffic and rapid detection of emerging cyber threats with reduced computational latency.

10. REFERENCES

[1] D. Akgun, S. Hizal, and U. Cavusoglu, "A new DDoS attacks intrusion detection model based on deep learning for cybersecurity," Computers & Security, vol. 118, p. 102748, Jul. 2022, doi: 10.1016/j.cose.2022.102748.

[2] A. D. Vibhute, C. H. Patil, A. V. Mane, and K. V. Kale, "Towards Detection of Network Anomalies using Machine Learning Algorithms on the NSL-KDD Benchmark Datasets," Procedia Computer Science, vol. 233, pp. 960–969, 2024, doi: 10.1016/j.procs.2024.03.285.

[3] S. Fraihat, S. Makhadmeh, M. Awad, M. A. Al-Betar, and A. Al-Redhaei, "Intrusion detection system for large-scale IoT NetFlow networks using machine learning with modified Arithmetic Optimization Algorithm," Internet of Things, vol. 22, p. 100819, Jul. 2023, doi: 10.1016/j.iot.2023.100819.

[4] M. Vishwakarma and N. Kesswani, "A new two-phase intrusion detection system with Naïve Bayes machine learning for data classification and elliptic envelop method for anomaly detection," Decision Analytics Journal, vol. 7, p. 100233, Jun. 2023, doi: 10.1016/j.dajour.2023.100233.

[5] I. Kerrakchou, A. Abou El Hassan, S. Chadli, M. Emharraf, and M. Saber, "Selection of efficient machine learning algorithm on Bot-IoT dataset for intrusion detection in internet of things networks," Indonesian Journal of Electrical Engineering and Computer Science, vol. 31, no. 3, pp. 1784–1793, Sep. 2023, doi: 10.11591/ijeecs.v31.i3.pp1784-1793.

[6] A. Aldaej, I. Ullah, T. A. Ahanger, and M. Atiquzzaman, "Ensemble technique of intrusion detection for IoT-edge platform," Scientific Reports, vol. 14, art. no. 12066, May 2024, doi: 10.1038/s41598-024-62435-w.

[7] M. Subramanian, K. Shanmugavadivel, P. S. Nandhini, and R. Sowmya, "DDoS Attack Detection Using Deep Learning and Bayesian Hyperparameter Optimization," in Proceedings of 7th International Conference on Harmony Search, Soft Computing and Applications, Lecture Notes on Data Engineering and Communications Technologies, vol. 140, Springer, 2022, pp. 395–404, doi: 10.1007/978-981-19-2948-9_38.

[8] X. Gao, C. Shan, C. Hu, Z. Niu, and Z. Liu, "An Adaptive Ensemble Machine Learning Model for Intrusion Detection," IEEE Access, vol. 7, pp. 82512–82521, 2019, doi: 10.1109/ACCESS.2019.2923640.

[9] X. Gao, C. Shan, C. Hu, and Z. Liu, "Anomaly Network Monitoring for Software-Defined Networking," in Proceedings of the 5th International Conference on Electronic Information Engineering and Computer Communication (EIECC), 2025, pp. 1–6.

[10] N. G. Pardeshi and D. V. Patil, "Binary and Multiclass Classification Intrusion Detection System using Benchmark NSL-KDD and Machine Learning Models," in Proceedings of the International Conference on Data Science and Network Security (ICDSNS), 2024, pp. 1–6.

[11] Amali, A. A. Sulaeman, A. M. Rifa'i, S. Raharjo, and E. Utami, "Detecting DDoS Attacks Through Decision Tree Analysis: An EDA Approach with the CIC DDoS 2019 Dataset," in Proceedings of the International Conference on Information Technology and Computer Science, 2024, pp. 1–6.

[12] A. Alsarhan, M. Barhoush, B. Khassawneh, M. Al-Essa, M. Aljaidi, and Q. Al-Na'amneh, "Deep Learning Utilization for DDoS Attack Detection with Federated Learning: A Case Study on the CICDDoS2019 Dataset," Engineering, Technology & Applied Science Research, vol. 16, no. 1, pp. 31203–31208, Feb. 2026, doi: 10.48084/etasr.14119.

[13] A. Hinojosa and N. E. Majd, "Edge Computing Network Intrusion Detection System in IoT Using Deep Learning," in Proceedings of the IEEE International Conference on Computer Science and Information Systems, 2024, pp. 1–6.

[14] A. Berguiga, A. Harchay, and A. Massaoudi, "HIDS-IoMT: A Deep Learning-Based Intelligent Intrusion Detection System for the Internet of Medical Things," IEEE Access, vol. 13, pp. 32145–32158, 2025, doi: 10.1109/ACCESS.2025.3543127.

[15] N. Hassouneh and S. al-sharaeh, "Intrusion Detection in IoT Networks using LSTM Deep Learning Models with the UNSW-NB15 Dataset," in Proceedings of the IEEE International Conference on Platform Technology and Service, 2024, pp. 1–6.

[16] H. Mliki, A. H. Kaceam, and L. Chaari, "Machine Learning for Attack Detection in IoT Networks," EAI Endorsed Transactions on Security and Safety, vol. 8, no. 29, pp. 1–12, 2023.

[17] J. Du, K. Yang, Y. Hu, and L. Jiang, "NIDS-CNNLSTM: Network Intrusion Detection Classification Model Based on Deep Learning," IEEE Access, vol. 11, pp. 24808–24821, 2023, doi: 10.1109/ACCESS.2023.3254915.

[18] T.-T.-H. Le, R. W. Wardhani, D. S. C. Putranto, U. Jo, and H. Kim, "Toward Enhanced Attack Detection and Explanation in Intrusion Detection System-Based IoT Environment Data," IEEE Access, vol. 11, pp. 131661–131676, 2023, doi: 10.1109/ACCESS.2023.3336678.

[19] U. C. Akuthota and L. Bhargava, "Transformer-Based Intrusion Detection for IoT Networks," IEEE Internet of Things Journal, vol. 12, no. 4, pp. 4432–4442, Feb. 2025, doi: 10.1109/JIOT.2025.3525494.

[20] J. Ashraf, G. M. Raza, B.-S. Kim, A. Wahid, and H.-Y. Kim, "Making a Real-Time IoT Network Intrusion-Detection System (INIDS) Using a Realistic BoT–IoT Dataset with Multiple Machine-Learning Classifiers," Applied Sciences, vol. 15, no. 4, art. no. 2043, Feb. 2025, doi: 10.3390/app15042043.

[21] M. Iwanowski, D. Olszewski, W. Graniszewski, J. Krupski, and F. Pelc, "The Choice of Training Data and the Generalizability of Machine Learning Models for Network Intrusion Detection Systems," Applied Sciences, vol. 15, no. 15, art. no. 8466, Aug. 2025, doi: 10.3390/app15158466.

[22] S. Yaras and M. Dener, "IoT-Based Intrusion Detection System Using New Hybrid Deep Learning Algorithm," Electronics, vol. 13, no. 6, art. no. 1053, Mar. 2024, doi: 10.3390/electronics13061053.

[23] S.-M. Tseng, Y.-Q. Wang, and Y.-C. Wang, "Multi-Class Intrusion Detection Based on Transformer for IoT Networks Using CIC-IoT-2023 Dataset," Future Internet, vol. 16, no. 8, art. no. 284, Aug. 2024, doi: 10.3390/fi16080284.

[24] S. Dhote and D. Agrawal, "A Comparative Machine Learning Framework for Network Intrusion Detection Using the NSL-KDD Dataset," International Journal of

IoT, Embedded Systems and Industrial Automation (IJIESIA), vol. 1, no. 2, art. no. e002, Jul.–Sep. 2026, doi: 10.66261/017smj60.

[25] D. Z. Rodríguez, O. D. Okey, S. S. Maidin, E. U. Udo, and J. H. Kleinschmidt, "Attentive transformer deep learning algorithm for intrusion detection on IoT systems using automatic Xplainable feature selection," PLoS ONE, vol. 18, no. 10, p. e0286652, Oct. 2023, doi: 10.1371/journal.pone.0286652.

[26] S. W. Nourildean, W. Mefteh, and A. M. Frihida, "Intrusion detection system-based ensemble machine learning to improve IoT network against cyber attacks," The Journal of Supercomputing, vol. 82, art. no. 370, 2026, doi: 10.1007/s11227-026-08529-y.

[27] A. Shebl, E. I. Elsedimy, A. Ismail, A. A. Salama, and M. Herajy, "DCNN: a novel binary and multi-class network intrusion detection model via deep convolutional neural network," EURASIP Journal on Information Security, vol. 2024, art. no. 36, Dec. 2024, doi: 10.1186/s13635-024-00184-1.

[28] P. K. Keserwani, M. C. Govil, E. S. Pilli, and P. Govil, "A smart anomaly-based intrusion detection system for the Internet of Things (IoT) network using GWO–PSO–RF model," Journal of Reliable Intelligent Environments, vol. 7, no. 1, pp. 3–21, Jan. 2021, doi: 10.1007/s40860-020-00126-x.

[29] F. S. Alsubaei, "Smart deep learning model for enhanced IoT intrusion detection," Scientific Reports, vol. 15, art. no. 6363, 2025, doi: 10.1038/s41598-025-06363-5.

[30] N. Dash, S. Chakravarty, A. K. Rath, N. C. Giri, K. M. AboRas, and N. Gowtham, "An optimized LSTM-based deep learning model for anomaly network intrusion detection," Scientific Reports, vol. 15, art. no. 1554, Jan. 2025, doi: 10.1038/s41598-025-85248-z.

[31] S. Kaushik, A. Bhardwaj, A. Almogren, S. Bharany, A. Altameem, A. U. Rehman, S. Hussen, and H. Hamam, "Efficient, lightweight cyber intrusion detection system for IoT ecosystems using mi2g algorithm," Scientific Reports, vol. 15, art. no. 88286, 2025, doi: 10.1038/s41598-025-88286-9.

[32] P. Waghmode, M. Kanumuri, H. El-Ocla, and T. Boyle, "Intrusion detection system based on machine learning using Least Square Support Vector Machine," Scientific Reports, vol. 15, art. no. 12066, Mar. 2025, doi: 10.1038/s41598-025-95621-7.

[33] S. Almahaqeri, M. H. Almourish, A. A. Nasser, A. S. A. Alghawli, A. A. K. Elsayed, and A. N. Alhejoj, "A multi-class classification model on CICIoT2023 using Kolmogorov–Arnold Networks and tree ensembles," Scientific Reports, vol. 16, art. no. 47399, Apr. 2026, doi: 10.1038/s41598-026-47399-5.

[34] A. Jyoti, M. Singh, V. K. Banga, J. Kaur, J. Gowrishankar, S. Hota, S. Murugan, A. Bhowmik, and N. Ashok, "Comparative performance evaluation of machine learning classifiers for multi-class intrusion detection on the NSL-KDD dataset," Scientific Reports, 2026, doi: 10.1038/s41598-026-59610-8.

[35] T. B. Ogunseyi and G. Thiyagarajan, "An Explainable LSTM-Based Intrusion Detection System Optimized by Firefly Algorithm for IoT Networks," Sensors, vol. 25, no. 7, art. no. 2288, Apr. 2025, doi: 10.3390/s25072288.